Beyond Quadratic Attention

The complete book of the project: what we built, why, how, what we found, and everything that went wrong along the way

A local research laboratory for attention algorithms
AMD Ryzen 5 7530U · CPU only · PyTorch 2.10 · October 2026

Contents

Chapter 1 — The whole project in one chapter

1.1 The problem in plain words
1.2 The one question
1.3 What was built
1.4 The machine
1.5 The most important findings
1.6 How the work proceeded
Key points to remember

Chapter 2 — The mathematics you need, from zero

2.1 Vectors: lists of numbers
2.2 The dot product: how similar are two vectors?
2.3 Matrices and matrix multiplication
2.4 Exponentials and logarithms
2.5 Softmax: turning scores into weights
2.6 Weighted averages
2.7 Probability and a few statistics
2.8 Derivatives and gradients: how learning works
2.9 Big-O notation: how cost grows
Key points to remember

Chapter 3 — How computers store and compute numbers

3.1 Bytes and sizes
3.2 The memory hierarchy: why moving data is expensive
3.3 Floating-point numbers
3.4 CPU cores, threads and parallelism
3.5 Python, C++ and the cost of "dispatch"
3.6 Tensors, autograd and memory in PyTorch
Key points to remember

Chapter 4 — Transformers and attention, from scratch

4.1 From text to vectors: tokens and embeddings
4.2 The Transformer in one picture
4.3 Queries, keys and values
4.4 A worked example by hand
4.5 Why divide by √d?
4.6 Multiple heads
4.7 Masks: forbidding some pairs
4.8 Where am I? Positional information
4.9 Exact versus approximate attention
Key points to remember

Chapter 5 — Why attention is "quadratic", and what that does and does not mean

5.1 Counting the work
5.2 How big the tables get
5.3 Four different kinds of "memory cost"
5.4 What each family of algorithms changes
5.5 Why "quadratic" is only the start of the story
Key points to remember

Chapter 6 — A guided tour of the codebase, and how the project started

6.1 Day one: the audit
6.2 The layout
6.3 One calling convention for everything

6.4 The registry
6.5 Where outputs go
Key points to remember

Chapter 7 — Implementation A: the reference ("naive") attention and the mask system

7.1 Why start with a slow implementation?
7.2 The code, step by step
7.3 The stable softmax and the detached maximum
7.4 The fully-masked row: choosing a policy
7.5 Masks are defined in exactly one place: MaskSpec
7.6 Input validation
7.7 How the reference itself is tested
Key points to remember

Chapter 8 — Implementation B: tiled exact attention and the online softmax

8.1 The problem
8.2 The idea: keep three running numbers and fix them up when the maximum changes
8.3 A worked example
8.4 Why it is exactly right (the proof)
8.5 The implementation
8.6 The backward pass: where tiling alone fails
8.7 The fix: recompute instead of store
8.8 What this implementation is not
Key points to remember

Chapter 9 — Implementation C: PyTorch's built-in kernels and how we found out which one runs

9.1 SDPA: one function, several implementations
9.2 Asking the profiler
9.3 The two adapters
9.4 What the flash CPU kernel does, and what it is not
9.5 How they compare (preview of Chapter 14)
Key points to remember

Chapter 10 — The approximate methods: local window, block sparse and linear attention

10.1 Two kinds of "approximate"
10.2 Local-window (sliding-window) attention
10.3 Block-sparse attention
10.4 Linear attention (elu + 1)
10.5 What each method costs, in theory
Key points to remember

Chapter 11 — The numerical-correctness laboratory

11.1 Three meanings of "the same"
11.2 The metrics
11.3 The protocol for one comparison
11.4 The tolerance policy, and how it got that way
11.5 Results of the full run
Key points to remember

Chapter 12 — The complexity model: theory you can test

12.1 Purpose
12.2 What it computes
12.3 Checking theory against the code
12.4 Checking theory against measured memory
12.5 Use in the memory guard
Key points to remember

Chapter 13 — The benchmarking system: measuring time and memory honestly
13.1 One configuration, step by step
13.2 Timing
13.3 Three memory instruments
13.4 The memory guard
13.5 Isolated workers
13.6 The scheduler
13.7 The shared-machine story
13.8 What a GPU benchmark would need
Key points to remember

Chapter 14 — Results I: the exact methods (correctness, memory, time)
14.1 They compute the same function
14.2 Memory: measured peak tensor allocation per call
14.3 Time: measured on one thread
14.4 How not to read these results
Key points to remember

Chapter 15 — Results II: approximate methods, tile sizes, threads, shapes and number formats
15.1 Approximate methods vs the fused exact kernel (E2)
15.2 The memory surprise
15.3 Tile size in the Python tiled kernels (E3)
15.4 Threads (E4)
15.5 Head dimension, batch and number format (E5)
Key points to remember

Chapter 16 — Results III: what a single attention operation can retrieve (probes)
16.1 The idea of a probe
16.2 Distance sweep (L = 1024)
16.3 Length sweep
16.4 Decoys: interference cuts both ways (L = 512, target 8 back)
16.5 The fairness check on linear attention
16.6 What probes do and do not show
Key points to remember

Chapter 17 — The tiny Transformer and how it was trained
17.1 The model
17.2 The training loop
17.3 The synthetic tasks
17.4 Lessons from training on a busy CPU
Key points to remember

Chapter 18 — Results IV: what models trained with each attention method can do
18.1 Task A: local copy (lag 3), 500 steps, trained at L = 64
18.2 Task B/C/D: variable-offset induction, 1500 steps, trained at L = 32
18.3 Summary of the quality picture
Key points to remember

Chapter 19 — The MQAR investigation: a detective story about a model that would not learn
19.1 The task
19.2 Act 1: the suspicious plateau (generator v1)
19.3 Act 2: stuck again, at a different number
19.4 Act 3: a systematic diagnosis
19.5 The verdict
19.6 Why the investigation stopped here
19.7 Lessons
Key points to remember

Chapter 20 — Operating the lab: install, run, explore, reproduce, extend
20.1 Requirements and setup
20.2 The commands
20.3 The web interface
20.4 Reading a report
20.5 Reproducing everything
20.6 Extending the lab
Key points to remember

Chapter 21 — The test suite: what is checked, and why it is trustworthy
21.1 Organization
21.2 What each file proves
21.3 Rules the tests follow
21.4 Problems solved to make the suite run
Key points to remember

Chapter 22 — The logbook: every bug, mistake, surprise and fix, in order
The ten most important lessons

Chapter 23 — Limitations, open questions and what to do next
23.1 What the results do not establish
23.2 Open questions
23.3 The most valuable next experiments
Key points to remember

Chapter 24 — Glossary

Chapter 25 — Self-test: know this project by heart
Part A — The project
Part B — The mathematics
Part C — Algorithms
Part D — Measurement
Part E — Results
Part F — The MQAR story
Part G — Operating

Chapter 26 — File-by-file index
Top level
src/attention_lab/algorithms/
Other library modules
scripts/
experiments/
docs/

Beyond Quadratic Attention

The complete book of the project: what we built, why, how, what we found, and everything that went wrong along the way

Written for TheOneAboveALL, and for anyone who wants to understand this project from zero.

Every number in this book was measured on this machine during the project. The raw data lives in `experiments/results/` *and the generated reports live in* `experiments/reports/`*. Wherever a number appears, the run it came from is named so that you can open the file and check it yourself.*

How to read this book

This book assumes **no prior knowledge**. It begins with what a vector is and ends with the details of how a fused attention kernel moves memory through a CPU cache. If you already know a topic, skip ahead; every chapter starts with a short "what you will learn" box and ends with a "key points to remember" box.

There are three kinds of text:

- **Explanations**, which teach an idea from first principles.
- **"In the code" boxes**, which point to the exact file and function that implements the idea, so you can open it and read along. File paths are relative to the project folder `C:\Users\omjee\OneDrive\Desktop\PROJECTS\Beyond Quadratic Attention`.
- **"What actually happened" boxes**, which tell the true story of the project: the mistakes, wrong turns, bugs and how each was found and fixed. These matter. Real research is mostly finding out why something does not behave as expected, and this project had many such moments. Chapter 22 collects all of them in one place.

A map of the book

Part	Chapters	What it covers
I. Foundations	1–5	The project in one chapter; the mathematics you need; how computers store and compute numbers; Transformers and attention from scratch; why attention is "quadratic"
II. What we built	6–10	A tour of the codebase; the reference implementation; tiled exact attention and online softmax; PyTorch's optimized kernels; the approximate methods
III. Measuring correctly	11–13	The numerical-correctness laboratory; the complexity model; the benchmarking system
IV. Experiments and results	14–16	Exact methods; approximate methods, tile sizes, threads and dtypes; what a single attention operation can retrieve
V. Models and quality	17–19	The tiny Transformer and its training; trained-task results; the MQAR investigation
VI. Operating the lab	20–21	Commands, interface, reports, reproduction; the test suite
VII. Reference	22–26	The complete bugs-and-fixes logbook; limitations and open questions; glossary; self-test questions; file index

If you only have one hour

Read Chapter 1 (the whole project in one chapter), Chapter 22 (the logbook) and Chapter 25 (self-test questions). You will then know what was done, what was found and what went wrong.

If you want to be able to defend this project in an interview

Read Chapters 4, 5, 8, 11, 13, 14 and 15 carefully, then answer every question in Chapter 25 without looking.

Chapter 1 — The whole project in one chapter

> **What you will learn:** what the project is for, the one question it asks, what was built, what was measured, the most important findings, and the shape of the rest of the book.

1.1 The problem in plain words

Modern AI language systems such as chatbots are built from a design called the **Transformer**. Its key ingredient is an operation called **attention**: every word (more precisely, every *token*) in a piece of text looks at every other token and decides how much to "pay attention" to it.

"Every token looks at every other token" is the problem. If a text has 1,000 tokens, there are 1,000 × 1,000 = one million token pairs. With 10,000 tokens there are 100 million pairs, a hundred times more work for only ten times more text. Mathematicians call this growth **quadratic**: double the length and the work quadruples. That is why long documents are expensive for AI systems, and why researchers have invented many cleverer ways to compute attention.

This project is a **laboratory** for studying those ways. Not a single program that does one thing, but a collection of tools to implement, verify, measure and compare different attention algorithms on an ordinary laptop.

1.2 The one question

The project brief stated the central question like this:

> *How do attention algorithms trade off computational complexity, memory traffic, numerical accuracy, and practical execution speed as sequence length increases?*

Each phrase in that sentence turned into a part of the project:

Phrase	What we built to study it
"computational complexity"	an analytical model that counts operations and memory for each algorithm (Chapter 12)
"memory traffic"	memory instruments that measure how many bytes each algorithm actually allocates (Chapter 13)
"numerical accuracy"	a correctness laboratory comparing every method with a high-precision reference (Chapter 11)
"practical execution speed"	a benchmarking system with careful timing and background-load monitoring (Chapter 13)
"as sequence length increases"	sweeps over lengths from 64 to 16,384 tokens (Chapters 14–15)

A second question came from the fact that some algorithms do not compute exactly the same thing as standard attention. They *approximate* or *restrict* it. For those, we also asked: *what do you lose?* That led to the quality experiments (Chapters 16 and 18).

1.3 What was built

Everything lives in one folder and runs locally. Nothing is sent to the internet.

Eight attention implementations (Chapters 7–10):

Name	Family	One-line description
`naive`	exact	the textbook formula, written plainly; the "ground truth" everything else is compared with
`sdpa_math`	exact	PyTorch's built-in attention, forced to use its plain "math" path
`sdpa_flash_cpu`	exact	PyTorch's built-in attention, forced to use its fused, tiled C++ kernel for CPUs
`tiled`	exact	our own implementation of the *online softmax* / tiling idea behind FlashAttention, written in Python
`tiled_recompute`	exact	the same, plus a custom backward pass that recomputes instead of storing; this is the trick that saves training memory
`local_window`	restricted	each token may only look at nearby tokens
`block_sparse`	restricted	tokens are grouped into blocks, and each block may look only at selected blocks
`linear_elu`	different function	"linear attention", which replaces the exponential in softmax with a simpler formula that can be reorganized to avoid the quadratic cost

Tools around them:

- a **numerical-correctness laboratory**: 795 comparisons of every exact method against the reference, in three number formats, with an explicit and justified tolerance policy;
- a **complexity model**: formulas for operations and memory per method, checked against counters built into the algorithms;
- a **benchmarking system**: repeated timing with statistics, three different memory instruments, a memory guard that refuses to run configurations that would exhaust RAM, isolated worker processes, and a monitor that records how busy *other* programs kept the CPU;
- **controlled tasks**: hand-built "retrieval probes" and small Transformer models trained on synthetic tasks, to measure what the approximate methods lose;
- a **command-line interface** (`python lab.py ...`), a **local web interface** (Streamlit), automatic **reports** with tables and figures, and a **test suite** of more than 400 tests.

1.4 The machine

The brief described an AMD Ryzen 5 5300U. The actual machine, checked on day one, has an **AMD Ryzen 5 7530U**: 6 cores, 12 hardware threads, AVX2 vector instructions, a 16 MB L3 cache, 16 GB of RAM, Windows 11, Python 3.13.3 and PyTorch 2.10 (CPU build). There is no NVIDIA graphics card, so **every result in this book is a CPU result**. GPUs behave differently, and Chapter 13 explains how.

A fact that shaped the whole project: **the computer was shared.** Throughout the work, other programs from other projects kept between 5 and all 12 logical CPUs busy, and free memory was often under 1 GB. The project had to learn to measure honestly under those conditions (Chapters 13 and 22).

1.5 The most important findings

Each of these is explained in detail later, with its evidence.

1. **All exact methods compute the same function.** In 64-bit floating point they agree with the reference to about 10^{-15}. In 32-bit, every method passed every forward and gradient check (Chapter

14).
2. **How you store intermediate numbers matters more than the algorithm.** Computing attention entirely in the 16-bit `bfloat16` format gave errors up to 38 %, while the kernels that keep intermediate values in 32-bit stayed at 0.3 % (Chapter 11).
3. **Tiling removes the quadratic *memory*, but not the quadratic *time*.** At 2,048 tokens the reference allocates 194 MiB per call, while the tiled and fused methods allocate under 5 MiB. Yet every exact method's time still grows like length squared (fitted exponents 1.9–2.1) (Chapter 14).
4. **Tiling alone does not fix training memory.** Letting PyTorch's automatic differentiation run through the tiled loop brings the quadratic memory back (exponent 1.72). A custom "recompute" backward pass keeps it linear, and was also **faster**: 0.98 s against 1.85 s for the autograd version at 2,048 tokens (Chapter 14).
5. **The same algorithm can be 3× faster or slower depending on how it is executed.** PyTorch's fused C++ kernel and our Python tiled code implement the same mathematics, but the fused one is about 3× faster (Chapter 14).
6. **Approximate methods are faster only beyond a crossover length**, about 2,048 tokens for local-window and block-sparse attention on this machine. Below it they are slower than the fused exact kernel (Chapter 15).
7. **Surprise: the approximate methods used *more* memory than the fused exact kernel**, 16–27× more at 4,096 tokens, because they store their restricted score tables while the fused kernel stores almost nothing (Chapter 15).
8. **Restriction has predictable costs and sometimes benefits.** Local attention cannot retrieve anything beyond its window (accuracy drops from 100 % to 0 % exactly at the window edge). Yet on a task needing only local information, it generalized to 4× longer texts *better* than full attention (100 % vs 87 %) (Chapters 16 and 18).
9. **A model that "learned" a task may have learned a shortcut.** Twice in this project a model appeared to solve a retrieval task but had found a shortcut: once in the data generator, once through fixed positions. Finding those shortcuts is told in Chapter 19.

1.6 How the work proceeded

Roughly in this order (the true, messy history is in Chapter 22):

1. **Audit:** inspect the empty folder, the hardware and the installed software.
2. **Reference implementation** and its tests.
3. **Tiled exact attention** (online softmax), then the recompute backward.
4. **Numerical validation** across shapes, masks, magnitudes and number formats.
5. **Approximate methods.**
6. **Benchmarking** system and **complexity** model.
7. **Experiments:** correctness, memory scaling, timing (repeated after the background-load problem was discovered), approximate-method scaling, tile sizes, threads, shapes and dtypes.
8. **Quality:** probes, then trained models, then the long MQAR investigation.
9. **Interface, reports and documentation**, followed by a final audit.

Key points to remember

- The project is a *laboratory* for comparing attention algorithms on speed, memory, accuracy and quality.
- Eight implementations: five exact, two restricted, one different-function.

- Everything was measured on a shared, CPU-only laptop, and the project measures and reports that load.
- The headline: memory efficiency, speed and accuracy are *different* things. An algorithm can win one and lose another, and the winner depends on length, threads and hardware.

Chapter 2 — The mathematics you need, from zero

> **What you will learn:** numbers in lists (vectors) and tables (matrices); the dot product; matrix multiplication; exponentials and logarithms; the softmax function; what a gradient is and why training needs it; and "big-O" notation for describing how cost grows. Everything else in the book builds on these ideas.

2.1 Vectors: lists of numbers

A **vector** is an ordered list of numbers, for example `[2, -1, 0.5]`. The number of entries is its **dimension**; this one has dimension 3. In AI, a vector is how we represent *a thing*: a word, an image patch, a position. Each entry captures some aspect of it, though usually no single entry has an obvious human meaning.

In this project, each token is represented by vectors of dimension **d** (we mostly used `d = 64`).

2.2 The dot product: how similar are two vectors?

The **dot product** of two vectors of the same dimension multiplies matching entries and adds the results:

```
a = [1, 2, 3]
b = [4, 0, -1]
a · b = 1×4 + 2×0 + 3×(−1) = 4 + 0 − 3 = 1
```

Why it matters: when two vectors "point in the same direction" their dot product is large and positive. When they point in opposite directions it is large and negative. When they are unrelated it is near zero. **Attention uses the dot product to measure how relevant one token is to another.**

Computing one dot product of dimension `d` costs `d` multiplications and `d − 1` additions. In this book one "multiply-add" counts as **2 floating-point operations (FLOPs)**, so a dot product costs about `2d` FLOPs.

2.3 Matrices and matrix multiplication

A **matrix** is a rectangular table of numbers with *rows* and *columns*. A matrix with `L` rows and `d` columns is called "`L × d`". You can think of it as `L` vectors stacked on top of each other, one per row.

Matrix multiplication `C = A × B` (requires `A` to be `m × k` and `B` to be `k × n`) produces an `m × n` matrix where **each entry `C[i][j]` is the dot product of row `i` of `A` with column `j` of `B`**:

```
A (2×3) = [[1, 2, 3],        B (3×2) = [[1, 0],
           [0, 1, 0]]                   [0, 1],
                                        [1, 1]]

C = A × B (2×2):
C[0][0] = 1·1 + 2·0 + 3·1 = 4      C[0][1] = 1·0 + 2·1 + 3·1 = 5
C[1][0] = 0·1 + 1·0 + 0·1 = 0      C[1][1] = 0·0 + 1·1 + 0·1 = 1
```

Cost: `m × n` dot products of length `k`, so about **`2·m·n·k` FLOPs**. This formula is used throughout the book.

The **transpose** of a matrix, written B^T, swaps rows and columns: an `L × d` matrix becomes `d × L`. If `Q` and `K` are both `L × d`, then `Q × K`^{T} is `L × L`, and entry `[i][j]` is the dot product of row `i` of `Q` with row `j` of `K`. **That single matrix product computes the similarity of every token with every other token, and its `L × L` shape is where the "quadratic" in this project's title comes from.**

Tensors

A **tensor** is the generalization to any number of dimensions: a vector is a 1-D tensor, a matrix is 2-D, and so on. Attention in this project uses 4-D tensors shaped `[B, H, L, d]`:

- `B`: **batch**, how many independent texts are processed together;
- `H`: **heads**, how many independent attention computations run side by side (Chapter 4);
- `L`: **length**, how many tokens per text;
- `d`: **head dimension**, the length of each token's vector.

2.4 Exponentials and logarithms

The **exponential** `exp(x) = e`^{x} (with `e ≈ 2.71828`) turns any number into a positive number: `exp(0) = 1`, `exp(1) ≈ 2.718`, `exp(−1) ≈ 0.368`, `exp(10) ≈ 22,026`, `exp(−10) ≈ 0.0000454`. Two properties matter here:

1. It is **always positive**, which makes it useful for building weights.
2. It **grows explosively**. A difference of 10 in the input is a factor of about 22,000 in the output. That is exactly what lets attention focus sharply on one item. It is also what causes overflow: `exp(89)` is already too large for a 32-bit float (Chapter 3).

A key identity: **`exp(a − b) = exp(a) / exp(b)`**. Online softmax (Chapter 8) is built on it.

The **logarithm** `log(x)` is the inverse: `log(exp(x)) = x`. The quantity `log(Σ exp(`x_i`))` is called **log-sum-exp** (LSE). It appears in Chapter 8 as the single number per row that the memory-efficient backward pass stores.

2.5 Softmax: turning scores into weights

Given a list of scores $x_1, \ldots, x_n$, the **softmax** turns them into weights that are positive and sum to 1:

$$\text{softmax}(x)_i = \exp(x_i) / (\exp(x_1) + \exp(x_2) + \ldots + \exp(x_n))$$

Worked example, scores `[2, 1, 0]`:

```
exp(2) ≈ 7.389,  exp(1) ≈ 2.718,  exp(0) = 1        total ≈ 11.107
weights ≈ [7.389/11.107, 2.718/11.107, 1/11.107] ≈ [0.665, 0.245, 0.090]
```

The largest score gets most of the weight, but not all of it: softmax is a "soft" version of picking the maximum. Scale the scores up and it becomes harder: `[20, 10, 0]` gives about `[0.99995, 0.00005, 0]`.

The shift trick. Adding the same number `c` to every score does not change the softmax, because `exp(`x_i `+ c) = exp(`x_i`)·exp(c)` and the `exp(c)` cancels between top and bottom. Programs therefore subtract the

maximum score before exponentiating, so the largest exponent is `exp(0) = 1` and nothing overflows. This is called the **stable softmax**, and every implementation in the project uses it (Chapters 7 and 8).

2.6 Weighted averages

If weights $w_1 \dots w_n$ are non-negative and sum to 1, then $w_1 \cdot v_1 + \dots + w_n \cdot v_n$ is a **weighted average** (also called a *convex combination*) of the vectors v_i. Attention's output for each token is exactly this: a weighted average of "value" vectors, with softmax weights. A consequence used in Chapter 11 is that **an attention output can never be larger than the largest value it averages**.

2.7 Probability and a few statistics

- A **probability distribution** over `n` outcomes is `n` non-negative numbers summing to 1, which is exactly what softmax produces.
- **Uniform guessing** among `n` options is right with probability `1/n`. For example, chance level for 16 possible answers is 6.25 %. That number appears in Chapter 19.
- The **median** of repeated measurements is the middle value after sorting. It is robust to a few extreme values, which is why our timing reports use it (Chapter 13).
- The **inter-quartile range (IQR)** is the spread between the 25th and 75th percentiles, a robust measure of variability.
- **Cross-entropy loss** measures how much probability a model gave to the correct answer: `loss = -log(p_correct)`. If a model spreads probability evenly over `n` answers, `p_correct = 1/n` and `loss = log(n)`. So a training loss stuck at `log(4) ≈ 1.386` means "uniform over 4 options", a fact that was crucial in the MQAR investigation (Chapter 19).

2.8 Derivatives and gradients: how learning works

A **derivative** says how much a function's output changes when its input changes a little. If $f(x) = x^2$, its derivative is $2x$: at $x = 3$, nudging x by 0.001 changes f by about 0.006.

For a function of many inputs, the list of derivatives with respect to each input is the **gradient**. A neural network is a big function with millions of adjustable numbers (**parameters**). **Training** repeatedly does three things:

1. compute the loss (how wrong the network is);
2. compute the gradient of the loss with respect to every parameter. This step is called the **backward pass**, or *backpropagation*, while computing the output is the **forward pass**;
3. nudge every parameter a little in the direction that reduces the loss (an **optimizer** such as AdamW decides exactly how).

Two things to remember for this project:

- The backward pass usually needs values computed during the forward pass. Keeping those values in memory is what makes **training** use much more memory than **inference** (just computing outputs). This is central to Chapter 8.
- The **chain rule** combines derivatives of nested functions. PyTorch's **autograd** applies it automatically: it records every operation in the forward pass, then replays them backward. Writing a **custom backward** means doing this by hand, which `tiled_recompute` does.

2.9 Big-O notation: how cost grows

"Big-O" describes how a cost grows as the input gets larger, ignoring constant factors:

Notation	Name	Doubling the input multiplies the cost by
`O(L)`	linear	2
`O(L log L)`	"n log n"	a bit more than 2
`O(L²)`	quadratic	4
`O(L³)`	cubic	8

Standard attention costs `O(L²·d)` time: the `L × L` table of dot products, each costing `O(d)`.

Measuring the exponent. If time `t` grows like `L^p`, then `log t = p·log L + constant`, so a plot of `log t` against `log L` is a straight line with slope `p`. The project fits that slope to measured data (`reporting/report.py::fit_exponent`). A fitted exponent near 2 means "quadratic in the measured range", and near 1 means "linear". This is how theory and measurement are connected in Chapters 14 and 15.

Big-O hides constants, and constants decide real speed. An `O(L)` method that does 100 operations per token loses to an `O(L²)` method doing 0.01 operations per pair until `L > 10,000`. Chapter 15 measures such a "crossover" on this machine.

Key points to remember

- Dot product = similarity. `Q × Kᵀ` = all pairwise similarities = an `L × L` table.
- Matrix multiply cost: `2·m·n·k` FLOPs.
- Softmax turns scores into positive weights summing to 1, and it is unchanged by subtracting the max.
- Attention output = weighted average of value vectors.
- Loss stuck at `log(n)` = uniform guessing among `n` options.
- Training needs the backward pass, which needs stored forward values, which needs memory.
- Fitted log-log slope ≈ growth exponent: 2 = quadratic, 1 = linear.

Chapter 3 — How computers store and compute numbers

> **What you will learn:** bytes and memory sizes; the memory hierarchy (registers, caches, RAM) and why moving data often costs more than computing with it; floating-point numbers and rounding; the three number formats used in the project (fp64, fp32, bf16); CPU cores, threads and parallelism; and what Python, PyTorch, tensors and autograd are.

3.1 Bytes and sizes

A **byte** is 8 bits. Memory sizes in this book use binary units: 1 KiB = 1,024 bytes, 1 MiB = 1,048,576 bytes (about one million), 1 GiB ≈ one billion bytes. "MB" and "GB" are used loosely for the same ideas.

A 32-bit floating-point number (fp32) occupies **4 bytes**. So an attention score table for one head with `L = 1024` tokens holds `1024 × 1024 = 1,048,576` numbers, which is **4 MiB**. With 4 heads that is 16 MiB. At `L = 4096` it is 64 MiB per head and 256 MiB for 4 heads. Keep these sizes in mind; they explain most of Chapter 14.

3.2 The memory hierarchy: why moving data is expensive

A CPU does arithmetic on numbers held in tiny storage slots called **registers**. Data must be brought there from main memory (**RAM**), which is large but slow, through a series of **caches**, which are progressively smaller and faster:

Level	Size on this machine	Rough speed
Registers	a few KB per core	instant
L1 cache	32 KB data per core (384 KB total)	~1 ns
L2 cache	512 KB per core (3 MB total)	a few ns
L3 cache	**16 MB shared**	~10 ns
RAM	16 GB	~100 ns and limited bandwidth

(The sizes were measured on this machine with Windows' `Win32_CacheMemory`; the speeds are typical orders of magnitude, not measurements.)

The crucial consequence: **an operation that does little arithmetic per byte is limited by how fast data can be moved, not by how fast the CPU computes.** This is called being **memory-bandwidth-bound**. Adding up a large table, or applying `exp` to every entry, is memory-bound. Multiplying two large matrices is **compute-bound**, because each number fetched is reused many times.

Naive attention creates `L × L` tables and makes several passes over them (scale, mask, subtract max, exp, sum, divide). Once a table exceeds the 16 MB L3 cache (at `L = 1024` with 4 heads), every pass goes to RAM. Much of the speed of an optimized attention kernel comes from **never writing that table to RAM at all** (Chapter 9).

3.3 Floating-point numbers

Computers store real numbers in **floating-point** format, a binary version of scientific notation: `value = sign × mantissa × 2^exponent`. The **mantissa** (significant digits) has a fixed number of bits, so most real numbers can only be stored **approximately**: they are **rounded** to the nearest representable value.

The size of the rounding is described by the **unit roundoff** `u`: the relative error of a single rounding is at most about `u`.

Format	Bits	Mantissa bits	Unit roundoff u	Largest value	Used here for
fp64 ("double")	64	52	1.1×10^{-16}	$\sim 10^{308}$	the reference ("ground truth")
fp32 ("float")	32	23	6.0×10^{-8}	$\sim 3.4 \times 10^{38}$	default for all benchmarks
bf16 ("bfloat16")	16	7	3.9×10^{-3}	$\sim 3.4 \times 10^{38}$	low-precision experiments
fp16 ("half")	16	10	4.9×10^{-4}	65,504	tolerance defined, not benchmarked

Three consequences matter in this project:

1. **Overflow.** `exp(x)` exceeds fp32's maximum when `x > 88.7`, and becomes `inf` (infinity). Then `inf / inf = NaN` ("not a number"). The stable softmax (Chapter 2) prevents this.
2. **Absolute precision shrinks as numbers grow.** bf16 has only 8 significant bits, so near 256 the gap between neighbouring representable numbers is about 1. A logit of 300.4 might be stored as 300. For softmax that is a disaster: an error of 0.5 in a logit multiplies its weight by `e^0.5 ≈ 1.65`. This is the cause of the biggest numerical finding in the project (Chapter 11).
3. **Addition is not associative.** `(a + b) + c` can differ from `a + (b + c)` in the last bits. Two correct programs that sum in different orders therefore give slightly different answers. That is why we compare with **tolerances**, not exact equality (Chapter 11).

Special values: `inf` and `-inf` (overflow, or deliberately used for masking: `exp(-inf) = 0`), and `NaN`, which poisons every computation it touches.

3.4 CPU cores, threads and parallelism

A **core** is an independent processor. This machine has **6 physical cores**. Each can run **2 hardware threads** (*simultaneous multithreading*, SMT), giving **12 logical CPUs**, but two threads on one core share its arithmetic units, so they rarely give 2× the speed.

PyTorch splits large operations across **intra-op threads** using a library called **OpenMP**: a matrix multiply might be divided into 6 pieces computed at the same time. Coordination costs time (starting the pieces, waiting for the slowest one, a *barrier*). For large operations the overhead is negligible. For tiny operations it can exceed the useful work. If another program is using a core, the piece running there is delayed and **every other thread waits at the barrier**. Chapter 15 shows this happening: the Python tiled code became 4.8× *slower* with 6 threads.

`torch.set_num_threads(n)` sets the thread count in our code. The environment variables `OMP_NUM_THREADS` and `MKL_NUM_THREADS` set it before PyTorch starts.

Vector instructions (SIMD). A single CPU instruction can operate on several numbers at once. This CPU supports **AVX2** (256-bit registers: 8 fp32 numbers per instruction) but not AVX-512, and it has **no native bf16 arithmetic**. That is why bf16 is not faster than fp32 here (Chapter 15).

3.5 Python, C++ and the cost of "dispatch"

Python is the language this project is written in. It is easy to read but slow at doing many small things, because every line is interpreted. **C++** is compiled to machine code and is fast.

PyTorch is a Python library whose heavy work is done in C++ (and specialized libraries: **MKL** and **oneDNN** for matrix math on Intel/AMD CPUs). When Python calls a PyTorch operation such as `a @ b`, PyTorch must:

1. parse the arguments and check types, shapes and devices;
2. choose the right C++ implementation (the **dispatcher**);
3. allocate memory for the result;
4. record the operation for autograd, if gradients are needed;
5. run the C++ code.

Steps 1–4 cost a few **microseconds**, regardless of how big the operation is. For one huge matrix multiply that is negligible. For 10,000 tiny ones it is 10–50 milliseconds of pure overhead. This **dispatch overhead** explains why our Python tiled attention, which issues roughly 10 small operations per tile, is slower than the C++ fused kernel that implements the same algorithm (Chapters 8, 14 and 15).

3.6 Tensors, autograd and memory in PyTorch

A PyTorch **tensor** is an n-dimensional array with a data type (`torch.float32`, ...) and a device (here always the CPU). Some operations return **views**, which reuse the same memory with a different shape (for example `transpose`, `expand`, `unfold`). Others allocate new memory. The project relies on views to avoid building large masks (Chapter 7).

Autograd: if a tensor has `requires_grad=True`, PyTorch records every operation applied to it in a graph and keeps the intermediate tensors the backward pass will need. `loss.backward()` walks the graph backward and fills each parameter's `.grad`. The kept intermediates are why training memory is larger than inference memory, and why "letting autograd differentiate through a tiled loop" keeps all the tiles alive (Chapter 8).

`torch.inference_mode()` and `torch.no_grad()` switch recording off. Our forward-only benchmarks use inference mode, so they measure the computation without autograd bookkeeping.

The CPU allocator. PyTorch requests memory for each new tensor from its CPU allocator. The profiler can record every allocation and free, with sizes and timestamps. The project replays those events to measure the peak memory of a call (Chapter 13).

Key points to remember

- One fp32 number = 4 bytes. One `L × L` fp32 table at `L = 1024` = 4 MiB; with 4 heads = 16 MiB = the whole L3 cache.
- Moving data is often the real cost, so avoiding large intermediate tables is the main lever for speed.
- Floating point rounds. `u` is fp64 1e-16, fp32 6e-8, bf16 4e-3. `exp` overflows fp32 above 88.7.
- bf16's coarse spacing at large magnitudes ruins softmax logits unless they are kept in fp32.
- Python dispatch costs microseconds per operation, so many tiny ops are slow.
- More threads help large operations and hurt tiny ones, especially on a busy machine.
- Autograd keeps forward intermediates alive for backward: that is training memory.

Chapter 4 — Transformers and attention, from scratch

> **What you will learn:** what tokens and embeddings are; what a Transformer is made of; exactly how attention computes its output, worked through by hand with real numbers; why the scores are divided by √d; multiple heads; masks (causal, padding, custom); and how a model knows *where* each token is (positional encodings, including RoPE, which this project uses).

4.1 From text to vectors: tokens and embeddings

A language model cannot read letters directly. Text is first cut into **tokens** (words or word pieces), and each token is replaced by an integer ID from a fixed **vocabulary**. In this project the synthetic tasks use a vocabulary of 64 IDs (Chapter 17), for example "ID 9 = a key symbol" and "ID 25 = a value symbol".

Each ID is then turned into a vector by an **embedding** table: a learned matrix with one row per vocabulary entry. Token ID 9 becomes row 9, a vector of `d_model` numbers (64 in our models). A sequence of `L` tokens becomes an `L × d_model` matrix.

4.2 The Transformer in one picture

A (decoder-only) Transformer is a stack of identical **blocks**. Each block has two parts, each wrapped in a "residual connection" (the input is added back to the output) and preceded by a **LayerNorm**, which rescales each token vector to a standard size:

```
x  →  x + Attention(LayerNorm(x))      ← tokens exchange information
   →  x + MLP(LayerNorm(x))            ← each token is processed on its own
```

After the last block, a final LayerNorm and a linear "head" turn each token vector into scores over the vocabulary: the model's prediction. The **MLP** (multi-layer perceptron) is two matrix multiplies with a nonlinearity (GELU) between them. It processes each token independently. **Attention is the only place where tokens interact.** That is why attention gets all the attention, and why its cost dominates long texts.

> **In the code:** `src/attention_lab/models/transformer.py` — `TinyTransformer`, `Block`, `Attention`. The `Attention` module computes Q, K and V, then calls whichever attention algorithm you choose by name. This "plug-in" design (Chapter 6) lets the same model run with any of the eight implementations.

4.3 Queries, keys and values

For each token, attention computes three vectors by multiplying its representation with three learned matrices:

- **Query (q)**: "what am I looking for?"

- **Key (k)**: "what do I contain?"
- **Value (v)**: "what will I hand over if you pick me?"

For a sequence, these form three matrices `Q`, `K`, `V`, each of shape `L × d` (one row per token). Then:

```
S = Q Kᵀ · (1/√d)          scores: how relevant is token j to token i?        (L × L)
P = softmax(S, per row)    attention weights: each row sums to 1                (L × L)
O = P V                    output: each token's weighted average of values      (L × d)
```

In one line, as in the original Transformer paper (Vaswani et al., 2017): **Attention(Q, K, V) = softmax(QKᵀ/√d) V.**

4.4 A worked example by hand

Three tokens, `d = 2`. (The numbers are invented for illustration, then checked with the project's own `naive_attention`.)

```
Q = [[1, 0],      K = [[2, 0],      V = [[1, 0],
     [0, 2],           [0, 1],           [0, 1],
     [1, 1]]           [1, 1]]           [5, 5]]
```

Step 1: scores `S = Q Kᵀ`. Entry `[i][j]` is query `i` dotted with key `j`:

```
token 0: q=[1,0]: with k0=[2,0] → 2;  with k1=[0,1] → 0;  with k2=[1,1] → 1
token 1: q=[0,2]: → 0, 2, 2
token 2: q=[1,1]: → 2, 1, 2

S = [[2, 0, 1],
     [0, 2, 2],
     [2, 1, 2]]
```

Step 2: scale by `1/√d = 1/√2 ≈ 0.7071`:

```
[[1.4142, 0.0000, 0.7071],
 [0.0000, 1.4142, 1.4142],
 [1.4142, 0.7071, 1.4142]]
```

Step 3: softmax each row. Row 0: `exp(1.4142) ≈ 4.113`, `exp(0) = 1`, `exp(0.7071) ≈ 2.028`; total ≈ 7.141, so the weights are `[0.576, 0.140, 0.284]`. Doing all rows:

```
P = [[0.5760, 0.1400, 0.2840],
     [0.1084, 0.4458, 0.4458],
     [0.4011, 0.1978, 0.4011]]
```

Every row sums to 1. Token 0 pays most attention to token 0. Token 1 splits equally between tokens 1 and 2.

Step 4: output `O = P V`. Each output row is a weighted average of the rows of `V`:

```
token 0: 0.576·[1,0] + 0.140·[0,1] + 0.284·[5,5] = [1.996, 1.560]
token 1: = [2.337, 2.675]
token 2: = [2.407, 2.203]
```

The project's `naive_attention` returns `[1.99595, 1.56001]` for token 0, matching to rounding.

4.5 Why divide by √d?

If the entries of `q` and `k` are random with mean 0 and variance 1, their dot product (a sum of `d` products) has variance `d`. With `d = 64` the typical score would be about ±8, and the softmax would almost always put nearly all weight on one token. Its gradients would then be tiny, and learning would stall. Dividing by `√d` brings the variance back to 1 (Vaswani et al., 2017, footnote 4). The project's default is `scale = d^(-1/2)`, and a test checks it (`test_scale_is_one_over_sqrt_d`).

4.6 Multiple heads

Instead of one attention with vectors of size `d_model`, the model splits the vectors into `H` **heads** of size `d = d_model / H` and runs `H` independent attentions in parallel. Each head can specialize: one may look at the previous token, another for matching content. The outputs are concatenated and mixed by a final linear layer. All of this is why our tensors are 4-D `[B, H, L, d]`: batch × heads × length × head-dimension.

4.7 Masks: forbidding some pairs

Often some query–key pairs must not interact. A mask removes them by setting their score to `−∞` before the softmax, because `exp(−∞) = 0` gives them zero weight.

- **Causal mask.** A model that predicts the next token must not peek at future tokens: token `i` may attend only to `j ≤ i`. In the example above the causal weights become `[[1, 0, 0], [0.196, 0.804, 0], [0.401, 0.198, 0.401]]`: token 0 can only see itself, so its output is exactly `v0 = [1, 0]`.
- **Padding mask.** When texts of different lengths are batched together, short ones are padded with filler tokens that must be ignored. Our `key_padding_mask` has shape `[B, L]`, with `True` meaning "real token".
- **Custom masks.** Any boolean table (`True` = allowed) or an additive table of numbers (added to the scores, `−∞` removes) can be supplied.

The project defines the semantics of all masks **in one place** (`algorithms/base.py`, class `MaskSpec`), matching PyTorch's own convention (Chapter 7).

Edge case: a fully-masked row. If every key is forbidden for some query, the softmax is `0/0`, which is undefined. The project's policy is **output zero and zero gradient**, chosen after checking that this is what PyTorch's own CPU kernels do. The story is in Chapter 7.

4.8 Where am I? Positional information

Attention by itself ignores order: shuffle the tokens and each output is the same weighted average, just shuffled. Models therefore add **positional information**:

- **Sinusoidal absolute encodings** add a fixed pattern of sines and cosines, different for each position, to the embeddings (Vaswani et al., 2017).
- **Rotary position embedding (RoPE)** rotates each query and key vector by an angle proportional to its position (Su et al., 2021). The dot product of a rotated query at position `i` and a rotated key at position `j` then depends only on the **difference** `i − j`. The model therefore naturally sees *relative*

positions. Our models use RoPE by default, and a test checks the relative-position property (`test_rope_scores_depend_only_on_relative_position`).

RoPE is applied **before** the attention algorithm is called, so every attention implementation receives the same Q and K. Keep RoPE's relative-position nature in mind: in Chapter 19 it is exactly what allowed a model to solve a supposedly content-based task with a purely positional trick.

4.9 Exact versus approximate attention

Everything above is **exact attention**: every allowed pair is scored, and the softmax is computed exactly (up to rounding). The project also studies methods that change the computation:

- **Restricted attention** (local window, block sparse) still uses the exact softmax but forbids many pairs, as if a fixed mask were applied.
- **Linear attention** replaces `exp(q·k)` with a different similarity that can be computed faster. It is not softmax attention at all, just a related function.

These are different research questions. For exact methods we ask "do they compute the same thing, and how fast?" For approximate methods we ask "how fast, and what do we lose?" (Chapter 10).

Key points to remember

- Attention(Q, K, V) = softmax($QK^T/\sqrt{d}$)V; each output is a weighted average of value vectors.
- `QKᵀ` is an `L × L` table, the source of the quadratic cost.
- Divide by √d so that scores have unit variance.
- Masks set forbidden scores to `−∞`. Causal: `j ≤ i`. Fully-masked rows output 0 by project policy.
- Our tensors are `[B, H, L, d]`. Heads are independent attentions.
- RoPE makes attention depend on relative position. That helps the model, and can also create shortcuts.

Chapter 5 — Why attention is "quadratic", and what that does and does not mean

> **What you will learn:** exactly which parts of attention grow with L^2, by counting; how big the score tables get; the difference between time, auxiliary memory, training memory and memory traffic; what each algorithm family changes; and why "quadratic" is only the start of the story.

5.1 Counting the work

For one head with `L` tokens and head dimension `d`:

Step	What it computes	Size of result	Arithmetic
`S = Q Kᵀ`	`L × L` dot products of length `d`	`L²` numbers	`2·L²·d` FLOPs
scale, mask, max, exp, sum, divide	elementwise on `S`	`L²` numbers each	about `4·L²` operations
`O = P V`	`L` weighted averages over `L` values of length `d`	`L·d` numbers	`2·L²·d` FLOPs

Total ≈ `4·L²·d` **multiply-add FLOPs per head**, plus `O(L²)` cheaper elementwise operations. With a batch of `B` texts and `H` heads, multiply by `B·H`. In big-O: **time** `O(B·H·L²·d)`.

The inputs and outputs (`Q`, `K`, `V`, `O`) hold only `4·B·H·L·d` numbers, which is *linear* in `L`. The quadratic part is the **intermediate** score and probability tables.

5.2 How big the tables get

The project's complexity model (`python lab.py complexity`, labeled THEORETICAL) gives, for `B = 1`, `H = 4`, `d = 64`, fp32:

L	logits evaluated (all exact methods)	matmul GFLOP per forward	naive: forward allocations	naive: saved for backward
1,024	4.2 million	1.07	49 MiB	48 MiB
4,096	67 million	17.2	772 MiB	768 MiB
16,384	1.07 billion	275	12,304 MiB (12 GiB)	12,288 MiB

Quadrupling `L` multiplies all of these by 16. At 16,384 tokens, *one forward call of one attention layer* written naively would need 12 GiB of temporary memory, which is most of this laptop's RAM. That is the practical meaning of "quadratic".

Compare the inputs: `q, k, v, out` at `L = 16,384` are `4 × 4 × 16384 × 64 × 4 bytes = 64 MiB`. **The temporary tables are 190 times larger than the actual inputs and outputs.**

5.3 Four different kinds of "memory cost"

Confusing these is the most common mistake in discussions of efficient attention. The project keeps them separate:

1. **Auxiliary memory of one forward call:** temporary tensors that exist only during the call. Naive: `O(L²)`. Tiled and fused: `O(tile² + L·d)`.
2. **Memory saved for the backward pass:** what autograd keeps alive between forward and backward when training. Naive: the `L²` probability tables. Autograd-tiled: *also* `L²` in total, because it keeps every tile. Recompute: only `O(L)` statistics.
3. **Total training memory of a model:** parameters, gradients, optimizer state (AdamW keeps 2 extra numbers per parameter), activations of every layer, plus the attention terms above.
4. **Memory traffic:** how many bytes travel between RAM and the caches. This is not storage at all; it is *movement*. A method can allocate little and still move a lot, or the other way round. Traffic is what FlashAttention's analysis is about (Chapter 8) and what largely decides speed (Chapter 3).

Chapter 13 explains which of these the project measures (1 and 2 directly, and process-level memory), which it estimates (all, in theory), and which it cannot measure directly (4, which would need hardware counters).

5.4 What each family of algorithms changes

Family	Time	Auxiliary memory	Computes exact attention?
Naive / materializing	`O(L²d)`	`O(L²)`	yes
Tiled / fused (FlashAttention idea)	`O(L²d)`, **unchanged**	`O(tile² + Ld)`	yes
Local window (width `w`)	`O(L·w·d)`	`O(L·w)`	no: only nearby pairs
Block sparse (`k` blocks per row)	`O(L·k·b·d)`	`O(L·k·b)`	no: only selected blocks
Linear attention	`O(L·d²)`	`O(d²)` (non-causal)	no: a different function

Two things stand out:

- **Tiling attacks memory, not arithmetic.** Every exact algorithm must look at every allowed pair, because every output depends on every pair. Tiling only decides *where* the scores live: in small, cache-resident tiles instead of a giant table. Chapter 14 confirms this experimentally: every exact method's time grows like `L^1.9–2.1`, while the tiled methods' memory grows like `L^0.8`.
- **Only methods that change the function can change the time complexity.** Local, block-sparse and linear attention are faster asymptotically *because* they compute something different. Whether that difference matters is a quality question (Chapters 16 and 18).

5.5 Why "quadratic" is only the start of the story

Big-O hides constant factors, and real performance depends on:

- **Memory traffic** (Chapter 3): the number of passes over large tables, not just their count.
- **Dispatch overhead**: Python-level loops pay microseconds per operation.
- **Parallel efficiency**: how well the work splits across threads.
- **Hardware**: cache sizes, vector width, supported number formats.

These factors explain every "surprising" result in this book:

- the C++ fused kernel is 3× faster than the Python tiled code, despite the same complexity (Chapter 14);
- local-window attention, which is `O(L)`, is *slower* than exact attention below about 2,048 tokens (Chapter 15);
- adding threads makes the Python tiled code *slower* (Chapter 15).

Key points to remember

- Exact attention: `4·L²·d` FLOPs per head; quadratic time **always**.
- Naive auxiliary memory: about `3·L²` numbers. At L = 16,384 and 4 heads that is 12 GiB.
- Separate auxiliary memory, saved-for-backward memory, total training memory and memory traffic.
- Tiling reduces memory, not arithmetic. Only changing the function reduces arithmetic.
- Measured speed depends on constants, traffic, dispatch, threads and hardware, not only on big-O.

Chapter 6 — A guided tour of the codebase, and how the project started

> **What you will learn:** what was on the machine on day one and how it was checked; how the project is organized and why; the single calling convention that ties everything together; the registry of algorithms; and where every kind of output is stored.

6.1 Day one: the audit

The project started in an **empty folder** (not even a git repository) inside OneDrive on the Desktop. The first job was to verify the environment rather than trust the brief:

What was checked	How	Finding
Folder contents	`ls -la`, `find`	empty
CPU	PowerShell `Get-CimInstance Win32_Processor`	**Ryzen 5 7530U** (the brief said 5300U), 6 cores / 12 threads
RAM	`Win32_ComputerSystem`, `psutil`	16.4 GB total, **only 0.86–1.27 GB free** at the time
GPU	`Win32_VideoController`	AMD Radeon integrated graphics, no CUDA
Disk	`Get-PSDrive C`	30 GB free (not the 500 GB the brief described)
Python	`python --version`, `py -0p`	3.13.3
Packages	import each	torch 2.10.0+cpu, numpy 2.5.2, scipy, pandas 3.0.0, matplotlib 3.10.8, streamlit 1.58, pytest 8.3.4, psutil 7.2.2, pyyaml
PyTorch internals	`torch.__config__.parallel_info()`, `torch.backends.cpu.get_cpu_capability()`	MKL and oneDNN present, OpenMP, 6 intra-op threads by default, capability **AVX2**
Attention kernels	profiler on `scaled_dot_product_attention`	CPU SDPA runs `aten::_scaled_dot_product_flash_attention_for_cpu` (Chapter 9)
Fully-masked rows	tiny experiment	SDPA returns zeros, a naive softmax returns NaN (Chapter 7)
Profiler memory events	tiny experiment	`[memory]` events carry `nbytes()` and timestamps, so a peak can be reconstructed (Chapter 13)
Python start-up cost	timed `import torch`	5.5 s cold, so running every benchmark in a fresh process would be expensive (Chapter 13)

Three decisions came straight out of the audit:

1. **Everything must work on CPU**, and must never be described as a GPU result.
2. **Every benchmark needs a memory guard**, because free RAM was below 1.3 GB.

3. **The project would not be installed into the system Python.** A tiny launcher, `lab.py`, adds `src/` to the import path, so `python lab.py ...` works from a fresh checkout. (`pip install -e .` also works, but is optional.)

> **What actually happened:** the free disk space (30 GB) and the CPU model (7530U) both differed from the brief. They are recorded in the documentation as found, not as described. Later, the free RAM and the CPU load turned out to change minute by minute because of other programs. That discovery reshaped the benchmarking methodology (Chapters 13 and 22).

6.2 The layout

```
Beyond Quadratic Attention/
├── lab.py                       launcher: python lab.py <command>
├── pyproject.toml               package metadata + pytest settings
├── README.md                    quick start and command list
├── src/attention_lab/           the library (≈ 4,450 lines)
│   ├── algorithms/              the 8 attention implementations + shared mask logic + registry
│   ├── correctness/             error metrics, tolerance policy, correctness laboratory
│   ├── complexity/              theoretical cost model
│   ├── benchmarking/            environment capture, timing, memory, load monitor, runner,
scheduler
│   ├── tasks/                   retrieval probes + synthetic token tasks
│   ├── models/                  tiny Transformer with pluggable attention
│   ├── training/                training loop + multi-backend task experiments
│   ├── diagnostics/             instrumented training for the MQAR investigation
│   ├── reporting/               results → tables, fitted exponents, markdown reports
│   ├── visualization/           matplotlib figures from measured data
│   ├── cli/                     command-line interface
│   └── ui/                      Streamlit web interface
├── tests/                       unit / numerical / integration / scientific / perf_smoke
├── scripts/                     diagnose_recall.py, summarize_diag.py
├── experiments/
│   ├── configs/                 YAML specifications of every benchmark sweep
│   ├── results/<run>/           raw results: results.jsonl + meta.json  (never edited by hand)
│   ├── reports/<run>/           generated summary.md, CSV tables, PNG figures
│   ├── diagnostics/mqar_v2/     logs of the MQAR diagnostic experiments
│   └── checkpoints/             saved model weights (git-ignored)
└── docs/                        guides, results, this book
```

The Python source is about 5,600 lines in total, including tests and scripts. The largest files are the CLI (360 lines), the benchmark runner (241), the tiled attention (230) and the reporting module (227).

Why this separation? The brief demanded that algorithms, benchmarks, model integration and UI be kept apart. The practical reason is trust: an algorithm file contains *only* computation (no timing, no file writing), so it can be tested in isolation, and a benchmark cannot accidentally change what it measures.

6.3 One calling convention for everything

Every attention implementation, exact or approximate, is called the same way:

```
out = method(q, k, v, causal=False, key_padding_mask=None, **params)
# q: [B, H, L, d]   k: [B, H, L, d]   v: [B, H, L, dv]   →   out: [B, H, L, dv]
```

`params` holds method-specific knobs: `q_tile` and `kv_tile` for the tiled methods, `window` for local attention, `block_size` and friends for block-sparse attention, `chunk` for linear attention. Because the signature is shared, the benchmark runner, the correctness laboratory, the probes, the Transformer model and the UI are each written **once** and switch algorithms by name.

6.4 The registry

> **In the code:** `src/attention_lab/algorithms/registry.py`

The registry maps each name to an `AttentionMethod` record:

```
AttentionMethod(
    name="tiled",
    fn=tiled_attention,
    family="exact",                      # or "restricted-softmax" or "different-kernel"
    computes="softmax(Q K^T / sqrt(d) + M) V",
    implementation="Python-loop online-softmax tiling; autograd through the loop ...",
    default_params={"q_tile": 128, "kv_tile": 128},
    supports_dense_mask=True, ...
)
```

The `family` and `computes` fields make the scientific status of each method explicit. Reports and the UI use `family` to decide whether an output difference should be called an **error** (exact methods) or a **deviation** (approximate methods). This matters because, for example, linear attention's difference from softmax is not a mistake: it is a different function (Chapter 10).

The eight registered names: `naive`, `sdpa_math`, `sdpa_flash_cpu`, `tiled`, `tiled_recompute`, `local_window`, `block_sparse`, `linear_elu`.

6.5 Where outputs go

- **Raw results** (`experiments/results/<timestamp>_<name>/`): `results.jsonl` holds one JSON object per configuration, *including* skipped and failed ones with a reason. `meta.json` holds the full spec and the environment. These files are the evidence and are never edited by hand.
- **Reports** (`experiments/reports/<same name>/`): generated from the raw results by `python lab.py report <run>`: `summary.md`, `summary.csv`, `exponents.csv` and PNG figures.
- **Documentation** (`docs/`): `results.md` collects the findings and names the run behind every number.

Keeping raw data, generated reports and interpretation separate means any number in this book can be traced back: book → `docs/results.md` → run name → `results.jsonl` line.

Key points to remember

- The audit found a different CPU (7530U), little free RAM and no GPU, and the design followed from that.
- Algorithms contain only computation. Benchmarks, models, probes and UI call them by name.
- Shared signature: `fn(q, k, v, causal=..., key_padding_mask=..., **params)` on `[B, H, L, d]` tensors.
- The registry labels each method's family, so errors and deviations are never confused.

- Raw results → reports → docs: every number is traceable.

Chapter 7 — Implementation A: the reference ("naive") attention and the mask system

> **What you will learn:** why a project like this needs a slow, obvious reference implementation; how the reference is written, line by line; how masks are defined once and shared by every algorithm; the fully-masked-row problem and how a policy was chosen; input validation; and how the reference itself is tested.

7.1 Why start with a slow implementation?

The first principle of the project was **correctness before performance**: an "optimization" that silently computes the wrong function is not an optimization. Every other algorithm is judged by comparing it with something we trust, so that thing must be as simple and transparent as possible. The reference (`naive_attention`) is written to be *read*, not to be fast. It literally builds the full `L × L` tables.

7.2 The code, step by step

> **In the code:** `src/attention_lab/algorithms/naive.py`, function `naive_attention`.

```
shp = validate_qkv(q, k, v)                     # check shapes, dtypes, devices
sc = default_scale(shp.d, scale)                # 1/sqrt(d) unless told otherwise
spec = MaskSpec(shp, mask=mask, key_padding_mask=key_padding_mask, causal=causal,
device=q.device)

s = torch.matmul(q, k.transpose(-2, -1)) * sc   # [B,H,Lq,Lk]   the quadratic table
allowed, additive = spec.tile(0, shp.Lq, 0, shp.Lk)
s = apply_tile_mask(s, allowed, additive)       # forbidden pairs -> -inf

m = safe_row_max(s)                             # [B,H,Lq,1]    row maximum (stable softmax)
e = torch.exp(s - m)                            # [B,H,Lq,Lk]   second quadratic table
l = e.sum(dim=-1, keepdim=True)                 # [B,H,Lq,1]    row sums
l_safe = torch.where(l == 0, torch.ones_like(l), l)
p = e / l_safe                                  # [B,H,Lq,Lk]   third quadratic table
out = torch.matmul(p, v)                        # [B,H,Lq,dv]
```

Every line maps to one step of Chapter 4's formula. Notice the three `L × L` tensors `s`, `e` and `p`. At the peak of the call they all exist at once, which is why the theory model predicts naive forward memory as "3 score tables + output". The measured memory matched to within 0.1 MiB (Chapter 14).

`naive_attention(..., return_lse=True)` also returns the **log-sum-exp** of each row, `m + log(l)`. The tiled implementations store exactly this per-row statistic for their backward pass (Chapter 8), and tests compare the two.

7.3 The stable softmax and the detached maximum

`safe_row_max` (in `base.py`) does two things:

```
m = s.detach().amax(dim=-1, keepdim=True)
return torch.where(torch.isneginf(m), torch.zeros_like(m), m)
```

1. **Detach.** The softmax result does not depend on which constant is subtracted (Chapter 2), so in exact arithmetic the gradient flowing "through" `m` cancels out. Detaching tells autograd not to compute that cancelling term, which avoids a little extra rounding noise.
2. **Replace `-∞` with 0.** If every score in a row is `-∞` (a fully-masked row), the maximum is `-∞`, and `s - m` would be `-∞ - (-∞) = NaN`. Using 0 instead gives `exp(-∞ - 0) = 0`, so the whole row of `e` is zero, `l` is zero, and the `l_safe` line divides by 1 instead of 0. The output row is zero.

7.4 The fully-masked row: choosing a policy

Mathematically, a query with no allowed keys has an undefined attention output (`0/0`). Implementations disagree about what to return. Some produce NaN, which then spreads through the network like a poison.

> **What actually happened:** during the audit, two tiny experiments were run. A plain `torch.softmax` over a row of `-∞` returned `[nan, nan, nan, nan]`. PyTorch's `scaled_dot_product_attention` on CPU, with a mask that removed a whole row, returned a **zero vector** for that row, and the gradients contained no NaN. Both of PyTorch's CPU backends (the fused flash kernel and the math kernel) were checked with boolean and with additive masks, and all returned zeros. The project therefore adopted the same policy, **fully-masked rows produce zero output and zero gradient**, wrote it into the docstring of `base.py`, and tested it for every exact method (`test_fully_masked_rows_and_padding`, `test_fully_masked_rows_give_zero_output_and_zero_grad`).

7.5 Masks are defined in exactly one place: `MaskSpec`

> **In the code:** `src/attention_lab/algorithms/base.py`, class `MaskSpec`.

Masks are easy to get subtly wrong. Is `True` "keep" or "remove"? Is causal "`j ≤ i`" or "`j < i`"? What happens when the query and key lengths differ? To avoid each algorithm answering these questions differently, all mask logic lives in one class, and the semantics are written at the top of `base.py`:

- **boolean `mask`**: `True` means the pair **takes part** (the same convention as PyTorch SDPA);
- **float `mask`**: added to the scaled scores, where `-∞` removes the pair; `+∞` and NaN are rejected;
- **`key_padding_mask` `[B, Lk]`**: `True` = real token;
- **`causal=True`**: query `i` may attend key `j` iff `j ≤ i` (top-left aligned, the same as SDPA's `is_causal`).

The key design idea: algorithms never ask for "the mask". They ask for **the mask of the tile they are about to compute**:

```
allowed, additive = spec.tile(qs, qe, ks, ke)   # queries [qs, qe) × keys [ks, ke)
```

For a causal mask, the tile is built on the fly from two small `arange` vectors (`kj <= qi`). For a user-supplied dense mask, the tile is a **slice of a broadcast view**, so no `L × L` copy is made. This is what lets the tiled algorithm support arbitrary masks while still never materializing an `L × L` table.

`spec.tile_fully_masked_by_causality(qs, qe, ks)` tells a tiled loop when a whole tile lies above the diagonal, so it can skip it entirely (Chapter 8).

Causal alignment when lengths differ. "Top-left" means query 0 lines up with key 0. When generating text with a cache of previous keys (`Lq < Lk`), the usual intent is "bottom-right" (the last query lines up with the last key). The project deliberately implements only top-left, matching SDPA, and documents that you need an explicit mask for the other convention. A test (`test_causal_is_top_left_aligned_like_sdpa`) pins this behaviour against PyTorch.

7.6 Input validation

`validate_qkv` rejects, with a clear message:

- tensors that are not 4-D `[B, H, L, d]`;
- non-floating-point dtypes, or q/k/v with different dtypes or devices;
- mismatched batch or head dimensions; q and k with different head dimensions; k and v with different lengths;
- zero-sized dimensions.

`default_scale` rejects non-positive scales. `MaskSpec` rejects masks that cannot broadcast to `[B, H, Lq, Lk]`, padding masks with the wrong shape or dtype, and additive masks containing `+∞` or NaN. Each rejection has a test in `tests/unit/test_base_and_naive.py`.

7.7 How the reference itself is tested

A reference must be checked against something *independent*. The tests compare it with a separate formulation, `softmax(QKᵀ·scale + bias) @ V` using `torch.softmax` (`textbook()` in the test file), and with PyTorch's own SDPA:

Test	What it proves
`test_matches_textbook_formula_and_default_scale`	same output as an independent formula, to 1e-12
`test_scale_is_one_over_sqrt_d`	default scale is exactly $1/\sqrt{d}$
`test_rows_of_probabilities_sum_to_one`	probabilities are non-negative and sum to 1
`test_bool_mask_true_means_attend`	with only key 2 allowed, every output equals v_2
`test_additive_mask_is_added_to_scaled_logits`	float masks are biases
`test_causal_is_top_left_aligned_like_sdpa`	causal = `tril` mask = PyTorch's `is_causal`
`test_key_padding_mask_excludes_padded_keys`	changing padded values to 10^6 changes nothing
`test_masks_combine_by_intersection`	mask + causal + padding = their logical AND
`test_lse_matches_logsumexp`	the returned LSE equals `torch.logsumexp`
`test_gradients_match_textbook`, `test_gradcheck_naive`	gradients are correct (including finite-difference `gradcheck`)

`torch.autograd.gradcheck` deserves a word. It perturbs each input by a tiny amount in float64, measures how the output changes (a numerical derivative), and compares that with the gradient computed by autograd. It is the gold standard for "is my backward pass right?".

Key points to remember

- The reference is deliberately simple and builds three `L × L` tables.
- The stable softmax subtracts a *detached* row max, with `−∞` replaced by 0 for empty rows.
- Fully-masked rows output zero, a policy chosen by checking what PyTorch's CPU kernels do.
- `MaskSpec` defines mask semantics once and serves masks *per tile*, without building `L × L` masks.
- Causal is top-left aligned, like SDPA.
- The reference is tested against an independent formula, PyTorch SDPA, and finite-difference gradients.

Chapter 8 — Implementation B: tiled exact attention and the online softmax

> **What you will learn:** the central trick of FlashAttention, called *online softmax*: how to compute an exact softmax-weighted average while seeing the scores one tile at a time; a worked numerical example; the proof that it is exact; how the project implements it; why letting autograd differentiate through it does *not* save training memory; and how a custom "recompute" backward pass fixes that.

8.1 The problem

Naive attention builds the whole row of scores $s_1 \dots s_L$ for a query before applying softmax, because the softmax denominator $\Sigma \exp(s_j)$ needs *all* of them. The question is whether the scores can be processed in small **tiles** (chunks of keys) so that the full row never exists, *and still get the exact answer*.

The difficulty is that the stable softmax subtracts the row **maximum**, and we only know the true maximum after seeing every tile.

8.2 The idea: keep three running numbers and fix them up when the maximum changes

For one query row, while streaming over tiles, keep:

- `m`: the largest score seen so far;
- `l`: the sum of $\exp(s_j - m)$ over keys seen so far (the denominator, *relative to the current* `m`);
- `acc`: the sum of $\exp(s_j - m) \cdot v_j$ over keys seen so far (the numerator, also relative to `m`).

When a new tile arrives with scores `S` and values `V_tile`:

```
U1   m_new = max(m, max(S))              new running maximum
U2   α     = exp(m − m_new)              how much to shrink the old sums (≤ 1)
U3   P     = exp(S − m_new)              this tile's weights, relative to m_new
U4   l     = α · l   + sum(P)            rescale the old denominator, add the new terms
U5   acc   = α · acc + P · V_tile        rescale the old numerator, add the new terms
     m     = m_new
```

At the end: `output = acc / l`, and the log-sum-exp is `lse = m + log(l)`.

The only "trick" is U2. Old sums were computed relative to the old maximum. Multiplying by $\exp(m - m_{new})$ re-bases them to the new maximum, because $\exp(s - m) \cdot \exp(m - m_{new}) = \exp(s - m_{new})$.

8.3 A worked example

One query, four keys, scores `s = [1, 3, 2, 5]`, scalar values `v = [10, 20, 30, 40]`, tiles of 2 keys.

Start: $m = -\infty$, $l = 0$, $acc = 0$.

Tile 0 (scores `[1, 3]`, values `[10, 20]`):

```
m_new = max(−∞, 3) = 3
α     = exp(−∞ − 3) = 0
P     = [exp(1−3), exp(3−3)] = [0.135335, 1]
l     = 0·0 + 1.135335 = 1.135335
acc   = 0·0 + 0.135335·10 + 1·20 = 21.353353
```

Tile 1 (scores `[2, 5]`, values `[30, 40]`): the maximum rises from 3 to 5, so the old sums shrink:

```
m_new = max(3, 5) = 5
α     = exp(3 − 5) = 0.135335
P     = [exp(2−5), exp(5−5)] = [0.049787, 1]
l     = 0.135335 · 1.135335 + 1.049787 = 1.203438
acc   = 0.135335 · 21.353353 + 0.049787·30 + 1·40 = 44.383474
```

Result: `acc / l = 36.880566`, and `lse = 5 + log(1.203438) = 5.185182`.

Check with the direct formula: weights `exp([1,3,2,5]) / Σ` and the weighted average of `[10,20,30,40]` give `36.880566`, with `lse = log Σ exp = 5.185182`. They are identical (to 15 digits in this calculation).

8.4 Why it is exactly right (the proof)

Invariant: after processing tiles 1..t, `l = Σ_{seen j} exp(s_j − m)` and `acc = Σ_{seen j} exp(s_j − m) v_j`, where `m` is the max over seen keys.

- It holds at the start (empty sums are 0; `m = −∞`).
- If it holds before a tile, then `α · l = Σ_old exp(s_j − m)·exp(m − m_new) = Σ_old exp(s_j − m_new)`, and U4 adds exactly the new terms `Σ_tile exp(s_j − m_new)`. So it holds after. The same argument works for `acc`.
- At the end, `acc / l = Σ exp(s_j − m) v_j / Σ exp(s_j − m)`; the common `exp(−m)` cancels, leaving `Σ exp(s_j) v_j / Σ exp(s_j)`, **the softmax-weighted average**. ■

Tile order and tile sizes do not matter *mathematically*. A unit test streams the same row with five different splits (`[10]`, `[1]×10`, `[3,3,4]`, `[7,1,2]`, `[2,8]`) and gets the same answer every time (`test_streaming_equals_softmax_for_any_split`).

Numerical safety. Every exponent is `≤ 0` (scores never exceed the running max), so nothing overflows. Even if a later tile is 1,000 larger, `α = exp(-1000) = 0` simply discards the negligible old mass (`test_update_handles_increasing_maxima_without_overflow`). If all scores seen so far are masked (`−∞`), the code shifts by 0 instead of `−∞` so that no NaN appears (`test_update_handles_all_masked_first_tile`).

8.5 The implementation

> **In the code:** `src/attention_lab/algorithms/tiled_exact.py`.

The update rule is a standalone function, so that the documentation, the unit tests and the kernel all use the same code:

```
def online_softmax_update(m, l, acc, s, v_j):
    m_new = torch.maximum(m, s.detach().amax(dim=-1, keepdim=True))           # U1
    m_shift = torch.where(torch.isneginf(m_new), torch.zeros_like(m_new), m_new)
    alpha = torch.exp(m - m_shift)                                             # U2
    p = torch.exp(s - m_shift)                                                 # U3
    l = alpha * l + p.sum(dim=-1, keepdim=True)                                # U4
    acc = alpha * acc + torch.matmul(p, v_j)                                   # U5
    return m_new, l, acc
```

The forward kernel, `tiled_forward_kernel`, loops over **query tiles** (outer loop, `q_tile` rows) and **key tiles** (inner loop, `kv_tile` keys). For each tile pair it computes a small `q_tile × kv_tile` score block, asks `MaskSpec` for that tile's mask, and applies the update. It works on whole `[B, H, …]` blocks at once, so all batches and heads advance together. Three details:

- **Ragged final tiles.** If `L` is not a multiple of the tile size, the last tile is simply shorter (Python slicing handles it). Tests use awkward sizes such as `L = 77` with tiles of 16 and 24.
- **Causal tile skipping.** If a key tile starts after the last query of the current block, every pair is above the diagonal, and every later tile is too, so the inner loop `break`s. For `L = 64` with 16 × 16 tiles, 10 of 16 tiles are computed (`test_causal_skips_tiles_above_diagonal`).
- **fp32 accumulators for 16-bit inputs.** If the inputs are `bfloat16`, the running `m`, `l`, `acc` and the score tiles are computed in fp32 (`_acc_dtype`). FlashAttention does the same. Chapter 11 shows this single choice is the difference between 0.3 % and 38 % error.

> **What actually happened:** the update function was originally written inline inside the kernel. Halfway through the first benchmark sweep it was moved into the standalone `online_softmax_update`, so that tests could exercise it alone. Because the sweep's worker processes import the code fresh, later configurations of that sweep ran the refactored version. The operations were identical, plus one Python function call per tile, which is negligible. This was noted, and that sweep's timings were later discarded anyway, for a different reason (Chapter 13).

8.6 The backward pass: where tiling alone fails

Training needs gradients. The simplest option, `tiled_attention`, lets PyTorch's **autograd** differentiate straight through the Python loop. That gives correct gradients (tests confirm it to 1e-15 in fp64). But autograd must **keep every tile's intermediate tensors** (each `P`, each `exp` output) alive until the backward pass. Added up over all tiles, that is again `O(L²)` memory. The forward pass was memory-efficient; training is not.

The measurement in Chapter 14 shows it plainly. The forward-plus-backward memory of the autograd-tiled version grows with exponent **1.72** in L (76 MiB at L = 2048), compared with **0.75** for the recompute version (9.9 MiB).

8.7 The fix: recompute instead of store

`tiled_attention_recompute` is a custom `torch.autograd.Function`, a piece of code with a hand-written `backward`:

- **Forward:** run the same tiled kernel, but save only `Q, K, V`, the output `O` and the per-row `lse = m + log(l)`. That is `O(L·d)` memory, nothing quadratic.

- **Backward:** loop over key tiles (outer) and query tiles (inner). **Recompute** each score tile `S = Q_i K_jᵀ · scale`, and recover the exact probabilities as `P = exp(S − lse)`. Because `lse` is the exact log-normalizer of each row, this gives exactly the softmax weights, with no need to know the maximum again.

The gradient formulas, derived in `docs/attention_math.md` §4, are:

```
D   = rowsum(dO ∘ O)            computed once, O(L·d)
dV += Pᵀ dO
dP  = dO Vᵀ
dS  = P ∘ (dP − D)
dQ += dS K · scale
dK += dSᵀ Q · scale
```

The trick is in `D`. The softmax Jacobian needs `Σ_k P_ik dP_ik` for each row, which looks as if it needs the whole row of `P`. But `dP_ik = dO_i · V_k` and `O_i = Σ_k P_ik V_k`, so the sum equals `dO_i · O_i`, which can be computed from `O` and `dO` alone. The same structure appears in FlashAttention-2's backward pass (Dao, 2023).

The cost of the trade: one extra `Q Kᵀ` per tile, about **25 % more matmul FLOPs** in the backward pass ($10 \cdot L^2 \cdot d$ instead of $8 \cdot L^2 \cdot d$). **The benefit:** saved memory falls from `O(L²)` to `O(L·d)`.

Validation: `torch.autograd.gradcheck` on a causal, padded, ragged-tile case (`test_gradcheck_tiled_recompute_custom_backward`), plus comparisons with naive gradients across 49–57 cases per dtype in the correctness laboratory. All pass.

> **What actually happened:** a measured surprise. The recompute version does *more* arithmetic, yet it was **faster** than the autograd version at every length from 512 tokens upward (980 ms vs 1,847 ms at L = 2048 on one thread), and faster than naive (1,411 ms). Allocating and keeping hundreds of MiB of tiles costs more than recomputing them. This is a CPU echo of FlashAttention's main argument: on modern hardware, moving memory is often more expensive than arithmetic (Chapter 14).

8.8 What this implementation is *not*

It is not FlashAttention. FlashAttention (Dao et al., 2022) is a **single fused GPU kernel** that keeps tiles in on-chip SRAM and never writes scores to main memory. Our Python version runs each line as a separate PyTorch operation, each of which reads and writes its operands through the cache hierarchy. It reproduces the **algorithm and its memory behaviour**, not the **hardware efficiency**. Chapter 9 introduces the fused CPU kernel that PyTorch provides, which does exploit the hardware, and Chapter 14 compares them.

Key points to remember

- Online softmax keeps `m`, `l` and `acc` per row and rescales by `α = exp(m_old − m_new)` when the max grows.
- It is exactly equal to softmax in exact arithmetic, for any tile order or size.
- Worked example: scores [1,3,2,5] and values [10,20,30,40] give 36.880566 by both methods.
- Tiles are processed in `[B,H]` blocks; ragged tiles are fine; causal tiles above the diagonal are skipped.
- 16-bit inputs use fp32 accumulators.
- Autograd through the loop makes training memory `O(L²)` again (measured exponent 1.72).

- The recompute backward stores only O, lse (`O(L·d)`), recomputes `P = exp(S − lse)`, and uses `D = rowsum(dO∘O)`.
- Recompute does 25 % more backward FLOPs, yet measured faster than storing on this CPU.

Chapter 9 — Implementation C: PyTorch's built-in kernels and how we found out which one runs

> **What you will learn:** what PyTorch's `scaled_dot_product_attention` (SDPA) is; why "which kernel ran?" is a real question; how the profiler answered it; the two adapters the project registers; how masks are translated; and why the CPU "flash" kernel must not be called FlashAttention.

9.1 SDPA: one function, several implementations

PyTorch offers `torch.nn.functional.scaled_dot_product_attention(q, k, v, attn_mask=None, is_causal=False, scale=None)`. Behind that single function, PyTorch chooses among several **backends** (implementations): FlashAttention-style kernels, "memory-efficient" kernels and cuDNN on CUDA GPUs, and a "math" fallback. You can force a backend with the context manager `torch.nn.attention.sdpa_kernel(SDPBackend.X)`.

The PyTorch documentation (checked during the project) describes the optimized kernels in terms of the CUDA backend and says that other backends use the plain PyTorch implementation. On a CPU-only machine it was therefore unclear what would actually run.

9.2 Asking the profiler

The PyTorch **profiler** records every operator executed inside a `with profile(...)` block. During the audit, SDPA was run with each backend and mask type, and the operator names were collected:

Forced backend	Operators observed
default (none forced)	`aten::_scaled_dot_product_flash_attention_for_cpu` (+ `_backward`)
`SDPBackend.FLASH_ATTENTION`	`aten::_scaled_dot_product_flash_attention_for_cpu` (+ `_backward`)
`SDPBackend.MATH`	`aten::_scaled_dot_product_attention_math` (backward = ordinary ops)

This was true with `is_causal=True`, with a boolean mask and with an additive float mask. So **this CPU build of PyTorch has a dedicated fused "flash" kernel for CPUs and uses it by default.** A research subagent then checked PyTorch's source: the operator is declared in `native_functions.yaml`, returns `(output, logsumexp)`, and is selected by the CPU backend selector. It came in through a pull request that initially supported forward fp32 only. The docs sentence about "CUDA only" is out of date for CPU (recorded in `docs/references.md`).

The same experiment confirmed the fully-masked-row behaviour: both CPU backends return zeros (Chapter 7).

9.3 The two adapters

> **In the code:** `src/attention_lab/algorithms/sdpa.py`.

```
_BACKENDS = {"flash": SDPBackend.FLASH_ATTENTION, "math": SDPBackend.MATH}

def sdpa_attention(q, k, v, *, mask=None, key_padding_mask=None, causal=False, scale=None,
backend="flash"):
    ...
    attn_mask, is_causal = _sdpa_mask(spec)
    with sdpa_kernel(_BACKENDS[backend]):
        return F.scaled_dot_product_attention(q, k, v, attn_mask=attn_mask, is_causal=is_causal,
scale=sc)
```

Two registry entries use it: `sdpa_flash_cpu` (forced flash) and `sdpa_math` (forced math). Forcing the backend means every benchmark row states which kernel ran, instead of relying on an automatic choice that might change between PyTorch versions.

Mask translation (`_sdpa_mask`). SDPA does not accept `attn_mask` and `is_causal` together, and it has no separate padding mask. The adapter therefore:

- passes `is_causal=True` with no mask tensor when causality is the only mask (the cheapest case);
- otherwise builds the combined mask at its *broadcast* shape: a padding mask alone stays `[B, 1, 1, L]`; anything combined with causality becomes `[…, L, L]` (an `O(L²)` boolean, a documented cost of the adapter, not of the kernel);
- merges boolean and additive masks into one additive mask (`additive.masked_fill(~allowed, −∞)`).

9.4 What the flash CPU kernel does, and what it is not

The ATen CPU kernel processes blocks of queries against blocks of keys inside one C++ function. Scaling, masking, the running max, `exp`, the running sums and the `P·V` accumulation all happen on small blocks held in cache. It uses vectorized `exp`, parallelizes over (batch, head, query block) with OpenMP, and stores the logsumexp for the backward pass. That is the same algorithm as Chapter 8, but **fused**: no intermediate ever goes to RAM, and there is no per-operation Python dispatch.

It is **not** the official FlashAttention library by Dao et al., which is a CUDA implementation for NVIDIA GPUs. The project never calls it FlashAttention. Its registry entry reads "PyTorch's fused tiled C++ CPU kernel; NOT the official FlashAttention CUDA library".

`sdpa_math` is PyTorch's own materializing implementation, essentially an optimized version of our naive reference.

9.5 How they compare (preview of Chapter 14)

Both adapters agree with the reference to about 1e-15 in fp64. On one thread at L = 4096 (forward), the fused kernel took **516 ms**, against about 1,500 ms for naive, the math backend, and the Python tiled code. All four compute the same mathematics, so the factor of about 3 comes entirely from *how* the computation is executed.

Key points to remember

- SDPA has several backends. On this CPU build, the default is a fused flash-style CPU kernel.
- The profiler (operator names) is how we *verified* which kernel ran, instead of assuming.
- `sdpa_flash_cpu` and `sdpa_math` force their backends, so results always name the kernel.

- The adapter merges masks into SDPA's format, and combined causal+padding masks cost an `L × L` boolean.
- The CPU flash kernel is PyTorch's own and must not be called "FlashAttention".

Chapter 10 — The approximate methods: local window, block sparse and linear attention

> **What you will learn:** what each approximate method computes *exactly* (restricted softmax versus a different function); how each is implemented efficiently in PyTorch (the `unfold` trick, gathering active blocks, the associativity trick and chunked prefix sums); how each is verified against its *own* definition; and where each one's cost comes from.

10.1 Two kinds of "approximate"

- **Restricted softmax** (local window, block sparse): exactly the softmax attention of Chapter 4, but with many pairs forbidden, as if a fixed mask were applied. Inside the allowed pairs everything is exact. What is lost is *information*: the forbidden pairs.
- **Different function** (linear attention): replaces `exp(q·k/√d)` by another similarity. It is not softmax attention with an error; it is a different model.

The project verifies each method against its **own exact definition**. For restricted methods that is naive attention with the corresponding mask. For linear attention it is a slow, direct evaluation of the same formula. Their difference from full softmax attention is then measured separately and called **deviation**, never "error".

10.2 Local-window (sliding-window) attention

What it computes: query `i` may attend to key `j` only if `i − w_left ≤ j ≤ i + w_right`. With `window = w` non-causal, both sides are `w`; causal means `[i − w, i]`. This idea is used, for example, in Longformer (Beltagy et al., 2020).

The naive way would waste the savings: build the full `L × L` table and mask most of it. That costs as much as full attention.

The project's way: chunks and `unfold`.

> **In the code:** `src/attention_lab/algorithms/local_window.py`.

1. Split the queries into chunks of `C` rows. Chunk `c` covers queries `[cC, cC + C)`.
2. Those queries can only see keys in `[cC − w_l, cC + C + w_r)`, a **span** of `S = C + w_l + w_r` keys.
3. Pad K and V with `w_l` zeros in front and `w_r` (plus padding to a multiple of `C`) behind, then call `kp.unfold(2, S, C)`. That produces, *as a strided view*, all overlapping windows of `S` keys starting every `C` positions: a `[B, H, nC, S, d]` tensor.
4. Compute scores chunk by chunk, all at once: `[B, H, nC, C, d] @ [B, H, nC, d, S] → [B, H, nC, C, S]`.
5. Build the band mask for a chunk: query row `a` and window slot `b` are allowed iff `0 ≤ b − a ≤ w_l + w_r`. Also remove positions that fall in the padding (`key_pos < 0` or `≥ L`), and apply the padding mask after unfolding it in the same way.
6. Stable softmax, multiply by the unfolded values, reshape back to `[B, H, L, dv]`.

Fully vectorized, with no Python loop over positions. Scores evaluated: `[L/C] · C · (C + w_l + w_r)`, which is linear in `L`. About `C/(C + w_l + w_r)` of them are masked waste; the default `C ≈ w_l + w_r + 1` wastes about half.

Verified by: `test_local_window_equals_band_masked_softmax`, a grid of lengths (1, 5, 37, 64), windows (0, 1, 4, 16, 100), causal/non-causal and chunk sizes, comparing output *and gradients* with naive attention under the band mask, to 1e-12. A window as large as the sequence equals full attention.

10.3 Block-sparse attention

What it computes: cut the sequence into blocks of `b` tokens. A boolean **layout** `layout[qb, kb]` says whether query block `qb` may attend key block `kb`. The project's layout, in the spirit of BigBird (Zaheer et al., 2020), combines:

- **local**: `|qb − kb| ≤ local_blocks` (causal: `0 ≤ qb − kb ≤ local_blocks`);
- **global**: the first `global_blocks` key blocks are visible to everyone, and (non-causal) the first query blocks see everything;
- **random**: each query block also sees `random_blocks` extra key blocks, chosen with a fixed seed so the layout is reproducible.

> **In the code:** `src/attention_lab/algorithms/block_sparse.py` — `make_layout`, `block_sparse_attention`.

Execution by gathering. For each query block, collect the indices of its active key blocks, pad every row to the widest row (`K_max`), and **gather** those key and value blocks into a dense `[B, H, n_rows, K_max·b, d]` tensor. Then compute exact softmax attention over just those keys: `[B, H, n_rows, b, K_max·b]` scores instead of `[B, H, L, L]`. Padded slots, padded positions (when `L` is not a multiple of `b`), padding-mask positions and (causal) pairs above the diagonal inside diagonal blocks are all masked.

Global rows are handled separately. A query block that sees *every* block would force `K_max = n_blocks` for all rows and make the whole computation dense. So rows with full coverage are computed against all keys directly, and the rest use the gather path.

Verified by: `test_block_sparse_equals_layout_masked_softmax` (lengths including ragged 77 and 130, five layout configurations, causal/non-causal, gradients) against naive attention with the layout expanded to an element mask. Also tested: the layout is causal when asked, always includes the diagonal and the global column, and is reproducible from its seed.

> **What actually happened:** at short lengths a "sparse" layout can cover everything. With `b = 8` at `L = 32` there are only 4 blocks, and local ±1 + global block 0 + 1 random block already includes all of them. The layout keeps 528 of 528 causal pairs, which is full attention. This was discovered when block_sparse scored exactly like exact attention on the induction task. The book and the results flag those cells as "not evidence about sparsity" (Chapter 18). Likewise, the deviation measurements show 0 at L = 256 with b = 64.

10.4 Linear attention (elu + 1)

What it computes (Katharopoulos et al., 2020): pick a feature map `φ(x) = elu(x) + 1` (applied to each entry; always positive), and define

```
out_i = Σ_j [φ(q_i)·φ(k_j)] v_j  /  Σ_j φ(q_i)·φ(k_j)
```

It looks like attention with weights `φ(q)·φ(k)` instead of `exp(q·k/√d)`. **It is not an approximation of softmax**: `elu + 1` is not designed to estimate `exp` in any sense. (Random-feature methods such as Performer do estimate it, but they are not implemented here.) Following the paper, `φ` is applied to `q` and `k` directly, with no `1/√d`, and passing a `scale` raises an error.

Why it can be linear in L: associativity. `Σ_j (φ(q_i)ᵀ φ(k_j)) v_jᵀ = φ(q_i)ᵀ (Σ_j φ(k_j) v_jᵀ)`. The bracket is a `d × d` matrix that does not depend on `i`. Compute it once (`O(L·d²)`) and reuse it for every query (`O(L·d²)`). No `L × L` table at all.

Causal is harder. Query `i` needs the prefix sum `S_i = Σ_{j≤i} φ(k_j) v_jᵀ`, a different `d × d` matrix for every position. Storing all of them would cost `O(L·d²)` memory. The project uses a **chunked** form:

- inside each chunk of `C` tokens, compute the small `C × C` causal attention directly (quadratic, but only in `C`);
- across chunks, keep one `d × d` summary per chunk, take **exclusive prefix sums** (each chunk sees only earlier chunks), and apply them to the chunk's queries.

The exclusive prefix is built as "zeros, then the cumulative sum shifted by one", rather than "cumulative sum minus own chunk", to avoid cancellation error. Padded or masked keys get `φ(k) = 0`, so they contribute nothing. The denominator gets `+ 1e-6` for safety (it is positive anyway, because `φ > 0`).

> **In the code:** `src/attention_lab/algorithms/linear.py` — `linear_attention` and `linear_attention_quadratic_reference`.

Verified by: `test_linear_chunked_equals_quadratic_definition` (lengths 1–100, chunks 1–128, causal and padded, gradients) against the slow direct formula. Two more tests show it is *not* softmax attention, and that its outputs are convex combinations of values.

10.5 What each method costs, in theory

From `complexity/theory.py`, for `B = 1, H = 4, L = 4096, d = 64` (theory, not measurement):

Method (default params)	Logits evaluated	Matmul GFLOP	Forward allocations
exact methods	67.1 million	17.2	naive 772 MiB; flash 4.1 MiB; tiled 9.2 MiB
local_window (w = 64)	4.2 million	1.09	68.6 MiB
block_sparse (b = 32, 1 local, 1 global, 1 random)	3.1 million	0.80	79.8 MiB
linear_elu (non-causal)	0	0.27	12.1 MiB

Note that the approximate methods' *theoretical* allocations are already much larger than the fused exact kernel's, because they materialize their restricted score tables or gathered blocks. Chapter 15 confirms this by measurement.

Key points to remember

- Local and block-sparse attention are **exact softmax on a restricted set of pairs**. Linear attention is a **different function**.
- Each is verified against its *own* definition (masked naive, or the direct formula), including gradients.
- Local window: chunks + `unfold` + band mask, `O(L·(C + 2w))` scores, about half of them masked waste.
- Block sparse: layout (local + global + random), gather active blocks, pad to `K_max`, global rows done densely.
- At short lengths a sparse layout may be dense (L = 32, b = 8 covers everything).
- Linear: associativity gives `O(L·d²)`; causal uses chunked exclusive prefix sums of `d × d` states.

Chapter 11 — The numerical-correctness laboratory

> **What you will learn:** how to decide whether two implementations "compute the same thing" when floating point never gives exact equality; the metrics the project uses and why; the comparison protocol; the tolerance policy; and the two investigations that changed that policy, each started by a failing check and ended by finding the real cause.

11.1 Three meanings of "the same"

Notion	Meaning
Mathematical equivalence	equal in exact real-number arithmetic (proved on paper, e.g. Chapter 8)
Numerical equivalence	equal within a tolerance justified by the number format and the operations involved
Bitwise equality	the exact same bits

Different summation orders give different roundings (Chapter 3), so two correct implementations are generally **not** bitwise equal. A test pins this: naive and tiled outputs in fp32 differ by under 1e-5, but are not bitwise equal (`test_numerical_vs_bitwise_equality_are_distinct_notions`). If two mathematically equivalent implementations are *not* numerically equivalent, something is wrong: a bug, an instability, or a tolerance that does not fit the situation.

11.2 The metrics

> **In the code:** `src/attention_lab/correctness/metrics.py`, function `compare`.

For an output `x` and a reference `r` (both converted to fp64):

- `max_abs`: the largest absolute difference;
- `mean_abs`: the mean absolute difference;
- `max_abs_normalized` = $\max|x - r| \,/\, \max|r|$: the number the tolerances are checked against. Attention outputs are weighted averages of values, so $\max|r|$ is the natural scale;
- `rel_fro` = $\|x - r\| \,/\, \|r\|$: a global relative error that is robust to near-zero entries;
- `max_rel`: elementwise relative error **with an explicit floor**. Entries where $|r| \le 10^{-3} \cdot \max|r|$ are excluded, and the excluded fraction is reported. Without the floor, a reference entry of 10^{-12} makes any rounding difference look like a 100 % error;
- `cosine`: cosine similarity of the flattened outputs;
- `nan_count`, `inf_count`;
- `bitwise_equal`.

11.3 The protocol for one comparison

> **In the code:** `src/attention_lab/correctness/lab.py`, `evaluate_case`.

1. Draw `q, k, v` in **fp64** from a seeded generator; multiply `q` and `k` by `magnitude`.
2. **Round** them to the test dtype (fp32, bf16, …).
3. The reference is **naive attention in fp64 applied to the already-rounded inputs**. That way the unavoidable error of rounding the inputs is not blamed on the algorithm.
4. Run the method in the test dtype, for forward output and for gradients against a fixed random upstream gradient.
5. Compute the metrics, look up the tolerance, record pass or fail.

The default matrix (`default_cases`) covers lengths 1, 7, 64, 129 (quick) plus 512 and 1024 (full); head dimensions 8–128; fp64, fp32 and bf16; causal and non-causal; tile sizes 32 × 48, which deliberately do not divide the lengths; input magnitudes 0.01 to 30; batch/head shapes `(1,1)` and `(3,5)`; random key padding; and fully-masked rows. The full run made **795 comparisons** (`python lab.py correctness --full`, run `20261011-194506_correctness`).

11.4 The tolerance policy, and how it got that way

The current policy, against `max_abs_normalized`:

- **fp64:** forward 1e-12, gradients 1e-10.
- **fp32:** forward 2e-5, gradients 2e-4, **or** `4·u·max|S|` (forward) / `16·u·max|S|` (gradients) if larger, where `max|S|` is the largest logit and `u = 2⁻²⁴`.
- **bf16 / fp16:** at most **2 × the error of a mixed-precision baseline** plus a small slack (4e-3 forward, 1.6e-2 gradients for bf16). The baseline is naive attention computed in fp32 on the same rounded inputs, with results rounded to bf16.

Neither of the two non-obvious parts was in the first version. Both came from failures that were investigated rather than "fixed" by loosening.

Investigation 1: the bf16 baseline that was too lenient

First version: the bf16 tolerance was "2 × the error of naive attention *computed in bf16*". The reasoning was that low-precision methods should be no worse than the obvious low-precision implementation.

What the run showed: all methods passed, but the naive-in-bf16 baseline's own error was huge and grew with input magnitude. The breakdown (L = 96, d = 32):

input magnitude	naive, all in bf16: fwd error	tiled / SDPA (fp32 accumulators): fwd error
1	~0.005	~0.002
3	0.059	~0.002
10	**0.147**	~0.002
30 (full run)	**0.377**	~0.003

Gradient errors were worse, up to 0.68 at magnitude 10 and 2.24 at magnitude 30.

Cause: at magnitude 10 the logits are in the hundreds. bf16 has 8 significant bits, so neighbouring representable numbers near 256 are 1 apart, and a logit can be off by up to 0.5. The softmax weight `exp(s − m)` is then off by up to `e^0.5 ≈ 1.65×`. The kernels that keep logits and running sums in fp32 do not suffer this.

Why the first policy was wrong: "2 × a 15 % error" accepts almost anything. A tolerance based on a bad baseline is meaningless.

Fix: use a **mixed-precision baseline** whose error is essentially the unavoidable rounding of the *output* to bf16 (about 2–3e-3). Under the new policy every fp32-accumulating method passes, and **naive-in-bf16 fails 14 of 53 cases**. That is now reported as a finding, not hidden: *computing attention entirely in bf16 is inaccurate at realistic logit scales, which is why FlashAttention and the CPU kernel keep these values in fp32*. A regression test pins it (`test_finding_naive_in_pure_bfloat16_degrades_with_logit_magnitude`).

Investigation 2: the fp32 "failure" that was the dtype's own floor

What happened: after the bf16 fix, one test failed: `sdpa_flash_cpu` in fp32 at input magnitude 10 had forward error **2.04e-5**, against a tolerance of **2e-5**.

The easy move would have been to change 2e-5 to 3e-5. Instead a hypothesis was formed. A logit `s` in fp32 can only be stored to about `u·|s|` absolute precision, and that becomes a relative error in its softmax weight of the same size. If so, the error should scale with the **largest logit**, for *every* implementation, including the naive formula. Measured (L = 64, d = 32, fp32), error divided by `u·max|S|`:

input magnitude	max|S|	naive	sdpa_math	sdpa_flash_cpu	tiled
1	4.6	1.3	1.5	1.4	1.1
3	41	0.6	0.7	0.6	0.6
10	459	0.7	0.2	0.7	0.2
30	4135	0.1	0.2	0.1	0.1

At magnitude 10, **the naive formula in fp32 had exactly the same error, 2.04e-5**, as the fused kernel. The error tracked the logit scale for every implementation. (It saturates at magnitude 30 because the softmax becomes nearly one-hot and stops being sensitive.)

Conclusion: not a kernel bug, but the fp32 floor for large logits. **Fix:** add the term derived from the error model, `4·u·max|S|`, which only matters for large logits. The fixed 2e-5 threshold still applies to ordinary inputs. A regression test pins the evidence (`test_finding_fp32_error_floor_scales_with_logit_magnitude_for_every_implementation`).

11.5 Results of the full run

method	dtype	cases	pass fwd	pass grad	worst fwd err	median fwd err
sdpa_math / sdpa_flash_cpu / tiled / tiled_recompute	fp64	49 each	100 %	100 %	≤ 3.7e-15	~3–5e-16
all five exact methods	fp32	57 each	100 %	100 %	≤ 6.5e-5 (magnitude 30)	~2.5–3e-7
the four fp32-accumulating methods	bf16	53 each	100 %	100 %	≤ 4.0e-3	~2.1e-3
naive (everything in bf16)	bf16	53	**73.6 %**	94.3 %	**0.377**	5.7e-3

Two more facts from the same run:

- **Unstable softmax demo** (fp32, softmax without subtracting the max): 0 % NaN at max logit 3.4 and 83.8, **17.9 % NaN at max logit 335**, 40.3 % at 1340. The stable version's error stayed below 2e-7.
- The exact methods' fp32 output error stayed flat (about 6e-7 to 2e-6) from L = 128 to 2048 in the scaling run; it did not grow with length.

Key points to remember

- Compare with tolerances: correct implementations differ in the last bits.
- The reference is fp64 naive *on the same rounded inputs*.
- The metric is `max|err| / max|ref|`, and relative errors use an explicit floor.
- fp64 1e-12, fp32 2e-5 (or `4·u·max|S|`), bf16 relative to a mixed-precision baseline.
- Investigation 1: a baseline that is itself bad (naive-in-bf16, error up to 38 %) makes tolerances meaningless.
- Investigation 2: when a check fails, test a hypothesis about the cause. Here the fp32 logit-rounding floor was shared by every implementation.
- All fp32-accumulating exact methods passed every check. Naive-in-bf16 failed 14 of 53.

Chapter 12 — The complexity model: theory you can test

> **What you will learn:** how the project turns big-O statements into concrete numbers for each method and configuration; why those numbers are always labeled "theory"; how the theory was *checked* against counters built into the algorithms; and how the memory guard uses it.

12.1 Purpose

The brief insisted that **theory and measurement must stay distinguishable**. The project therefore has one module that produces only theoretical estimates, and every estimate it produces is stored under a name beginning with `theory_`. A measured value never carries that prefix, and a theoretical value never appears without it.

> **In the code:** `src/attention_lab/complexity/theory.py`, function `theoretical_cost(method, shape, params)`.

12.2 What it computes

For a `Shape(B, H, L, d, dtype, causal)` and a method's parameters, it returns (among others):

- `score_entries`: how many query–key logits the implementation **actually evaluates**, including masked-but-computed entries (local window wastes about half; padded block slots also count);
- `matmul_flops`: `2 × 2 × entries × d` for softmax methods (QK^T plus PV);
- `elementwise_ops`: rough counts of exp, max, divide (reported separately, because they cost very differently from matmul FLOPs on a CPU);
- `aux_fwd_bytes`: peak intermediate storage of one inference forward, excluding inputs and outputs;
- `fwd_alloc_bytes = aux_fwd_bytes` + the output tensor, comparable with the measured tensor peak;
- `saved_for_bwd_bytes`: what autograd keeps between forward and backward;
- `asymptotic_time`, `asymptotic_aux_memory`: the big-O strings.

Method-specific details:

- **tiled**: counts tiles *exactly*, loop by loop, including ragged final tiles and causal tile skipping. Autograd-tiled saves all tiles; recompute saves only `lse`.
- **local_window**: `⌈L/C⌉ · C · (C + w_l + w_r)` scores, and `band_pairs` (the useful ones) counted exactly.
- **block_sparse**: builds the actual layout with `make_layout`, then counts `rows × b × K_max × b` plus dense global rows, the layout density and the number of active blocks.
- **linear_elu**: zero scores for non-causal; for causal, `C × C` per chunk plus the `d × d` states.

12.3 Checking theory against the code

A model is only useful if it is right, so the algorithms can report what they actually did. They accept an optional `stats={}` dictionary and fill in counters as they run (`score_entries`, `tiles`, `band_entries`, `active_blocks`, `k_max`).

Scientific tests (`tests/scientific/test_complexity_and_measurement.py`) then assert **exact equality** between the theory and the counters:

Test	Checks
`test_tiled_score_entries_theory_equals_instrumented`	8 cases incl. ragged tiles (L = 70 with 16 × 24 tiles; L = 100 with 32 × 7) and causal
`test_local_window_entries_theory_equals_instrumented_and_scale_linearly`	entries and band pairs; 4× longer input ≤ 4.4× entries
`test_block_sparse_entries_theory_equals_instrumented`	entries and active blocks, incl. ragged L = 130 causal
`test_naive_entries_are_quadratic`	doubling L quadruples entries
`test_linear_noncausal_has_no_quadratic_term`	0 score entries; doubling L doubles FLOPs

> **What actually happened:** the first version of the tiled formula counted `tiles × q_tile × kv_tile`, which overcounts when the last tile is ragged. It was rewritten to loop over the real tile boundaries so that it equals the counter exactly. Separately, local_window's `band_entries` statistic was first computed by building a dense `L × L` band mask, which is `O(L²)` work inside a supposedly `O(L)` method. It was replaced by an `O(L)` formula (`band_entry_count`) before any benchmark used it.

12.4 Checking theory against measured memory

Chapter 14 compares `theory_fwd_alloc_bytes` with the measured tensor peak. For naive attention the model predicted 0.875, 3.25, 12.5, 49.0 and 194 MiB at L = 128, 256, 512, 1024 and 2048. The measurements were 0.88, 3.26, 12.52, 49.05 and 194.09 MiB. For methods whose internals are a black box (the fused kernel's per-thread scratch), the model can only give a lower bound, and it says so.

12.5 Use in the memory guard

Before running any configuration, the benchmark runner asks the model for an `estimated_peak_bytes` for each mode (forward, forward+backward) and refuses modes that would exceed `min(50 % of currently available RAM, max_mem_gb)`. The estimate deliberately overcounts (backward is assumed to need another set of `L²` temporaries). Refused configurations are **recorded** with the reason, never silently dropped (Chapter 13).

Key points to remember

- `theory_*` fields are estimates, always labeled; measured fields never carry that prefix.
- The model counts the scores each implementation *actually* computes, including masked waste and padding.
- Algorithms expose `stats=` counters; tests assert theory == counters exactly.
- The naive memory model matched measurement to within 0.1 MiB.

- The memory guard uses the model to refuse (and record) configurations that could exhaust RAM.

Chapter 13 — The benchmarking system: measuring time and memory honestly

> **What you will learn:** how one benchmark configuration is run, step by step; how timing is done and summarized; the three memory instruments and what each one really measures (and how the main one was validated); the memory guard; isolated worker processes; the sweep scheduler; and the story of the shared machine, which led to the background-load monitor, CPU-time measurements and single-threaded runs.

13.1 One configuration, step by step

> **In the code:** `src/attention_lab/benchmarking/runner.py`, `run_one(BenchConfig)`.

A configuration is: method + parameters + `B, H, L, d` + dtype + causal + threads + warm-up/repeats + seed. The runner executes, **in this order** (chosen so that the instruments do not contaminate each other):

1. **Theory** and **environment** are recorded (CPU, cores, RAM, OS, Python/torch/numpy versions, thread settings).
2. **Memory guard** (per mode): modes whose estimate exceeds the budget are skipped and recorded.
3. **Inputs** `q, k, v, dout ~ N(0, 1)` from a seeded generator.
4. **Timing** of each mode, wrapped in the background-load monitor:
   - `fwd`: forward under `torch.inference_mode()`;
   - `fwd_bwd`: forward with gradient-requiring inputs, then `out.backward(dout)`.
5. In isolated mode, the OS **peak working set** is read after each mode.
6. **Tensor peak** of each mode, from a separate untimed call under the profiler.
7. **Error** against the fp64 naive reference. This runs last, because the reference is the most memory-hungry step.

The result is one JSON record with every number and every setting.

13.2 Timing

> **In the code:** `src/attention_lab/benchmarking/timing.py`, `time_callable`.

- The first call is timed separately (`first_call_s`), because it includes one-off costs such as allocator growth and oneDNN setup.
- `warmup` untimed calls follow, then `repeats` timed calls with `time.perf_counter_ns()`, a monotonic, high-resolution clock.
- Python's garbage collector is paused during the timed loop.
- The summary reports the **median** (the headline number), q1, q3, IQR, MAD, min, max, mean and coefficient of variation, plus every raw sample. The minimum is never reported alone, because it describes the luckiest run rather than typical behaviour.

- A time budget (`max_seconds_per_mode`) stops repetition early for slow configurations, and the actual number of repeats done is recorded.
- **Process CPU time** per call is recorded too (explained in §13.7).

No test ever asserts a timing threshold. A test that fails because the machine is busy would teach nothing. Correctness and performance are kept separate.

13.3 Three memory instruments

Instrument	What it measures	What it misses
Tensor peak (`memory.tensor_peak_bytes`)	peak bytes allocated through PyTorch's CPU allocator *during one call*, reconstructed from profiler `[memory]` events	memory existing before the call (inputs); scratch memory allocated inside MKL/oneDNN/OpenMP outside PyTorch's allocator; Python objects
Process peak Δ (`memory.ProcessPeak`)	Windows' peak working set of the whole process minus its size before the first call	it is a *lifetime high-water mark*, so it needs a fresh process per configuration and the fwd+bwd value is really max(fwd, fwd+bwd); it includes allocator caches; page granularity
Theory (`complexity/theory.py`)	analytical estimate	everything not modeled; never reported as measured

How the tensor peak works. The profiler with `profile_memory=True` records an event every time PyTorch's CPU allocator hands out memory (`+bytes`) or takes it back (`-bytes`), with a timestamp. Sorting these events by time and keeping a running sum gives the memory in use at each moment. Its maximum is the peak.

How it was validated. Before any result relied on it, it was tested on a known pattern: allocate 10 MiB, allocate 5 MiB, free the 10 MiB, allocate 3 MiB, free everything. The true peak is 15 MiB (10 + 5), and the instrument reported **exactly 15.0 MiB**, with a net of 0. Computing one 16 MiB score matrix from 1 MiB of inputs reported 17.0 MiB. Both are now tests (`test_tensor_peak_bytes_exact_on_known_allocation_pattern`, `test_tensor_peak_sees_materialized_scores_and_tiling_reduces_it`).

13.4 The memory guard

Before each mode, the runner compares the theoretical estimate with `budget = min(mem_fraction × available RAM, max_mem_gb)` (defaults 50 % and a few GB). If forward fits but forward+backward does not, it runs forward only and records `status = "partial"` with `modes_skipped_memory`. If nothing fits, it records `status = "skipped_memory"` with the estimate and budget in the reason.

> **What actually happened:** the first guard decided for the whole configuration using the forward+backward estimate. That would have skipped useful forward measurements at large L, so it was changed to decide **per mode**. Then the scheduler was briefly made to stop growing a series after a "partial" result. That was wrong, because forward-only may still fit at the next length, so it was reverted: a series stops only after a full `skipped_memory`. The guard did its job repeatedly. For example, at L = 4096 the naive and math methods (about 0.8 GB for forward alone) were skipped when only 0.6–0.8 GB was budgeted, and the free RAM varied from run to run because of other programs.

13.5 Isolated workers

The OS peak working set can only be reset by starting a new process, so the runner can execute a configuration in a fresh Python process (`run_isolated` → `python -m attention_lab.benchmarking.worker`, config in on stdin, JSON record out on stdout). The parent sets `OMP_NUM_THREADS` and `MKL_NUM_THREADS` **before** torch is imported in the child, which controls threads more faithfully than `set_num_threads` alone. Isolation also protects the main process if a configuration runs out of memory.

The cost is that starting Python and importing torch took about 5 s per worker on this machine.

> **What actually happened:** each worker initially also spent 1–2 s asking PowerShell for the CPU name. The parent now passes the name through an environment variable (`ATTENTION_LAB_CPU_NAME`).

13.6 The scheduler

> **In the code:** `src/attention_lab/benchmarking/scheduler.py`; specs in `experiments/configs/*.yaml`.

A YAML spec lists methods (with params), a grid (`B, H, L, d, dtype, causal, threads`), repeats, budgets and a **hypothesis** sentence. The scheduler:

- expands the grid and **sorts by L**, so every method is measured small before anything large runs;
- runs configurations one at a time (in-process or isolated);
- **appends each result to `results.jsonl` immediately**, so an interrupted sweep keeps everything measured so far;
- stops growing a series (same method/shape, larger L) after a memory skip, an error, a timeout, or a median time above `stop_if_slower_than_s`, recording later configurations as `skipped_after_budget` with the reason;
- optionally waits, for a bounded time, for the machine to go quiet before each configuration (`wait_for_quiet`).

13.7 The shared-machine story

> **What actually happened.** The first full exact-scaling sweep ran with 4 threads in isolated workers. Its numbers looked wrong. At L = 512, `tiled` and `tiled_recompute`, whose forward passes run *identical* code, measured 13.1 ms and 34.7 ms. Causal tiled (half the work) was slower than non-causal tiled. A quick check showed **all 12 logical CPUs at 100 %**, while the benchmark used only 4 threads.
>
> Listing the Python processes showed why: **workloads from other projects** were running at the same time (three training scripts, another Transformer experiment, a chess-engine server with its engine processes, and a test run). They were not ours, so they were not touched. But they made fine-grained timing unreliable: a 4-thread OpenMP computation competing with up to 11 busy logical CPUs is slowed by different amounts for different methods.

The response, in steps:

1. **Measure the load.** `benchmarking/load.py` adds a `LoadMonitor`: a background thread samples system CPU use and this process's CPU use every 0.2 s during each timed region, and records `other_load_cores` (how many logical CPUs *other* processes kept busy). Records with more than 1 other busy core are flagged `contended`. *Bug found while testing it:* short timed regions finished

before the first sample, giving no data. Fixed by priming the counters on entry and always taking a final sample on exit.

2. **Record CPU time.** Process CPU time per call excludes time spent waiting for a core. *Bug found:* on Windows, `time.process_time` ticks in about 15.6 ms steps, so per-call samples of fast calls read 0.0. Fixed by measuring CPU time over the whole timed loop and dividing by the number of calls.
3. **Wait for quiet, within limits.** `wait_for_quiet` waits up to a timeout for the load to drop. The load stayed high for hours, so the timeout was reduced from 60 s to 10 s per configuration; the load is recorded either way.
4. **Run the definitive timing sweeps single-threaded.** With one thread, no OpenMP barrier can stall on a descheduled sibling thread, so contention degrades every method more evenly. The check that this worked: CPU time per call matched wall time within a few percent (for example 509 ms CPU vs 516 ms wall), and the fitted exponents became physically sensible (1.92–2.03, instead of up to 3.0 in the contaminated run).
5. **Never mix contended and clean claims.** The first sweep's *memory* data is used (memory is not affected by CPU contention). Its *timing* data is reported as contaminated and not used as evidence.

How noisy is the machine? The same naive forward at L = 2048 on one thread measured **392 ms in one sweep and 784 ms in another** an hour later. The book therefore compares methods only *within* a sweep (measured close together in time), and states background load next to every timing table.

13.8 What a GPU benchmark would need

None of this is GPU-specific, and adding a GPU would mean: synchronizing the device before and after each timed call (GPU work is asynchronous), measuring peak memory with `torch.cuda.max_memory_allocated` after resetting it, warming up kernel autotuning, and recording the GPU model and driver. CPU and GPU numbers must never share a plot without labels. The algorithms themselves need no changes.

Key points to remember

- Order of measurement: guard → inputs → timing (+ load) → process peak → tensor peak → error.
- Median + IQR, warm-up, monotonic clock, GC paused, raw samples kept, and no timing assertions in tests.
- Three memory instruments with different scopes. The tensor peak was validated byte-exactly (15.0 MiB on a 15 MiB pattern).
- The guard decides per mode and records every skip.
- Isolated workers give clean process-peak numbers and set threads before torch loads (about 5 s overhead each).
- The machine was shared. The load was measured, CPU time recorded, definitive sweeps run on 1 thread, and contaminated timings discarded.

Chapter 14 — Results I: the exact methods (correctness, memory, time)

> **What you will learn:** what the experiments showed about the five exact implementations: that they agree numerically; how their memory scales (measured); how their time scales (measured on one thread); and how to read these results without over-reading them.

Conditions for everything in this chapter: B = 1, H = 4, d = 64, fp32, non-causal unless stated. Runs: `20261011-191952_scaling_exact` (memory, isolated workers, 4 threads, timing contaminated and not used) and `20261011-210946_scaling_exact_timing` (timing, 1 thread, in-process; background load 4.5–10.6 other logical CPUs). Figures are in `experiments/reports/<run>/`.

14.1 They compute the same function

From Chapter 11: all exact methods agree with the fp64 reference to about 10^{-15} in fp64 and pass every fp32 check (forward and gradients) across 57 cases each. In the scaling run, the fp32 output error against fp64 stayed between 6×10^{-7} and 2×10^{-6} from L = 128 to 2048. It does not grow with length.

14.2 Memory: measured peak tensor allocation per call

L	naive fwd	sdpa_math fwd	flash fwd	tiled fwd	naive fwd+bwd	flash fwd+bwd	tiled (autograd) fwd+bwd	tiled_recompute fwd+bwd
128	0.88	0.69	0.22	1.13	1.75	0.75	1.13	2.01
512	12.5	9.5	1.07	1.78	25.0	3.51	7.03	3.78
1024	49.0	37.0	3.27	2.54	98.0	9.02	22.0	5.81
2048	194.1	146.0	4.29	4.57	388.0	14.0	76.4	9.88
4096	skipped	skipped	6.32	8.63	skipped	24.1	skipped	18.0

(MiB. "skipped" = refused by the memory guard at the time.)

Fitted exponents of memory vs L (L ≥ 512): naive 1.98 / 1.98 (fwd / fwd+bwd); sdpa_math 1.97; flash 0.81 / 0.90; **tiled (autograd) 0.77 / 1.72**; **tiled_recompute 0.84 / 0.75**.

What this shows:

1. **Materializing methods are quadratic in memory, exactly as predicted.** The theory said 0.875 / 12.5 / 49.0 / 194 MiB for naive; the measurements were 0.88 / 12.5 / 49.0 / 194.1.
2. **Tiling and fusion make forward memory almost linear:** 4.3–4.6 MiB at L = 2048, against 194 MiB for naive, a 42–45× reduction.
3. **Autograd through the tile loop brings quadratic memory back for training** (exponent 1.72, 76 MiB at 2048).
4. **The recompute backward keeps training memory linear** (0.75; 9.9 MiB at 2048, 39× less than naive).
5. At the process level (OS peak working set, isolated workers), naive needed 446 MiB more than its starting size at L = 2048 for fwd+bwd, against 57 MiB for the flash kernel. About 35–40 MiB of that

is fixed overhead, visible even at L = 128.

Figure 14.1 (below) shows all methods on one log-log plot against dashed L and L² guides. Naive and math track the L² line; flash and recompute track the L line; autograd-tiled bends toward L² in fwd+bwd.

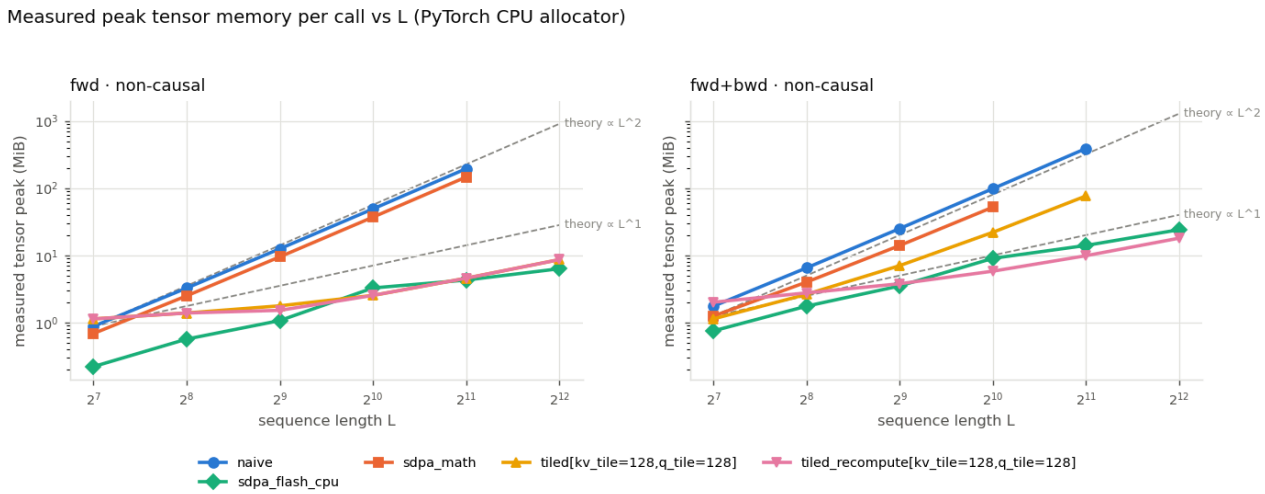


Figure 14.1 — Measured peak tensor memory per call vs L, all exact methods on shared log-log axes (run 20261011-191952_scaling_exact). Dashed grey lines are theory slopes ∝ L and ∝ L².

14.3 Time: measured on one thread

Median milliseconds per call:

L	naive fwd	sdpa_math fwd	flash fwd	tiled fwd	recompute fwd	naive fwd+bwd	math fwd+bwd	flash fwd+bwd	tiled fwd+bwd	recompute fwd+bwd
256	4.1	5.3	2.6	5.1	5.5	16.0	10.5	10.7	19.6	20.1
512	28.1	23.6	8.4	21.8	23.3	82.4	54.7	36.4	75.1	68.7
1024	105.5	86.5	31.5	73.4	87.7	304.4	210.0	135.9	353.1	268.7
2048	392.4	355.6	146.7	313.0	345.0	1411.2	803.7	503.9	1847.0	979.9
4096	1540.4	1512.9	515.6	1445.4	1600.3	(guard)	(guard)	2017.0	14489.5	4637.2

Fitted exponents of time vs L (wall clock / CPU time): every exact method's forward is 1.92–2.03 (CPU 1.97–2.10). Forward+backward is 1.91–2.05 for all of them except **autograd-tiled at 2.52**.

What this shows:

1. **Every exact method is quadratic in time, including the memory-efficient ones.** Tiling moves the scores into small tiles; it does not reduce how many scores there are.
2. **The fused CPU kernel is the fastest exact method at every length**: about 3× faster than naive in forward (516 vs 1,540 ms at L = 4096) and 2.8× in fwd+bwd at 2048. Same arithmetic, so the gain is fusion (no `L²` tensor ever written to RAM) and no Python dispatch per step.
3. **On one thread, our Python tiled forward was *not* slower than naive.** It was 6–30 % faster at L ≥ 512 (313 vs 392 ms at 2048). Its per-tile dispatch overhead is offset by avoiding three `L²` tables that overflow the 16 MB L3 cache. *This depends on threads:* in the saved 4-thread runs, tiled was much slower than naive (6.3 vs 0.7 ms at L = 256 in the smoke run; 13.1 vs 5.2 ms at 512 in the contended run), and Chapter 15's thread experiment explains why.
4. **Storing costs more than recomputing.** tiled_recompute does 25 % more backward arithmetic than autograd-tiled, yet fwd+bwd took 980 ms vs 1,847 ms at 2048, and 4.6 s vs 14.5 s at 4096.

Autograd-tiled's super-quadratic exponent (2.52) indicates growing memory-system cost from keeping every tile alive.

5. **Fusion still wins over the best Python version:** flash fwd+bwd 504 ms vs recompute 980 ms at 2048. The algorithm is the same; the execution is not.

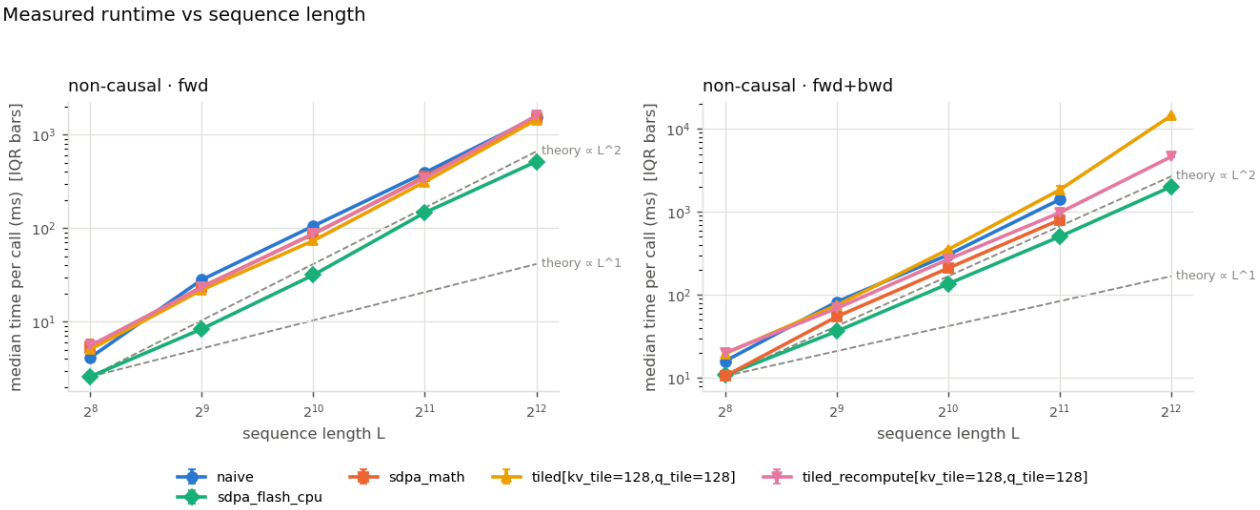


Figure 14.2 — Measured median time per call vs L on one thread (run 20261011-210946_scaling_exact_timing), with IQR bars. Every exact method runs parallel to the L² guide.

14.4 How *not* to read these results

- They are **CPU results**. A GPU has different cache sizes, bandwidth, launch costs and bf16 support, and the ratios would differ.
- The absolute times include background load (flagged `contended`). Ratios measured in the same sweep, close together in time, are more trustworthy than absolute numbers.
- "Tiled ≈ naive on one thread" is a statement about *this* Python implementation, *this* tile size (128) and *one* thread, not about tiling in general.

Key points to remember

- All exact methods agree numerically (≈ 1e-15 fp64, ≈ 1e-6 fp32).
- Memory: naive/math ∝ L² (194 MiB at 2048); flash/tiled ∝ L (≈ 4.5 MiB); training: autograd-tiled ∝ L^1.72, recompute ∝ L^0.75.
- Time: everything exact ∝ L² (1.9–2.1).
- Fused kernel ≈ 3× faster than naive; recompute faster than storing (980 vs 1,847 ms).
- Single-thread Python tiled ≈ naive or better; multi-thread it is much worse (Chapter 15).

Chapter 15 — Results II: approximate methods, tile sizes, threads, shapes and number formats

> **What you will learn:** when the approximate methods actually beat the best exact kernel, and when they do not; the surprising memory result; how tile size shapes the Python tiled kernel's speed and memory; how each method responds to more threads; and how head dimension, batch and bf16 affect time.

15.1 Approximate methods vs the fused exact kernel (E2)

Run: `20261011-212017_scaling_approx`: isolated workers, 1 thread, B = 1, H = 4, d = 64, fp32. local_window w = 64; block_sparse b = 64 (±1 local, global 0, 1 random); linear_elu chunk 64. 55 of 56 configurations measured. The fused kernel at L = 16,384 non-causal was skipped by the time budget, because its fwd+bwd at 8,192 took 8.4 s. Background load was mostly 6–11 other logical CPUs, so single points are noisy and trends are reliable.

Median forward time (ms), non-causal / causal:

L	flash (exact)	local_window	block_sparse	linear_elu
256	2.7 / 2.6	9.7 / 6.9	8.2 / 10.7	2.0 / 5.3
1,024	32.7 / 27.5	37.4 / 21.8	57.0 / 44.4	8.4 / 19.7
2,048	131.2 / 82.5	86.4 / 50.7	143.9 / 97.1	16.6 / 48.6
4,096	439.4 / 269.4	92.2 / 71.8	147.4 / 109.3	25.4 / 69.1
8,192	2,446.1 / 1,033.5	392.2 / 229.4	366.1 / 277.1	60.0 / 176.1
16,384	(budget) / 3,959.4	779.7 / 191.1*	962.4 / 742.6	133.3 / 361.2

* measured during a lull in background load (2.7 other cores vs about 10 for neighbouring points); treat as an outlier.

Fitted time exponents: local_window 0.82–1.02, block_sparse 0.89–0.97, linear_elu 0.90–0.97 (linear in L); flash 1.73–1.99 (quadratic).

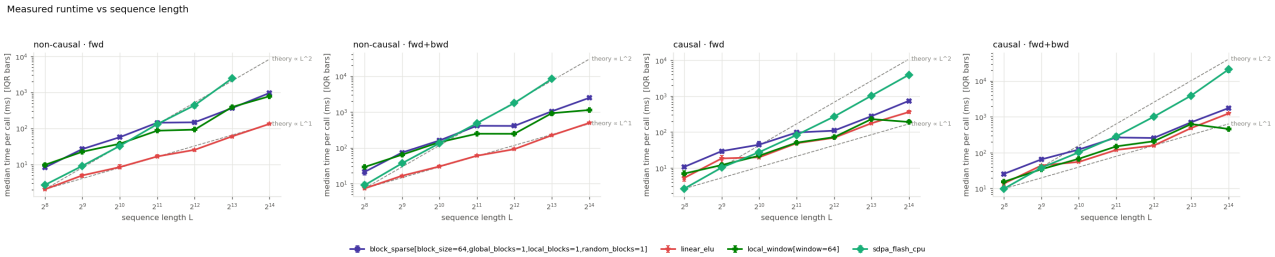


Figure 15.1 — Measured runtime vs L for the approximate methods and the fused exact kernel (run 20261011-212017_scaling_approx).

Findings.

1. **Linear in theory, linear in measurement**: every approximate method's time grows about ∝ L.

2. **But they only win beyond a crossover.** In non-causal forward, linear_elu is faster at every measured length, local_window from about 2,048 tokens, and block_sparse is roughly even at 2,048 and clearly faster from 4,096. Below about 1,024 tokens, local_window and block_sparse are **1.1–3.6× slower** than the fused exact kernel, even though they evaluate far fewer scores: padding, `unfold` copies, gathering and masked waste cost more than they save at short lengths.
3. **At long lengths the gap is large**: at L = 8,192 non-causal, flash takes 2.4 s, local_window 0.39 s (6.2×) and linear_elu 0.06 s (41×).
4. **Causal attention narrows the gap**, because the fused kernel skips tiles above the diagonal: its causal forward is 1.2× faster than its non-causal one at L = 1,024, rising to 2.4× at 8,192.

15.2 The memory surprise

Measured tensor peak per forward call (MiB), non-causal:

L	flash (exact)	local_window	block_sparse	linear_elu	naive (Chapter 14)
1,024	**1.6**	19.0	29.4	4.1	49.0
4,096	**4.6**	74.5	122.6	16.2	(guard)
16,384	—	296.3	496.1	64.6	—

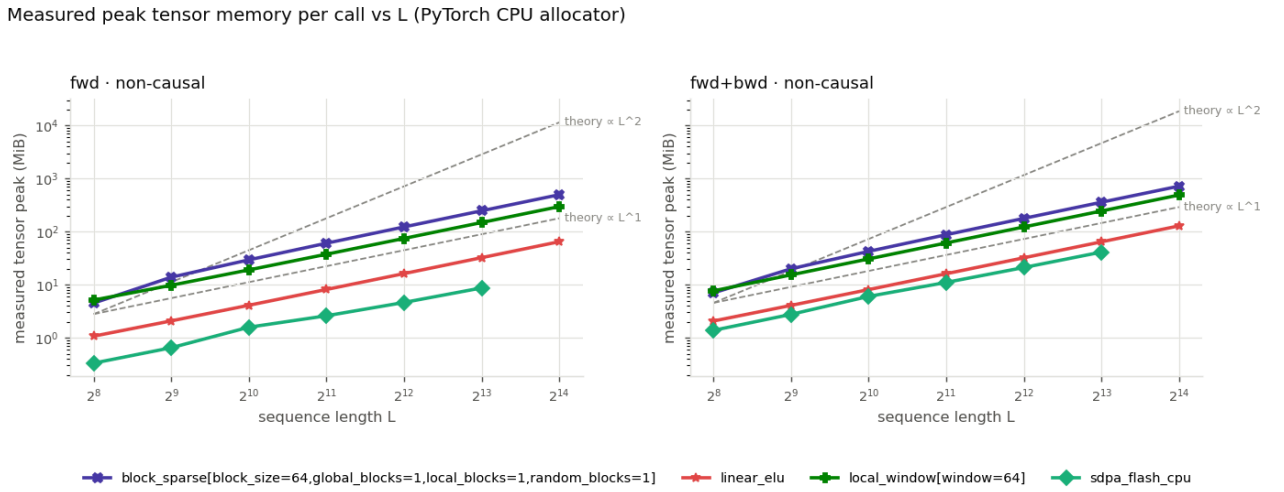


Figure 15.2 — Measured peak tensor memory, approximate methods vs the fused exact kernel.

"Approximate attention saves memory" is false relative to a fused exact kernel. All three approximate methods allocate *more* than `sdpa_flash_cpu` at every length: 16× (local) and 27× (block sparse) more at 4,096 tokens. They materialize O(L·window) score tables or gathered key blocks, while the fused kernel streams small tiles and allocates little beyond its output. The approximate methods save memory only relative to **materializing** exact methods: at L = 1,024, local_window needs 19 MiB against naive's 49 MiB, and the gap widens with L. The theory model predicted this (Chapter 10 §10.5). The measurement confirms it.

This is Principle B of the project in its purest form: **memory efficiency, computational efficiency and asymptotic class are three different things.**

Deviation from full softmax (relative Frobenius norm, random inputs, non-causal): local_window grows from 1.15 to 3.85 between L = 256 and 2,048, block_sparse from 0 (its layout is dense at L = 256) to 2.32, and linear_elu stays about 0.75–0.79. On random inputs attention is diffuse, so dropping most keys

changes outputs by more than their own size. This number describes how much the *function* changes. Quality is measured by the probes and tasks.

15.3 Tile size in the Python tiled kernels (E3)

Run: `20261011-215718_tile_sizes` (L = 2,048, 1 thread). **Heavily contended** (about 11 other logical CPUs for every config), so compare only within this run. (naive measured 784 ms here, against 392 ms in Chapter 14.)

implementation	tile	Python tile iterations	fwd (ms)	fwd tensor peak (MiB)	fwd+bwd (ms)	fwd+bwd peak (MiB)
tiled (autograd)	32	4,096	1,941	4.1	**41,418**	79.0
tiled (autograd)	64	1,024	704	4.3	4,693	77.1
tiled (autograd)	128	256	**528**	4.6	4,177	76.4
tiled (autograd)	256	64	639	5.6	2,190	75.8
tiled (autograd)	512	16	899	14.6	1,391	79.1
tiled_recompute	32	4,096	1,478	4.1	4,523	8.3
tiled_recompute	128	256	466	4.6	1,362	9.9
tiled_recompute	512	16	442	14.6	1,273	33.1
naive	—	—	784	194.1	2,258	388.0
sdpa_flash_cpu	—	—	152	2.6	773	11.0

1. **Forward time is U-shaped in tile size, with the minimum at 128.** Tiny tiles drown in Python and dispatcher overhead (4,096 iterations at tile 32, 3.7× slower than tile 128). Big tiles slow down again: a 512 × 512 tile for 4 heads is 4 MiB, against the 512 KB per-core L2 cache, while tile 128 is 256 KiB. The cache explanation fits but was not measured with hardware counters.
2. **Forward memory stays flat until the tile² term dominates** (4.1 → 4.6 → 14.6 MiB).
3. **Tile size cannot fix autograd's training memory**: 76–79 MiB at every tile size. Small tiles are disastrous for autograd's backward time (41 s at tile 32), because it records and replays about 10 graph nodes per tile.
4. **The recompute backward stays at 8–10 MiB** for tiles ≤ 128 and is 1.1–9.2× faster than autograd at equal tile size.
5. Even at its best tile, Python tiling is **3.1–3.5× slower than the fused kernel**.

15.4 Threads (E4)

Run: `20261011-221221_threads` (L = 2,048, isolated workers, threads 1/2/4/6). Background load about 9–11 other logical CPUs for naive/flash/tiled, about 3–9 for linear/local. Indicative.

method	1 thread (ms)	2	4	6	speed-up at 2 / 4 / 6
sdpa_flash_cpu	124.3	85.9	42.7	35.1	1.45 / 2.91 / **3.54**
naive	402.9	319.2	234.3	230.0	1.26 / 1.72 / 1.75
linear_elu	7.8	4.5	4.0	3.5	1.74 / 1.95 / 2.23
local_window	35.6	29.1	23.4	42.0	1.22 / 1.52 / 0.85
tiled (Python)	368.9	564.3	839.1	1,772.1	**0.65 / 0.44 / 0.21**

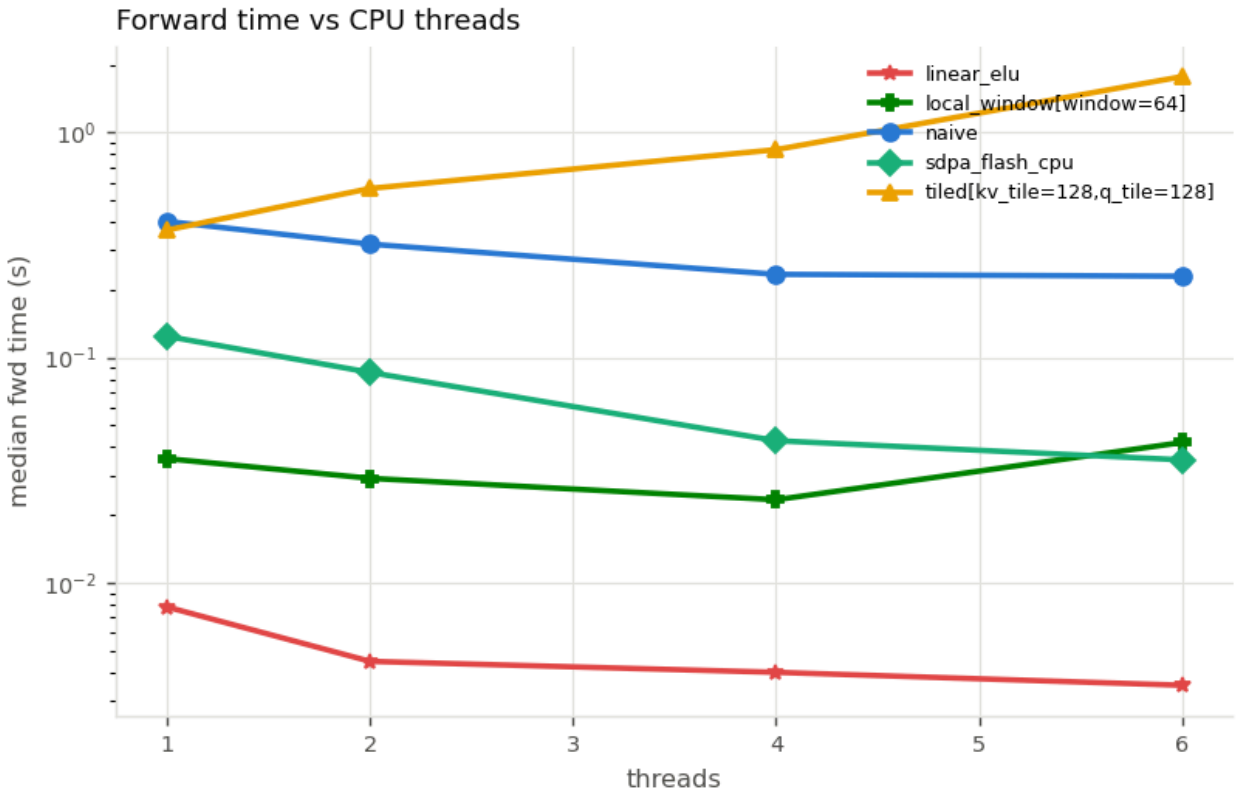


Figure 15.3 — Forward time vs number of threads at L = 2,048.

1. **The fused kernel scales best** (3.5× on 6 cores): its parallel units are large blocks of work.
2. **Naive scales poorly beyond 4 threads** (1.75×): its passes over L^2 tensors are limited by memory bandwidth, and more cores do not add bandwidth.
3. **The Python tiled code gets slower with every added thread**, 4.8× slower at 6 threads than at 1. Its CPU time per call rose from 342 ms to 3,692 ms: the extra threads burned CPU synchronizing on operations too small to split, and under background load a single descheduled thread stalls every operation. This explains why the 4-thread runs showed tiled far slower than naive, while the 1-thread runs showed it slightly faster.
4. local_window's drop at 6 threads coincides with a large load increase, so it cannot be interpreted on its own.

15.5 Head dimension, batch and number format (E5)

Run: `20261011-222558_shapes_dtypes` (L = 1,024, H = 4, 1 thread; B ∈ {1, 4}; d ∈ {16, 32, 64, 128}; fp32 and bf16). 46 configurations measured; naive fwd+bwd at B = 4, d = 16 and 32 was skipped by the memory guard. Background load was 7–11 other logical CPUs throughout, so individual cells are noisy.

Median forward time (ms), B = 1:

d	naive fp32	naive bf16	flash fp32	flash bf16	tiled fp32	tiled bf16
16	102	1,009	15.0	74.8	60.1	44.4
32	74	1,230	18.2	64.1	52.4	75.0
64	128	1,587	77.1	119.0	165.9	80.2
128	132	3,152	69.8	168.6	185.7	198.3

Median forward time (ms), B = 4: naive fp32 332–561 vs **bf16 4,234–12,529**; flash fp32 57–284 vs bf16 312–514; tiled fp32 266–866 vs bf16 324–922.

1. **bf16 is never faster than fp32 on this CPU, and for naive attention it is catastrophically slower**: 10–30× (for example 1,587 vs 128 ms at B = 1, d = 64, and 12.5 s vs 0.41 s at B = 4, d = 128). This AVX2 processor has no native bf16 arithmetic, so every bf16 elementwise operation on the large L^2 tensors is converted to and from fp32.
2. **The fused kernel is 1.5–5.5× slower in bf16**, which is less severe because it converts small tiles internally.
3. **Our tiled code is roughly unaffected** (ratios 0.5–1.4×, within the noise), because it casts each tile to fp32 accumulators anyway (Chapter 8).
4. B = 4 is 1.6–5.5× slower than B = 1 (about 4× expected for 4× the work), but the trend with `d` is buried in load noise. These data do not support a quantitative claim about time $\propto$ d.

On a GPU with tensor cores bf16 is usually *faster* than fp32, which is one more reason CPU conclusions must not be transferred to GPUs.

Key points to remember

- Approximate methods are linear in L but win only beyond about 2,048 tokens (local, block sparse); linear_elu wins everywhere here.
- At L = 8,192: local 6.2× and linear 41× faster than the fused exact kernel.
- They use 16–27× **more** memory than the fused exact kernel at 4,096. They save memory only against materializing methods.
- Tile size: U-shaped time (best 128), and autograd's training memory is about 77 MiB regardless; recompute is about 9 MiB.
- Threads: the fused kernel reaches 3.5× at 6 threads; Python tiled becomes 4.8× slower.
- bf16 on this AVX2 CPU: never faster; naive bf16 is 10–30× slower.

Chapter 16 — Results III: what a single attention operation can retrieve (probes)

> **What you will learn:** how to test what an attention method *can* do, without any training, using hand-constructed inputs; the four probe experiments (distance, length, decoys, sharpness); the results for each method; and a fairness check that changed how the linear-attention result is stated.

Run: `20261011-195716_probes` (`python lab.py probes --full`, 128 trials per point). Code: `src/attention_lab/tasks/probes.py`. Report: `experiments/reports/20261011-195716_probes/probes.md`.

16.1 The idea of a probe

Before training any model, we can ask a sharper question: **if the right answer is stored at one specific position, can this attention operation route it to the query?** That tests the operation itself, free of any learning effects.

Construction (one trial = one batch element, one head):

- `L` keys: random directions, scaled so that a query equal to a key direction gets a score `q·k/√d = β` (default **β = 10**), while unrelated keys get scores of typical size `β/√d`.
- The **query** sits at the last position. It is set equal to the key at the **target** position `t = L − 1 − distance` (the "stored association").
- **Values** are random vectors. The method's output at the last position is decoded as "whichever value vector it is most similar to" (cosine similarity). The trial succeeds if that is the target's value.
- **Decoys** (optional): some other keys are made similar to the query (cosine 0.9), which gives them scores near `0.9β`. These are misleading near-matches.

Every method receives *identical* q, k, v per trial, with causal attention.

16.2 Distance sweep (L = 1024)

distance to target	1–63	64	65	128	256	512	1023
exact softmax	1.00	1.00	1.00	1.00	1.00	1.00	1.00
local_window (w = 64)	1.00	1.00	**0.00**	0.00	0.00	0.00	0.00
block_sparse (b = 64, ±1 local, global 0, 1 random)	1.00	1.00	1.00	**0.00**	0.00	0.00	**1.00**
linear_elu	≤ 0.008	0	0	0	0.008	0.008	0.008

- **local_window** drops from 100 % to 0 % between distance 64 and 65: exactly the window edge.
- **block_sparse** succeeds within its own and the neighbouring block, fails at 128–512, and **succeeds again at 1023**, because the target at position 0 is inside global block 0. What a block-sparse method can reach is a property of its layout, not of distance.
- **Exact softmax** succeeds everywhere.
- **linear_elu** is at chance level (≈ 1/L).

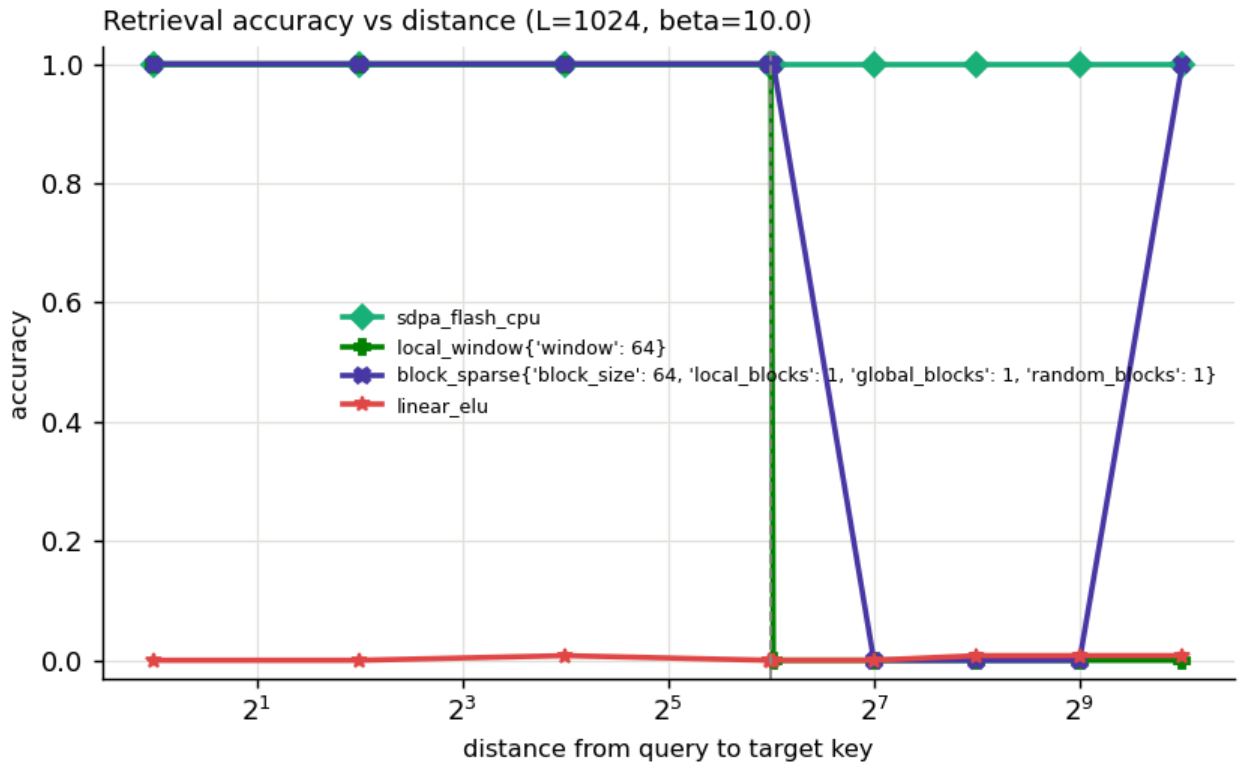


Figure 16.1 — Retrieval accuracy vs distance at L = 1024 (run 20261011-195716_probes). The dashed line marks the local window w = 64.

16.3 Length sweep

With the target 8 tokens back, exact, local and block-sparse attention scored 100 % for L = 128 … 4096. With the target at position 0 (distance L – 1), exact softmax stayed at **100 % up to L = 4096**: the target's weight was never diluted away at β = 10. local_window scored 0 % (out of reach), block_sparse 100 % (global block), and linear_elu was at chance.

16.4 Decoys: interference cuts both ways (L = 512, target 8 back)

decoys (cos 0.9)	0	4	8	16
exact softmax	1.00	1.00	0.977	**0.867**
local_window	1.00	1.00	1.00	1.00
block_sparse	1.00	1.00	1.00	1.00
linear_elu	0	0	0	0

Exact attention sees every decoy, and with 16 near-matches it starts confusing them with the target. The restricted methods are **immune here, because the decoys fall outside their pattern**: they cannot see them. A restriction can protect against distractors it cannot see, and it can equally hide a needed target it cannot see. Both effects follow from the same mechanism.

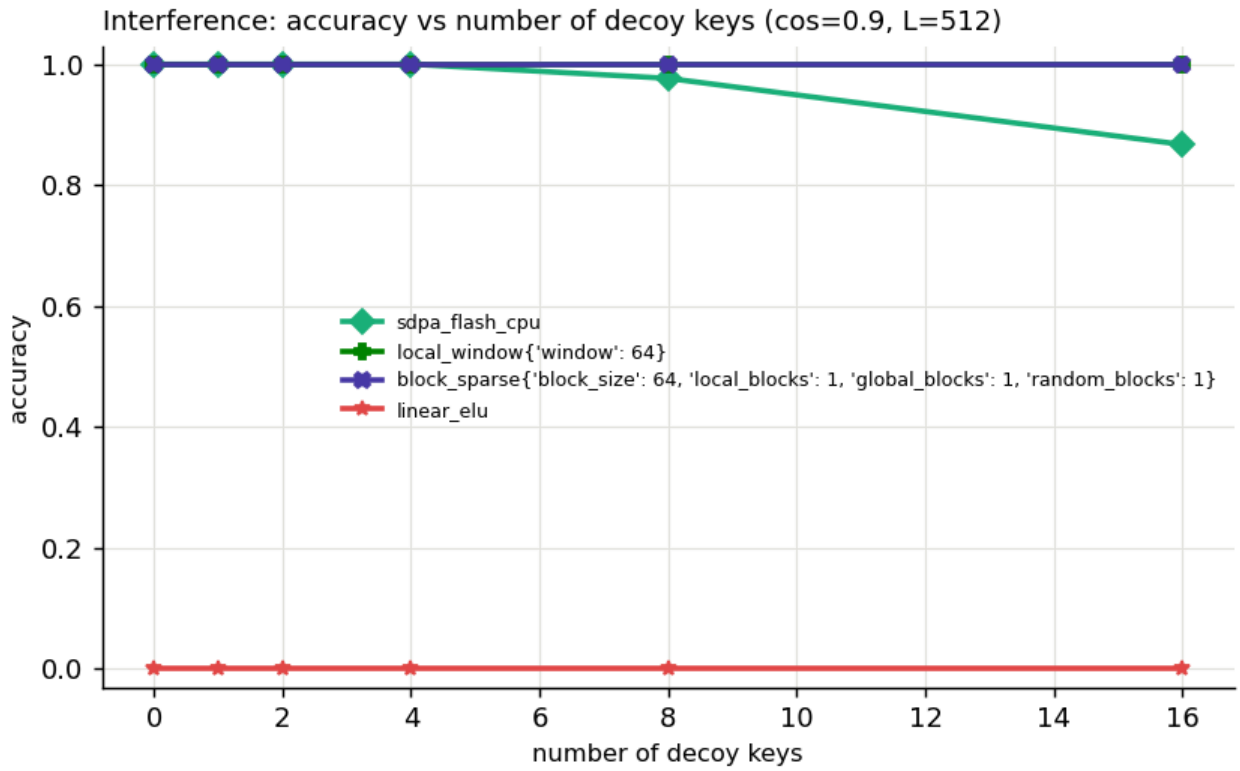


Figure 16.2 — Accuracy vs number of decoy keys (cosine 0.9) at L = 512.

16.5 The fairness check on linear attention

Reporting "linear attention retrieves at chance" would have been misleading if the probe was simply unfair to it, for example if β = 10 suited softmax but not elu + 1. So a **sharpness sweep** was added: distance 4, L = 16 and 64, β = 1 … 100.

β	1	3	10	30	100
exact softmax, L = 16	0.914	1	1	1	1
linear_elu, L = 16	0.062	0.125	0.242	0.469	**0.727**
exact softmax, L = 64	0.305	1	1	1	1
linear_elu, L = 64	0.023	0.031	0.031	0.086	**0.188**

So linear attention is **not zero by construction**. Its accuracy rises with input scale, but slowly, and it falls quickly with L. The reason: `exp(q·k)` sharpens **exponentially** as scores grow, so one key can take nearly all the weight. `ϕ(q)·ϕ(k)` with `ϕ = elu + 1` grows only about **polynomially**, so the target's weight stays comparable to the sum of many others. That is the precise statement the project makes, rather than "linear attention cannot retrieve". Chapter 18 adds that a *trained* linear model can do well when there are few candidates.

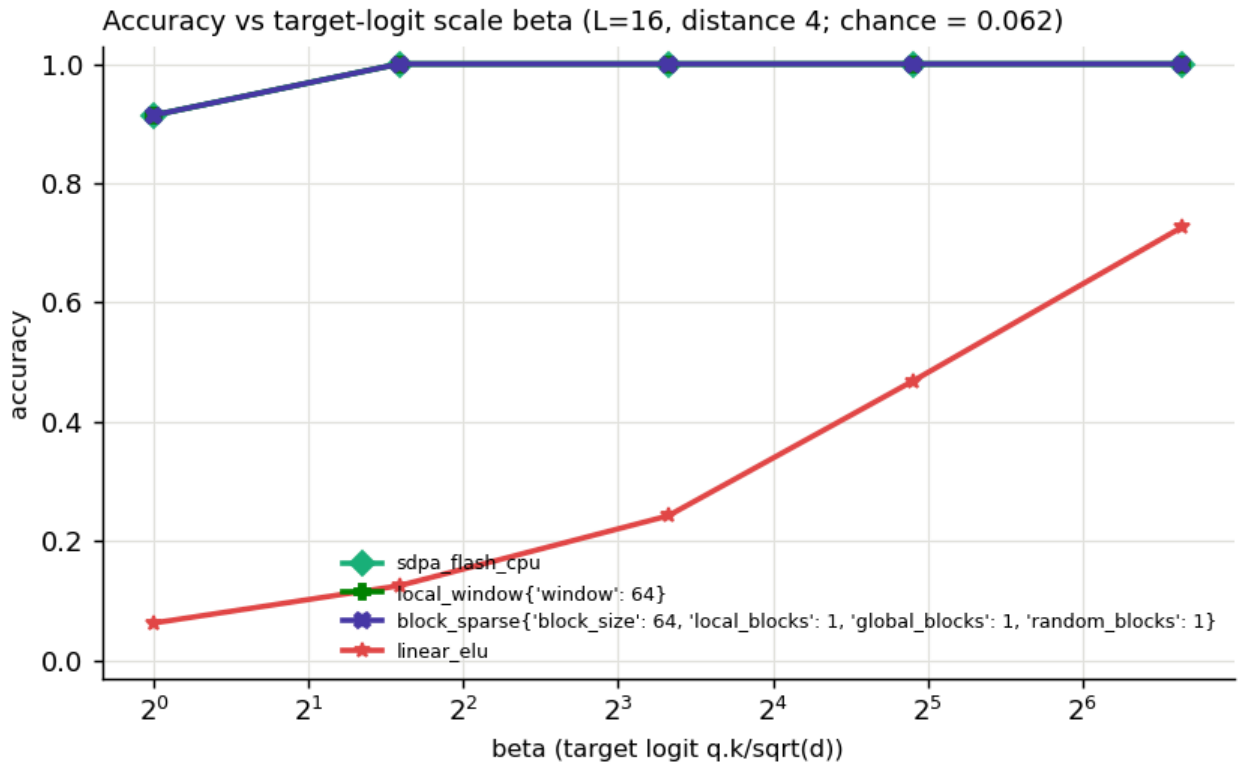


Figure 16.3 — Sharpness check: accuracy vs target-logit scale β at L = 16.

16.6 What probes do and do not show

Probes test **one attention operation with hand-made inputs**. A trained multi-layer model can learn inputs that suit its attention, and can route information around restrictions by relaying it through intermediate tokens. Both effects appear in Chapter 18. Probes show the *mechanism*; trained tasks show the *consequences*.

Key points to remember

- Probe = known target position + identical inputs for every method + nearest-value decoding.
- local_window: 100 % → 0 % exactly at the window edge. block_sparse: reach is set by the layout (global block 0 rescues distance 1023).
- Exact softmax: 100 % to L = 4096; degrades with many decoys (86.7 % at 16).
- Restricted methods were immune to decoys outside their pattern.
- Fairness check: linear_elu improves with β (L = 16: 6 % → 73 %) but stays far below softmax, because elu + 1 is polynomially, not exponentially, sharp.

Chapter 17 — The tiny Transformer and how it was trained

> **What you will learn:** the exact model used for the quality experiments; how its attention is swapped by name; how the training loop works; how fairness between backends is enforced; the synthetic tasks and their generators; and the practical lessons about training on a busy CPU.

17.1 The model

> **In the code:** `src/attention_lab/models/transformer.py`.

`TinyTransformer(ModelConfig(...))` is a decoder-only, pre-LayerNorm Transformer:

```
tokens → Embedding(64 → 64) → [Block] × 2 → LayerNorm → Linear(64 → 64) → logits
Block: x = x + Attention(LayerNorm(x));  x = x + MLP(LayerNorm(x))
MLP:   Linear(64 → 256) → GELU → Linear(256 → 64)
```

Defaults: `vocab_size 64`, `d_model 64`, `n_heads 4` (head dimension 16), `n_layers 2`, `d_mlp 256`, positional encoding **RoPE**, causal attention. About 100,000 parameters. Weights are initialized from a normal distribution with standard deviation 0.02.

Pluggable attention. The `Attention` module computes Q, K and V with one linear layer, reshapes them to `[B, H, L, d]`, applies RoPE, then calls `self.backend(q, k, v, causal=..., **params)`, where `backend` is a registry entry chosen by name. `model.set_attention("local_window", {"window": 8})` swaps the algorithm in every layer **without touching any weights**. That makes "swap" experiments possible: train with exact attention, then evaluate the same weights with another algorithm.

Tested properties (`tests/integration/test_transformer.py`): output shapes for every positional mode; **exact backends give identical logits** (to 1e-10 in fp64) and gradients (relative difference < 1e-4 in fp32); an approximate backend changes the logits by a measurable amount; **no backend lets future tokens affect past logits**; RoPE scores depend only on relative position; a training step reduces the loss.

17.2 The training loop

> **In the code:** `src/attention_lab/training/train.py`, `train(TrainConfig)`.

Each step draws a fresh batch from the task generator, computes the cross-entropy loss **only at positions with a target** (others are labeled −100 and ignored), backpropagates, clips the gradient norm to 1.0, and takes an AdamW step (lr 3e-3, weight decay 0.01, betas 0.9/0.98) with 100 warm-up steps and cosine decay. Every `eval_every` steps it evaluates on a fixed held-out set.

Fairness controls in `run_task_experiment`: every backend gets the same initialization seed, the same stream of training batches (same generator seed), the same step budget and optimizer, and the **same fixed**

evaluation sets (`fixed_eval_set`, seeded separately from training). Results go to JSONL with the full history. Checkpoints are saved locally to `experiments/checkpoints/` (excluded from version control).

17.3 The synthetic tasks

> **In the code:** `src/attention_lab/tasks/synthetic.py`.

Vocabulary layout: 0 = padding; 1–8 = NOISE tokens; 9–24 = KEY tokens; 25–40 = VALUE tokens.

- **Task A, local copy (`local_copy`)**: random tokens; the target at position `i` is the token at `i - lag` (lag 3). It only needs information 3 positions back.
- **Task B, variable-offset induction (`induction`)**: a random segment of 12 tokens, a gap of 0–8 noise tokens, then the same segment again. Inside the repeat, each target is the **next** segment token. To produce it, the model must find where the current token occurred before and copy its successor. Because the gap varies, the offset (12–20) varies, so **the source must be found by content, not by position**.
    - **Task C (interference):** the same, with the gap filled by segment-vocabulary tokens (distractors).
    - **Task D (offset generalization):** evaluated at L = 64 with offsets 28–36 or 36–52, never seen in training.
- **MQAR (`mqar`)**: multi-query associative recall in the style of Zoology (Arora et al., 2023). Key–value pairs $k_1\ v_1\ k_2\ v_2\ \ldots$, then noise in which some keys reappear; the target at a reappeared key is its bound value. Its story is Chapter 19.

Each generator also returns `distance` (how far back the answer's source is) and `source` (its position), so accuracy can be broken down by distance. A shortcut-baseline function (`shortcut_baseline`) computes the best possible accuracy *without* binding keys to values (Chapter 19).

17.4 Lessons from training on a busy CPU

> **What actually happened (1): starved threads.** The first quick training run used 4 threads. After 7 minutes it had not finished its first 200-step model. The process had used only 102 CPU-seconds in about 420 seconds of wall time, a quarter of one core. With all 12 logical CPUs busy, a 4-thread OpenMP job running tiny matrices stalls at every synchronization point. Fix: **train with 1 thread** (now the CLI default for `tasks`), which is faster *and* more robust for models this small.
>
> **What actually happened (2): "why is one step 500 ms?"** Even single-threaded, one step took about 500 ms, which looked impossibly slow for a 100k-parameter model. Before changing anything, the step was profiled. Matrix multiplications dominated, at about 4 ms each. The arithmetic explains it: a batch of 64 sequences × 64 tokens is 4,096 tokens, and training costs about 6 × parameters × tokens ≈ 2.5 GFLOP per step, on one contended core. The expectation of 10–20 ms had simply been wrong. Fix: batch size 32 (a `--batch-size` option was added).
>
> **What actually happened (3): invisible progress.** Output redirected to a file showed nothing for minutes, because Python buffers stdout when it is not a terminal. Fix: run with `python -u` (unbuffered), and rely on the JSONL results, which are flushed after every record.
>
> **What actually happened (4): a misleading warning.** `float(loss)` on a tensor that requires gradients triggered a PyTorch warning. Fix: `float(loss.detach())`.

Key points to remember

- 2-layer, d_model 64, 4 heads, RoPE, about 100k parameters; attention is swapped by name with weights untouched.
- Exact backends give identical model outputs; no backend leaks future tokens.
- Fairness: same seed, data stream, budget, optimizer and fixed evaluation sets for every backend.
- Tasks: A local copy; B/C/D variable-offset induction (content-based); MQAR.
- On a busy CPU, tiny models train faster on 1 thread; profile before assuming code is slow.

Chapter 18 — Results IV: what models trained with each attention method can do

> **What you will learn:** the results of training the same small Transformer with exact, local-window, block-sparse and linear attention on the local-copy and variable-offset induction tasks; the "swap" experiments; and the analyses that turned surprising numbers into understood ones (relay through layers, a layout that was secretly dense, and distractors that are not explained by ambiguity).

Run: `20261011-205830_tasks` (`python lab.py tasks --which AB --steps 1500 --threads 1 --batch-size 32`). Single seed. Backends: exact (`sdpa_flash_cpu`), `local_window` (w = 8), `block_sparse` (block 8, ±1 local, global block 0, 1 random), `linear_elu` (chunk 16). **Swap** = the model trained with exact attention, evaluated with another backend substituted, no retraining.

18.1 Task A: local copy (lag 3), 500 steps, trained at L = 64

backend	trained: L = 64	trained: L = 256	swapped into exact model: L = 64	swapped: L = 256
exact	1.000	0.869	1.000	0.869
local_window	1.000	**1.000**	1.000	**1.000**
block_sparse	1.000	0.978	1.000	0.989
linear_elu	0.560	0.293	0.240	0.151

An earlier 200-step run (`20261011-200406_tasks_quick`) gave the same pattern.

1. **A task needing only local information is solved perfectly by restricted attention.**
2. **Restriction improved length generalization.** The exact model fell to 86.9 % at 4× its training length. *The same weights* with a window of 8 swapped in scored 100 %. So the exact model's errors at long lengths come from attending to far positions it never saw in training, not from any missing information. Removing those pairs fixes it.
3. **linear_elu learned the 3-token copy only partly** (56 %), consistent with the probes: its kernel cannot easily select one position sharply.

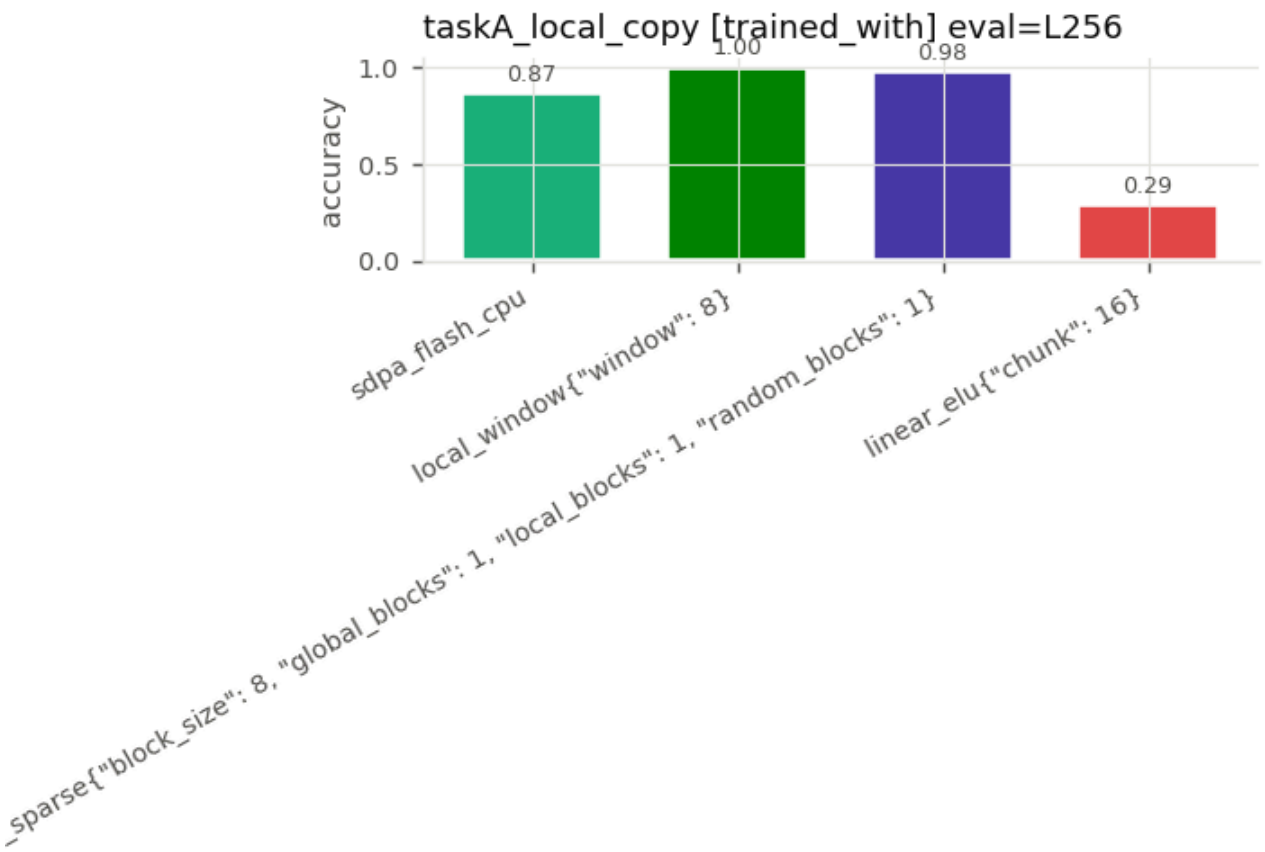


Figure 18.1 — Task A at L = 256 (4× the training length): models trained with each backend.

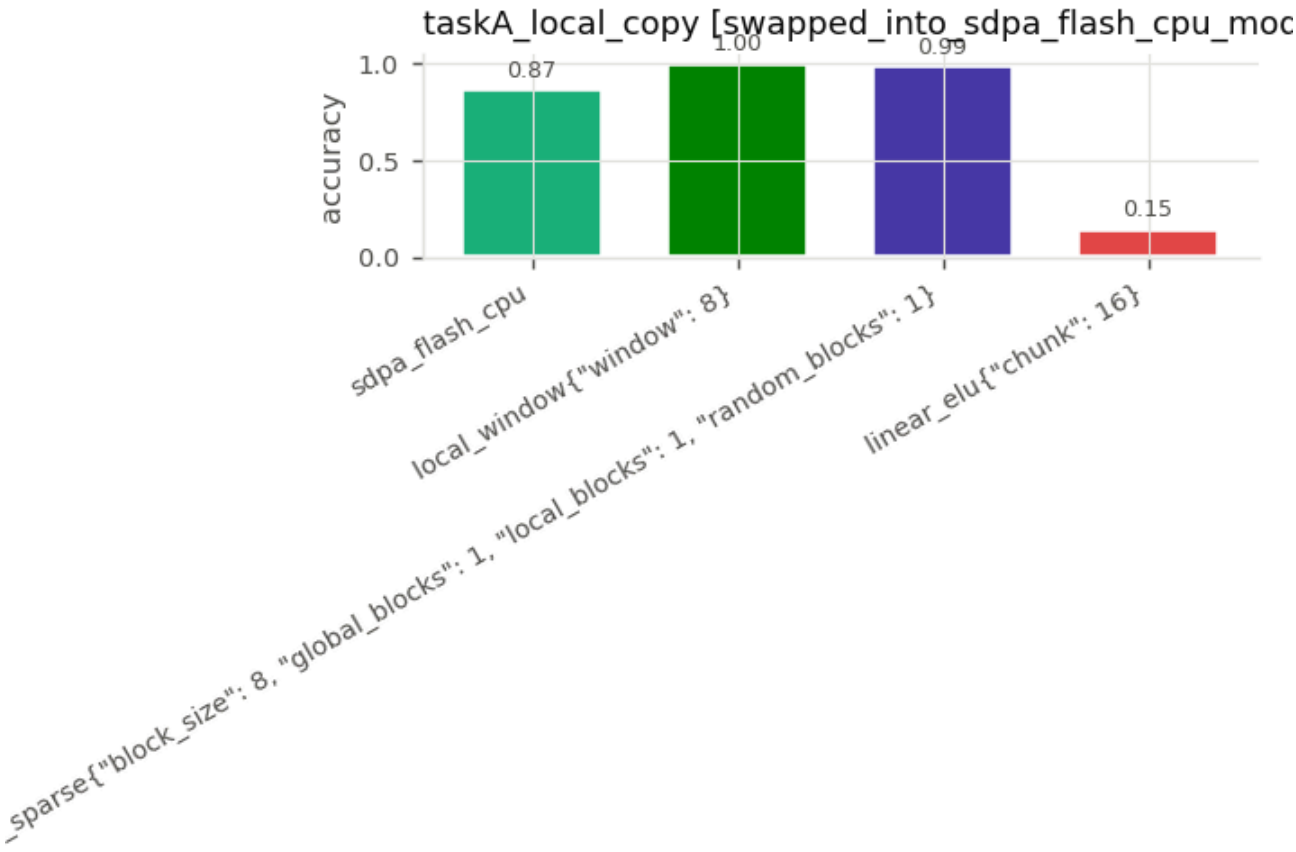


Figure 18.2 — Task A at L = 256: the exact-trained model with each backend swapped in at inference.

18.2 Task B/C/D: variable-offset induction, 1500 steps, trained at L = 32

backend	B: in distribution (offset 12–20)	C: distractor gap	D: offset 28–36	D: offset 36–52
exact	**1.000**	0.183	0.031	0.125
local_window (w = 8)	0.491	0.155	0.029	0.029
block_sparse (b = 8)	1.000 †	0.167 †	0.029	0.084
linear_elu	0.966	**0.421**	**0.174**	**0.245**
swap → local_window	0.030	0.030	0.029	0.031
swap → block_sparse	1.000 †	0.183 †	0.028	0.082
swap → linear_elu	0.149	0.147	0.089	0.073

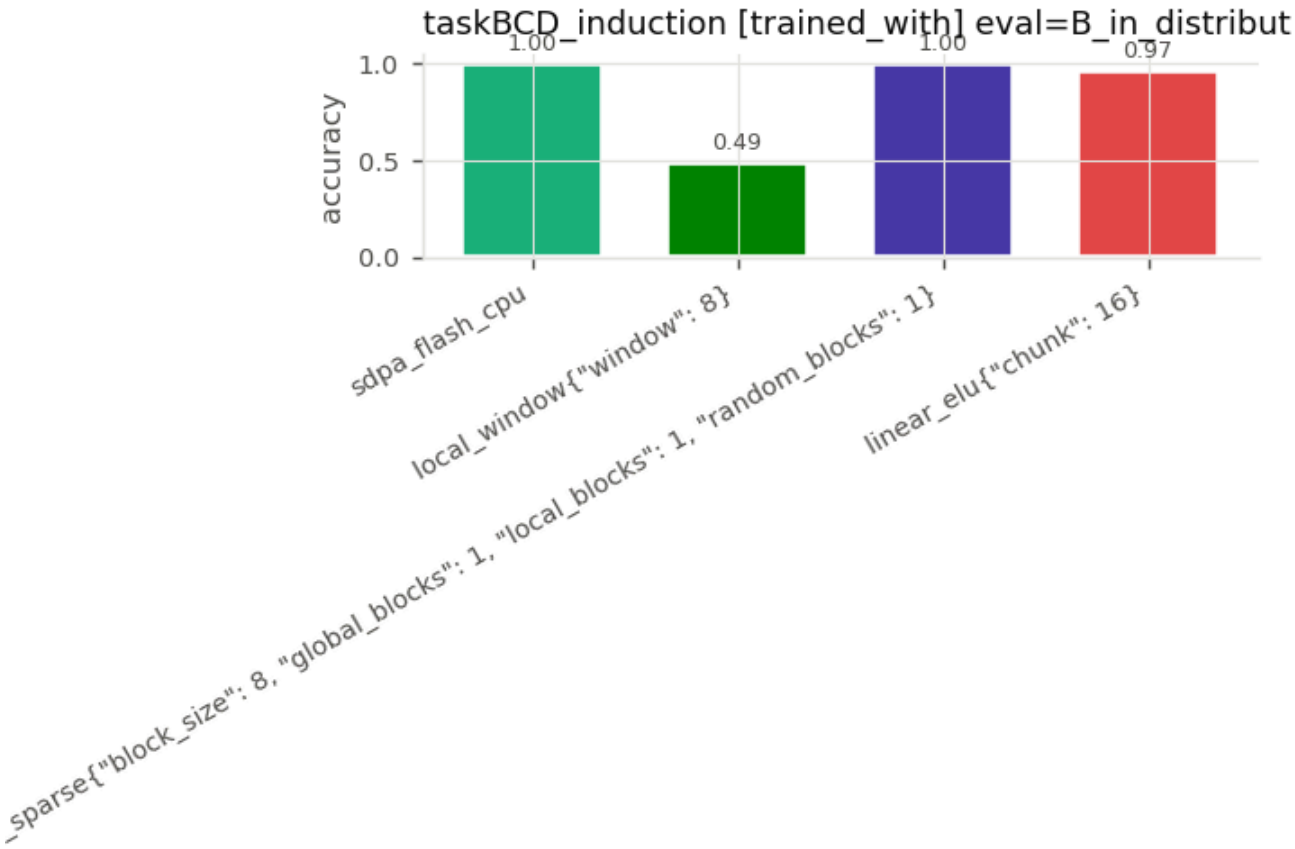


Figure 18.3 — Task B (in distribution): accuracy of models trained with each backend.

Finding 1: exact attention learns content-based retrieval

100 % in distribution, and in the diagnostic run it got there by step 500 (Chapter 19, D3).

Finding 2: stacking layers lets a restricted model reach beyond its window

One layer with window 8 cannot see a source 12 or more tokens back. Yet the local-window model scored 49 %. The accuracy-by-distance breakdown explains it:

local_window, task B	sources 8–16 back	sources 16–32 back
accuracy	**0.99**	0.06

Two layers of window 8 can **relay** information 16 tokens: layer 1 copies it 8 positions forward, layer 2 another 8. Offsets 12–16 are reachable, offsets 17–20 are not, and the measured split matches. The swap confirms it: the exact-trained model with a window swapped in scores 3 %, because it never learned to relay (it had no need to).

Finding 3: a "sparse" layout that was secretly dense

block_sparse scored exactly like exact attention at L = 32. Checking the layout (`make_layout(4, local=1, global=1, random=1, causal=True)`) showed it keeps **528 of 528 causal pairs**: at L = 32 with block 8 there are only 4 blocks, and local ±1 + global 0 + 1 random covers all of them. So those cells (marked †) are **full attention under another name**, not evidence about sparsity. At L = 64 the layout keeps 69.2 % of pairs, so the D columns do test a real restriction.

Finding 4: trained linear attention can retrieve when there are few candidates

linear_elu reached 96.6 %, far better than the untrained probes suggested. Training can learn query and key projections that make the elu + 1 kernel sharp enough to pick one of about 30 positions. The swap result (15 %) shows it learned a different solution from the exact model. In this single-seed run linear_elu also degraded least on C and D. That is a hint, not an established result.

Finding 5: nobody generalizes to unseen offsets

At offsets 28–52 every model collapses to 3–25 %. RoPE gives the model *relative* positions, but a circuit trained only on offsets 12–20 does not transfer to 28+.

Finding 6: distractor gaps break every model, and ambiguity does not explain it

Filling the gap with segment-vocabulary tokens dropped exact attention from 100 % to 18 %. One possible explanation is that the task becomes ambiguous (the current token might also occur in the gap, so "its earlier occurrence" is not unique). That was **measured**, not assumed:

eval set	targets whose current token occurs more than once earlier
B (normal gap)	30.1 %
C (distractor gap)	36.7 %

The exact model handles B's 30 % of ambiguous targets *perfectly*, so a rise to 36.7 % cannot explain a drop to 18 %. The failure is **out-of-distribution brittleness**. An untested hypothesis: the models locate the segment by its boundary with noise tokens, and that cue disappears when the gap uses segment tokens.

18.3 Summary of the quality picture

question	evidence
Does restriction lose long-range information?	Yes, deterministically at the pattern's edge (probes; local_window 0.06 beyond its 2-layer reach)
Can depth compensate?	Partly: two layers relay 2w tokens (0.99 within 16)

question	evidence
Can restriction help?	Yes: length generalization on local tasks (100 % vs 87 %); immunity to out-of-pattern decoys
Is linear attention useless for retrieval?	No: trained, it reached 96.6 % with about 30 candidates; untrained, it is weak at L ≥ 64
Are these general truths about language models?	**No.** They are single-seed, tiny models on synthetic tasks: evidence about mechanisms, not about LLM quality

Key points to remember

- Task A: restricted attention solves local tasks and length-generalizes better (window swap: 86.9 % → 100 %).
- Task B: exact 100 %; local_window 49 % via 2-layer relay (99 % within 16 back, 6 % beyond).
- block_sparse at L = 32 was dense (528/528 pairs): those cells are not sparsity evidence.
- Trained linear attention: 96.6 % in distribution, a different solution from exact.
- All models fail beyond their training offsets and on distractor gaps. The distractor drop is brittleness, not ambiguity (30.1 % vs 36.7 %).

Chapter 19 — The MQAR investigation: a detective story about a model that would not learn

> **What you will learn:** how an innocent-looking training experiment stalled at a suspicious accuracy; how a flaw in our own data generator was found by computing a "shortcut baseline"; how, after fixing it, the models stalled again; how a systematic diagnosis ruled out bugs one by one; how an earlier "success" turned out to be a positional shortcut; and why the investigation was deliberately stopped where it was. This chapter is also the best illustration in the book of the scientific method applied to machine learning.

19.1 The task

MQAR (multi-query associative recall) tests whether a model can bind keys to values and retrieve them, a known weakness of many efficient attention alternatives (Arora et al., 2023). A sequence starts with pairs $k_1\ v_1\ k_2\ v_2\ \ldots\ k_8\ v_8$, followed by noise tokens among which some keys reappear. At each reappeared key the model must output that key's value. With 16 possible values, random guessing scores 6.25 %.

19.2 Act 1: the suspicious plateau (generator v1)

The first quick training run (600 steps) gave, for every backend that could see the pairs:

backend	accuracy (8 pairs)	accuracy (12 pairs)
exact	0.258	0.222
block_sparse	0.253	0.197
linear_elu	0.256	0.214
local_window (cannot see the pairs)	0.068	0.064

Everyone except local_window landed at almost the *same* value, about 25–26 %. A separate learning-curve run of the exact model was stuck at 24–26 % from step 250 to step 1,500. Identical plateaus across very different mechanisms suggest that the plateau is a property of the **task**, not of the models.

Hypothesis: the models had found a way to score 25 % *without* binding keys to values. In generator v1, values were drawn **with replacement**, so a sequence often contained the same value twice. A model that ignores keys and **predicts the most frequent value in the context** would then be right fairly often.

Test: compute that strategy's accuracy on the exact evaluation set: **25.7 % for 8 pairs, 21.4 % for 12 pairs.** That matches the observed plateaus (25–26 % and 20–22 %) almost exactly.

Fix (generator v2): draw values **without replacement**, so each value appears once and the best binding-free strategy is a uniform guess among the `n` values present: exactly `1/n_pairs`. A `shortcut_baseline` metric was added to every MQAR evaluation, a `TASK_VERSION` constant was introduced so old and new results are never mixed, a note was placed in the v1 results folder, and a regression test pins the fix.

19.3 Act 2: stuck again, at a different number

With generator v2, the models stalled at exactly the new baseline:

- 4 pairs: loss **1.386 = log 4**, accuracy 25 %, unchanged after 3,000 steps;
- 8 pairs: loss 2.08, accuracy 12.5 %, unchanged at 2,500 steps.

A loss of `log(n)` means the model spreads its probability evenly over the `n` values present: it knows *which values are in the context* but not *which one belongs to the key*. Changing the learning rate (3e-3 → 1e-2) and the training loop (constant learning rate, no warm-up, no clipping) changed nothing.

An apparently strong clue existed: a simple **induction** task (the second half of the sequence repeats the first, predict the next token) had reached 100 % with the same model. That seemed to show the model *could* learn retrieval, which would point at something MQAR-specific.

19.4 Act 3: a systematic diagnosis

At this point the instruction was: stop launching runs, preserve everything, diagnose rigorously, one experiment at a time, and distinguish bugs from optimization problems, insufficient training and genuine difficulty.

Lesson recorded on the way: three exploration runs had been writing to the same log file at once, which garbled the lines. From then on, each diagnostic ran alone with its own log.

Step 1: targeted tests for every link of the pipeline

`tests/unit/test_mqar_pipeline.py`, 10 tests, **all passing**:

suspected failure	result
wrong token types in regions	correct
duplicate keys or values; unbound query keys	none
the answer is leaked or derivable without binding	each answer occurs exactly once, right after its key
biased query/target marginals	uniform
loss at wrong positions or shifted targets	loss only at the 4 query positions, aligned correctly
plateau value	"uniform over the 4 context values" gives *exactly* log 4
query cannot see its key or the pairs (mask bug); future leakage	visible; no leakage
dead parameters / broken gradient flow	every parameter gets a gradient
metric bug	oracle 100 %, shortcut 25 %

Since the exact attention backends compute the same function to about 1e-6 (Chapter 11), **the attention implementation cannot be the cause**.

Step 2: build trustworthy instruments

`src/attention_lab/diagnostics/recall.py` records, during training: per-module gradient norms, per-module relative update sizes, prediction categories (correct / another context value / absent value / key / other), and **circuit probes**:

- `prevtok_l0`: how much layer-0 attention at each value position goes to the position just before it (its key). An induction circuit needs this "previous-token head";
- `target_l1`: how much layer-1 attention from the query goes to the *correct* value position, compared with the other values.

Before use, the instruments were themselves tested: the attention maps they replay reproduce each layer's real output to 1e-5, and the probes read exactly the intended positions on hand-made maps (`tests/unit/test_diagnostics.py`).

Step 3 (D1): can the model fit MQAR data at all?

64 fixed sequences, trained repeatedly for 2,000 steps:

step	train acc	held-out acc	held-out loss	prev-token attn (L0)	attn on correct value (L1)	on each other value
1	0.14	0.08	4.03	0.27	0.06	~0.06
100	**1.00**	0.20	5.37	0.37	0.14	~0.14
2000	**1.00**	**0.20**	7.84	0.36	0.13	~0.13

The model **memorized** the 64 sequences perfectly (so forward, backward, loss and optimizer all work), but held-out accuracy stayed below the 25 % shortcut and held-out loss rose: classic memorization. The binding circuit never formed. Layer-1 attention weighted the correct value no more than any other.

Step 4 (D2): was the induction "success" real?

The induction task that reached 100 % had a hidden property: the second half always started exactly 12 positions after the first, so **the answer was always at the same relative offset**. With RoPE (Chapter 4), a single head can learn "copy from 12 positions back" without matching any content. To test this, the same task was trained with a fixed offset and evaluated with a **random gap** (offset 12–20):

step	accuracy, fixed offset	accuracy, random offset
100	1.00	0.18
2000	1.00	**0.18**

It was a positional shortcut. The earlier conclusion "the model can learn content-based retrieval" was unsupported, and was withdrawn.

Step 5 (D3): can the model learn *content-based* induction?

Trained directly on random offsets, where content matching is required:

step	held-out acc	layer-0 attention gradient norm
100	0.09	0.30
200	0.24	**0.75**
500	**0.999**	0.52
1000	**1.00**	0.003

Yes, and quickly. The spike in layer-0 gradient just before accuracy jumps is consistent with a previous-token circuit forming. So the architecture and the training loop **can** learn content-based retrieval.

19.5 The verdict

hypothesis	verdict
bug in the exact attention implementations	**rejected** (numerical equivalence; identical model outputs)
bug in generator, targets, loss, masks, metric, gradients	**rejected** for v2 (10 targeted tests)
v1 generator shortcut	**confirmed and fixed**
the model cannot fit MQAR data	**rejected** (D1: 100 % train)
the architecture cannot learn content-based retrieval	**rejected** (D3: 100 %)
"plain induction works" proves content-based retrieval	**rejected** (D2: positional shortcut)
learning rate / loop settings are the cause	**not supported** (same plateau with several settings; not exhaustively swept)
insufficient training budget	**open**
MQAR-specific optimization difficulty (sparser supervision; the "uniform over context values" solution as an attractor)	**plausible, untested**

19.6 Why the investigation stopped here

The project's question is about the **efficiency and correctness of attention algorithms**. The diagnosis had established that the MQAR plateau says nothing about the attention implementations. Resolving MQAR's learning dynamics would be a separate research project. So the decision was:

- **preserve** everything (logs in `experiments/diagnostics/mqar_v2/`, the full write-up in `docs/mqar_diagnosis.md`);
- **use the task the exact model demonstrably learns** (variable-offset induction, D3) for the trained quality comparison (Chapter 18);
- keep MQAR available (`python lab.py tasks --which M`) and list concrete next experiments: more queries per sequence (denser supervision), a curriculum 1 → 2 → 4 pairs, and a longer budget with the circuit probes, run one at a time.

19.7 Lessons

1. **Identical plateaus across different models point at the task.** Compute the best "cheating" baseline.
2. **A loss equal to `log(n)` is a fingerprint**: uniform over `n` options.
3. **Test each link of a pipeline separately** before blaming optimization.
4. **Validate the instruments** before trusting what they show.
5. **"It learned X" requires ruling out shortcuts.** Position, frequency and other cues can mimic the intended skill. Test out of distribution.
6. **Know when to stop.** Answer the question your project needs answered, document the rest, and move on.

Key points to remember

- v1 plateau 25–26 % = "predict the most frequent context value" (25.7 %), a generator flaw, fixed in v2.
- v2 plateau at loss log(n) = uniform over context values; no binding learned.

- 10 pipeline tests pass; the model memorizes (D1) but the circuit never forms.
- The induction "success" was positional (fixed 100 %, random-gap 18 %); content-based induction is learnable (D3: 100 % by step 1000).
- The cause is MQAR-specific learning dynamics or budget, not the attention code. Stopped and documented deliberately.

Chapter 20 — Operating the lab: install, run, explore, reproduce, extend

> **What you will learn:** every command, what it produces and where; how to use the web interface; how to read a report; how to reproduce all results; and how to add a new attention algorithm or a new experiment.

20.1 Requirements and setup

- Python ≥ 3.10 (developed on 3.13.3).
- Packages: `torch` (the CPU build is fine), `numpy`, `pandas`, `matplotlib`, `psutil`, `pyyaml`, `pytest`, and `streamlit` for the UI. On this machine all were already installed.
- **No installation step is required.** Open a terminal in the project folder and run `python lab.py ...`. The launcher adds `src/` to Python's search path. Optionally, `pip install -e .` installs the package in development mode and adds an `attention-lab` command.

20.2 The commands

All commands are `python lab.py <command> [options]`:

Command	What it does	Where output goes
`env`	prints the environment audit (CPU, cores, RAM, versions, thread settings)	terminal
`test [--all]`	runs the test suite	terminal
`correctness [--full] [--methods ...]`	the numerical-correctness laboratory	`experiments/results/<time>_correctness/` (results.jsonl, summary.md, meta.json)
`bench --smoke`	every method on tiny shapes, about 20 s	`experiments/results/<time>_smoke/` + report
`bench --spec experiments/configs/<name>.yaml [--isolate yes/no] [--threads ...]`	a benchmark sweep	results + `experiments/reports/<run>/`
`report experiments/results/<run>`	rebuilds the report of a run	`experiments/reports/<run>/`
`complexity --L 1024 4096 [--causal] [--d 64]`	prints the theoretical cost table, labeled THEORETICAL	terminal
`probes [--full]`	attention-level retrieval probes	results + report (`probes.md`, figures)

Command	What it does	Where output goes
`tasks [--which AB(M)] [--steps N] [--threads 1] [--batch-size 32] [--quick]`	trains the tiny Transformer with each backend	results + report (`tasks.md`, figures) + checkpoints
`reproduce [--dry-run] [--only E2 Q1] [--keep-going]`	runs every experiment behind the results, one after another	all of the above
`ui [--port 8501] [--headless]`	starts the local web interface	browser at `http://localhost:8501`

Diagnostics for training (Chapter 19): `python scripts/diagnose_recall.py overfit|pairs|induction ...` and `python scripts/summarize_diag.py <log>`.

Sweep specs (`experiments/configs/`): `smoke`, `scaling_exact` (memory, isolated), `scaling_exact_timing` (1 thread), `scaling_approx`, `tile_sizes`, `threads`, `shapes_dtypes`. Each contains a `hypothesis:` line, the method list, the grid and the budgets. To change an experiment, copy a spec and edit it. No code changes are needed.

20.3 The web interface

`python lab.py ui` opens a Streamlit app bound to **localhost only**, with usage statistics disabled. Tabs:

1. **Environment**: the live audit.
2. **Algorithms**: the registry: family, what each computes, how it executes.
3. **Correctness**: compare any exact method with the fp64 reference for chosen shapes, dtype, magnitude and tiles.
4. **Benchmark**: run one guarded configuration and see its timing, memory and theory numbers.
5. **Results & plots**: pick a stored sweep and see its hypothesis, status counts, the full table, interactive runtime and memory charts, fitted exponents and the report figures.
6. **Complexity**: the theoretical table and chart for chosen lengths.
7. **Quality probes**: run a probe interactively, or browse stored probe runs.
8. **Task training**: browse trained-task results.
9. **History & export**: every run, plus a button that builds and downloads a report.

Everything displayed is read from `experiments/results/` or computed live. Nothing is hard-coded.

> **What actually happened:** when the UI was first tested in a browser, the Results tab crashed with `FileNotFoundError: ...probes/meta.json`. The function listing "benchmark runs" included every folder containing a `results.jsonl`, including probe and correctness runs, which had no `meta.json` and are not sweeps. Fixes: list only folders whose `meta.json` contains a sweep spec, and make the probe and correctness commands write a `meta.json` too. A second lesson: Streamlit reloads `app.py` on change but keeps imported modules cached, so the server had to be restarted to pick up the fix. A third: on first launch Streamlit can ask for an email on the console and open a browser, so a `--headless` flag was added for automated launches.

20.4 Reading a report

`experiments/reports/<run>/summary.md` contains: the hypothesis from the spec; the environment; status counts; a **measurement-scopes** section (what each column means); the background-load summary; fitted exponents; the full results table (`summary.csv` has the same data); figures; and a list of every skipped, partial or failed configuration with its reason.

Columns: `*_ms_median` / `*_ms_iqr` (wall time), `*_cpu_ms_per_call` (process CPU time), `*_tensor_peak_MiB` (PyTorch allocator peak), `proc_peak_delta_*_MiB` (OS working set, isolated runs), `theory_*` (estimates), `err_vs_ref` (exact methods) or `deviation_from_softmax` (approximate methods), `other_load_cores` and `contended`.

20.5 Reproducing everything

```
python lab.py reproduce --dry-run     # list the steps
python lab.py reproduce               # run them all (hours on a busy laptop)
python lab.py reproduce --only E2     # one step
```

Steps, in order: tests; correctness (C1); exact scaling memory (E1); exact timing (E1b); approximate scaling (E2); tile sizes (E3); threads (E4); shapes and dtypes (E5); probes (Q1); trained tasks (Q2). They run sequentially, because running benchmarks in parallel would contaminate each other's timings. A reproduction gives new timestamped folders; compare them with the runs named in `docs/results.md`. Expect identical correctness, memory and accuracy results (they are deterministic given the seeds), and different absolute timings (they depend on machine load).

20.6 Extending the lab

A new attention algorithm: write a function `fn(q, k, v, *, causal=False, key_padding_mask=None, mask=None, **params)` in `src/attention_lab/algorithms/`, use `validate_qkv` and `MaskSpec` from `base.py`, and add one `AttentionMethod(...)` entry to `registry.py` with its family and a precise `computes` string. Add its cost formulas to `complexity/theory.py`. Then add tests: exact methods join `EXACT_METHODS` automatically and are picked up by the numerical tests; restricted methods need an equality test against naive with the corresponding mask. Benchmarks, probes, the model and the UI will offer it by name with no further changes.

A new experiment: copy a YAML spec, edit the grid and hypothesis, and run `python lab.py bench --spec ...`.

GPU support: see Chapter 13 §13.8. Only the benchmarking instruments need device-aware additions.

Key points to remember

- `python lab.py env | test | correctness | bench | report | complexity | probes | tasks | reproduce | ui`.
- Experiments are YAML specs with a stated hypothesis; outputs are timestamped result and report folders.
- The UI reads only stored or live-computed results, runs on localhost, and has no telemetry.
- `reproduce` runs everything sequentially; deterministic results reproduce exactly, timings do not.
- Adding an algorithm = one function + one registry entry + theory formulas + tests.

Chapter 21 — The test suite: what is checked, and why it is trustworthy

> **What you will learn:** how the tests are organized; what each test file proves; the rules the tests follow; and the small configuration problems that had to be solved to make the suite run cleanly.

21.1 Organization

Folder	Purpose	Files
`tests/unit/`	single components in isolation	`test_base_and_naive.py`, `test_online_softmax.py`, `test_tasks.py`, `test_mqar_pipeline.py`, `test_diagnostics.py`
`tests/numerical/`	agreement with the reference (forward and gradients)	`test_exact_vs_reference.py`, `test_approx_methods.py`
`tests/integration/`	the Transformer with pluggable attention	`test_transformer.py`
`tests/scientific/`	theory vs instrumented counts; validity of measurement tools	`test_complexity_and_measurement.py`
`tests/perf_smoke/`	every method benchmarks end to end; guards and scheduler work (no timing thresholds)	`test_benchmark_smoke.py`

`tests/conftest.py` limits each test to at most 4 threads and provides `rand_qkv`, which builds seeded random q/k/v.

21.2 What each file proves

- **`test_base_and_naive.py`**: the reference matches an independent formula and PyTorch SDPA; scale, masks (boolean, additive, causal, padding, combinations), fully-masked rows, LSE, gradients (`gradcheck`), and every input-validation rejection (Chapter 7).
- **`test_online_softmax.py`**: the update rule equals softmax for any split; it survives huge later maxima and all-masked first tiles; ragged tiles and tiles larger than the sequence work; causal tile skipping counts correctly; invalid tile sizes are rejected (Chapter 8).
- **`test_exact_vs_reference.py`**: every optimized exact method × fp64/fp32 × causal × 4 shapes, plus batch/head shapes, input magnitudes 0.01–30, bf16 policy, fully-masked rows, dense masks for the tiled methods, `gradcheck` of the custom backward, LSE agreement, flat error vs L, numerical-vs-bitwise equality, and the two "finding" tests that pin the investigations of Chapter 11.
- **`test_approx_methods.py`**: local window, block sparse and linear each equal their own definitions (outputs and gradients) over grids of lengths and parameters; limit cases equal full attention; layouts are causal and reproducible; linear attention is *not* softmax; unsupported dense masks raise errors (Chapter 10).
- **`test_transformer.py`**: shapes; exact backends give identical outputs and gradients; approximate backends change outputs; no future leakage for any backend; RoPE relativity; a training step reduces the loss (Chapter 17).

- `test_tasks.py`: generator invariants for local copy, MQAR (v2 distinctness, decoys, source positions) and induction (successor of the earlier occurrence, offsets 12–20, distractor vocabulary); fixed eval sets are deterministic; probe construction gives target logit β; probes behave as expected (Chapters 16, 17, 19).
- `test_mqar_pipeline.py`: the 10 failure-mode tests of the MQAR diagnosis (Chapter 19).
- `test_diagnostics.py`: the diagnostic instruments reproduce the model's real attention and read the intended positions (Chapter 19).
- `test_complexity_and_measurement.py`: theory equals instrumented counts exactly; the tensor-peak instrument is byte-exact on a known pattern and sees the score matrix; statistics and exponent fitting are correct (Chapters 12–13).
- `test_benchmark_smoke.py`: every method runs and produces a complete record; the memory guard skips without allocating, and can skip backward only; the scheduler orders small-first and writes JSONL + meta (Chapter 13).

21.3 Rules the tests follow

1. **No timing thresholds.** Correctness is asserted; speed is measured and reported elsewhere.
2. **Independent references.** The naive reference is checked against `torch.softmax` and SDPA; approximate methods against masked naive or the direct formula; gradients by finite differences.
3. **Findings are tests too.** When an investigation established a fact (bf16 naive degrades; the fp32 floor scales with logits; v2 removes the MQAR shortcut; the induction generator's offset varies, so it cannot be solved positionally), a test pins it, so a future change that breaks the fact is noticed. (The positional-shortcut finding itself, D2 in Chapter 19, is evidence in a diagnostic log, not a unit test; it needs training.)
4. **Tolerances come from the numerical policy**, never adjusted to make a test pass.

21.4 Problems solved to make the suite run

> **What actually happened:**
>
> - Tests import a helper as `from tests.conftest import rand_qkv`. That failed until the project root was added to pytest's path (`pythonpath = ["src", "."]` in `pyproject.toml`) and `tests/__init__.py` was created.
> - A globally installed `pytest-asyncio` plugin printed a deprecation warning on every run. It is not used by this project, so it is disabled locally (`-p no:asyncio` in `addopts`).
> - Passing `-q` on the command line *and* in `addopts` made pytest so quiet that it hid the pass/fail summary line.
> - The profiler printed "Profiler clears events at the end of each cycle" warnings; they are filtered in the config.

Key points to remember

- Five test categories: unit, numerical, integration, scientific, perf_smoke.
- Every claim in the results has a test behind it, including the investigations' findings.
- No timing assertions; tolerances come from the documented policy.
- Run with `python lab.py test` (fast set) or `python -m pytest tests` (all).

Chapter 22 — The logbook: every bug, mistake, surprise and fix, in order

> **What you will learn:** the true history of the project, as a numbered log. Each entry gives the **symptom** (what was seen), the **cause** (what was really going on), **how it was found**, the **fix**, and **where** the fix lives. Read this chapter twice: it contains most of the practical lessons of the project.

Legend: 🐞 bug in our code · ⚠️ methodological mistake · 🔍 surprise that needed investigation · 🧭 decision.

Phase 0–1: audit and reference implementation

1. 🔍 The hardware was not what the brief said. *Symptom:* the brief said Ryzen 5 5300U and 500 GB disk. *Found by:* PowerShell `Win32_Processor`, `Get-PSDrive`. *Reality:* Ryzen 5 **7530U**, 30 GB free disk, and only about 0.9–1.3 GB of free RAM at the time. *Fix:* documented as found; a memory guard became mandatory. *Where:* `docs/systems_guide.md`, `benchmarking/runner.py::guard`.

2. 🔍 Importing torch takes 5.5 seconds. *Found by:* timing `import torch`. *Consequence:* running every benchmark in a fresh process costs about 5 s each. *Decision:* process isolation is **optional** (`isolate:` in specs); it is used where the OS peak memory is needed.

3. 🔍 What does attention return for a row with no allowed keys? *Symptom:* a plain softmax over `[-∞, -∞, …]` gives NaN. *Found by:* a 10-line experiment comparing `torch.softmax` with PyTorch SDPA (both CPU backends, bool and float masks). *Reality:* SDPA returns zeros with no NaN gradients. *Fix/decision:* project-wide policy "fully-masked rows → zero output, zero gradient", implemented via `safe_row_max` (−∞ → 0) and `l_safe` (0 → 1). *Where:* `algorithms/base.py`, `naive.py`; tests in `test_base_and_naive.py`, `test_exact_vs_reference.py`.

4. 🔍 Which kernel does PyTorch SDPA actually run on a CPU? *Question:* the docs describe optimized kernels in CUDA terms. *Found by:* the profiler's operator names. *Reality:* `aten::_scaled_dot_product_flash_attention_for_cpu`, PyTorch's own fused CPU kernel. *Decision:* force the backend explicitly (`sdpa_flash_cpu`, `sdpa_math`) and **never call it FlashAttention**. *Where:* `algorithms/sdpa.py`; the docs discrepancy is recorded in `docs/references.md`.

5. 🧭 Citations checked, not remembered. A research subagent verified every paper's title, authors, arXiv ID and the specific claims. Corrections applied: Milakov & Gimelshein compute a single-pass *normalizer*, not a single-pass softmax; FlashAttention's IO bounds use Θ, not Ω; FlashAttention-2 has a single author; the Zoology paper studies gated-convolution models (not linear attention) in its main results; induction heads are not called "associative recall" in their paper. *Where:* `docs/references.md`.

Phase 2–3: tiled attention and numerical validation

6. ⚠️ The first bf16 tolerance accepted almost anything. *Symptom:* everything passed, but the baseline, naive attention computed in bf16, had errors of 0.147 (magnitude 10). *Cause:* bf16 logits in the hundreds are rounded to the nearest ~1, which multiplies softmax weights by up to 1.65. *Found by:* breaking errors down by input magnitude. *Fix:* use a **mixed-precision baseline** (fp32 compute, rounded

output). Naive-in-bf16 now *fails* 14/53 cases, and that is reported as a finding. *Where:* `correctness/lab.py`, `metrics.py`; `test_finding_naive_in_pure_bfloat16_degrades_with_logit_magnitude`.

7. 🔍 sdpa_flash_cpu "failed" fp32 at magnitude 10 (2.04e-5 vs 2e-5). *Temptation:* raise the threshold. *Instead:* the hypothesis "the error scales with logit size for every implementation" was tested. *Evidence:* the naive formula in fp32 had the *identical* error, 2.04e-5; errors were 0.1–1.5 × u·max|S| for all methods. *Fix:* tolerance = max(2e-5, 4·u·max|S|). *Where:* `metrics.py::tolerance_for`; `test_finding_fp32_error_floor_scales_with_logit_magnitude_for_every_implementation`.

8. 🐞 Mark the lse output as non-differentiable. The custom autograd function returns `(out, lse)`. Without `ctx.mark_non_differentiable(lse)`, the backward would receive a gradient for `lse` and had an expensive check for it. *Fix:* mark it non-differentiable and drop the check. *Where:* `tiled_exact.py::_TiledRecomputeFn`.

9. 🐞 An O(L²) statistic hidden inside an O(L) method. local_window's `band_entries` counter built a dense `L × L` band mask. *Fix:* `band_entry_count`, an O(L) formula. *Where:* `local_window.py`.

10. 🐞 Theory overcounted ragged tiles. The tiled formula `tiles × q_tile × kv_tile` overcounts when the last tile is shorter. *Found by:* the scientific test comparing theory with instrumented counts. *Fix:* loop over real tile boundaries. *Where:* `complexity/theory.py`.

11. 🧭 Refactor during a sweep. The online-softmax update was extracted into `online_softmax_update` while the first sweep was running; later workers imported the new version. Same operations, plus one function call per tile; noted, and that sweep's timings were discarded anyway (entry 15).

Phase 4: benchmarking

12. 🐞 Leftover no-op line. A meaningless line (`rec.pop(...) if False else None`) was left in the runner's `finally` block during writing. Removed.

13. ⚠️ The memory guard decided for the whole configuration. Using the forward+backward estimate would have skipped valid forward measurements. *Fix:* decide **per mode**, with status `partial`. Then a second mistake: the scheduler briefly stopped a series after a `partial`; reverted, because forward may still fit at the next length. *Where:* `runner.py::guard`, `scheduler.py`.

14. 🐞 Workers asked PowerShell for the CPU name every time (1–2 s each). *Fix:* the parent passes it through the environment variable `ATTENTION_LAB_CPU_NAME`. *Where:* `env.py`, `runner.py::run_isolated`.

15. 🔍 The timings made no physical sense, and the machine was shared. *Symptom:* identical forward code measured 13.1 vs 34.7 ms; causal slower than non-causal; tiled forward exponent 3.0. *Found by:* `psutil.cpu_percent` showed all 12 logical CPUs busy; listing processes showed other projects' training jobs, servers and tests. *Fix:* a series of changes (entries 16–19); that sweep's timing data was declared contaminated. Its memory data remained valid. *Where:* `docs/results.md` §0.

16. 🐞 The load monitor recorded nothing for short timings. The first sample came after 0.2 s, but fast regions had already finished. *Fix:* prime the counters on entry and always take a final sample on exit. *Where:* `benchmarking/load.py`.

17. 🐞 CPU time per call read 0.0. Windows' `process_time` ticks in about 15.6 ms steps. *Fix:* measure over the whole timed loop and divide by the number of calls. *Where:* `timing.py`.

18. 🧭 **Definitive timing on one thread.** Multi-threaded OpenMP code under contention stalls unevenly. Single-threaded runs gave CPU ≈ wall time and sensible exponents (1.92–2.03). The quiet-wait timeout was cut from 60 s to 20 s, then to 10 s, because the load never went away. *Where:* `experiments/configs/*.yaml`.

19. ⚠️ **"Other users".** Early comments called the background load "other users' jobs". It came from the owner's other projects on the same account. Wording corrected everywhere.

20. ⚠️ **Plots with independent y-axes.** The first memory figure gave each method its own axis, which hid the comparison. *Fix:* added a shared-axis comparison figure with L and L^2 guides. *Where:* `visualization/plots.py::plot_memory_compare`.

Phase 5–6: probes, training and tasks

21. 🔍 **Was the probe unfair to linear attention?** It scored at chance everywhere. *Check:* a sharpness sweep over β. *Result:* it improves with β (L = 16: 6 % → 73 %) but stays far below softmax. The claim was rephrased accordingly. *Where:* `tasks/probes.py` (sharpness experiments).

22. 🐞 **Training output invisible.** Redirected stdout is block-buffered. *Fix:* `python -u`; rely on per-record JSONL flushes.

23. 🔍 **Training was starved.** 4 threads on a saturated CPU got about 0.25 cores. *Fix:* 1 thread by default for `tasks`.

24. ⚠️ **A wrong expectation about step time.** 500 ms per step looked like a bug. *Found by:* profiling. Matrix multiplies dominated; about 2.5 GFLOP per step is real work. *Fix:* expectation corrected; `--batch-size 32` option.

25. 🐞 **`float(loss)` warning** on a tensor requiring grad → `float(loss.detach())`.

26. 🔍 **MQAR v1 plateau at 25–26 % for every backend.** *Cause:* a data-generator shortcut (values drawn with replacement, so "predict the most frequent value" scores 25.7 %). *Fix:* v2 with distinct values; `shortcut_baseline`; `TASK_VERSION`; note in the v1 folder; regression test. *Where:* `tasks/synthetic.py`, Chapter 19.

27. ⚠️ **Concurrent runs garbled a shared log.** Three explorations appended to one file. *Fix:* one diagnostic at a time, one log each. *Where:* `experiments/diagnostics/mqar_v2/`.

28. 🔍 **MQAR v2 plateau at log(n).** Diagnosed with 10 pipeline tests, validated instruments and three diagnostics. Not an implementation bug. *Where:* Chapter 19, `docs/mqar_diagnosis.md`.

29. ⚠️ **A false "success": plain induction was solved positionally.** Fixed offset → 100 %; random offset → 18 %. The earlier conclusion was withdrawn. *Where:* Chapter 19, D2.

30. 🧭 **Use variable-offset induction for the trained comparison**, since the exact model demonstrably learns it (D3). *Where:* `tasks/synthetic.py::induction`, `cli/main.py::cmd_tasks`.

31. 🔍 **block_sparse = exact at L = 32.** The layout covered 528/528 pairs. Reported as "not evidence about sparsity".

32. 🔍 **The distractor collapse is not ambiguity.** Measured: ambiguous targets 30.1 % (normal) vs 36.7 % (distractors), yet accuracy fell from 100 % to 18 %. Reported as brittleness, with a labeled hypothesis.

Phase 7–9: interface, reports and final checks

33. 🐞 **UI crash:** `FileNotFoundError ... meta.json`**.** The run list included non-sweep folders. *Fix:* list only folders whose meta has a spec; probes and correctness now write meta. Streamlit had to be restarted to reload modules. `--headless` added. *Where:* `scheduler.py::list_runs`, `cli/main.py`.

34. 🐞 **Relative spec paths broke outside the project folder.** *Fix:* resolve against the project root, with a clear error listing the available specs.

35. ⚠️ **Untraceable evidence in a draft.** A draft of the results cited 4-thread numbers from an ad-hoc check that was never saved. *Fix:* replaced with numbers from saved runs (smoke run and first scaling run). Rule adopted: **every number must come from a file**.

36. ⚠️ **Arithmetic slips in drafts, caught by re-checking.** "1.0–1.6 %" became 0.95–1.6 %; a causal speed-up of "1.8–4×" became 1.2–2.4×; the tile ratios "1.3–9×" became 1.1–9.2×; "about 3×" became 3.1–3.5×; the library size was 4,451 lines, not about 4,000. Each was found by recomputing from the data before publishing.

37. 🐞 **Test-suite plumbing.** `tests.conftest` import path; the `pytest-asyncio` warning; the double `-q` that hid the summary; the profiler warnings (Chapter 21).

38. ⚠️ **"Tiled code is slow" was too simple.** On one thread it matched or beat naive; with more threads it got slower (4.8× slower at 6 threads than at 1). The limitation text was corrected to the measured, conditional statement.

The ten most important lessons

1. Verify the environment; do not trust a description of it.
2. Decide edge-case policies (empty rows) by checking a trusted implementation, then test them everywhere.
3. A tolerance is a model of error. Derive it, and change it only when the model is shown to be wrong.
4. When a check fails, form a hypothesis about the cause and test it before changing anything.
5. Measure the conditions of your measurements (background load), and say when they are bad.
6. Validate every instrument on a case with a known answer.
7. Identical results from different mechanisms point at the task or the data, not the mechanisms.
8. Rule out shortcuts before claiming a model learned something.
9. Every number must be traceable to a saved file. Recompute before you publish.
10. Know when an investigation has answered the question your project needs, and stop there deliberately.

Chapter 23 — Limitations, open questions and what to do next

> **What you will learn:** exactly what this project cannot tell you, which questions remain open, and the most valuable next experiments, in priority order.

23.1 What the results do not establish

Hardware scope. Every measurement is from one CPU (Ryzen 5 7530U, AVX2, 16 MB L3) running PyTorch 2.10. GPUs have different memory hierarchies, launch overheads and bf16 hardware, so crossover lengths, dtype speeds and the fused vs unfused ratios do **not** transfer. What does transfer: exactness, the storage-scaling laws, the store-vs-recompute logic and the numerical findings about accumulator precision.

Measurement conditions. The machine was shared and often fully loaded. Timings carry measured background load; the same configuration varied up to 2× between sweeps an hour apart. Ratios measured within one sweep are the reliable quantities. Run-to-run variance was not estimated by repeating whole sweeps.

Memory instruments. The tensor peak sees only PyTorch-allocator memory, not MKL/oneDNN/OpenMP scratch. The process peak is coarse and cumulative. Memory *traffic* was reasoned about but never measured (that needs hardware performance counters).

Implementations. The Python tiled kernels illustrate the algorithm, not peak performance. The block-sparse implementation uses gather/pad, and a dedicated kernel would have different constants. Only one linear-attention feature map was tested. Approximate methods support causal and padding masks but not arbitrary dense masks. There is no KV-cache decoding benchmark and no attention dropout.

Quality experiments. Synthetic tasks with tiny models, single seeds and short CPU budgets: evidence about *mechanisms*, not about language-model quality. Small accuracy differences are within seed noise. At $L = 32$, the block-sparse layout was dense.

23.2 Open questions

1. **MQAR binding.** Why does the model learn content-based induction in 500 steps but not MQAR binding in 3,000? Candidates: sparser supervision; the "uniform over context values" solution acting as an attractor; budget.
2. **Distractor brittleness.** Do the induction models really locate the segment by its noise boundary? An attention probe on the trained checkpoints could test this directly.
3. **Linear attention's better out-of-distribution behaviour** in Task C/D: real, or seed noise? It needs several seeds.
4. **Where exactly the CPU time goes** in naive vs fused attention: needs hardware counters (cache misses, bandwidth).

23.3 The most valuable next experiments

1. **Multiple seeds for the trained tasks** (3–5 seeds), to put error bars on Chapter 18.
2. **A clean timing re-run on an idle machine**, with whole-sweep repetition to measure run-to-run variance.
3. **The MQAR follow-ups** listed in Chapter 19: denser supervision, a curriculum, and longer training with circuit probes.
4. **A GPU run** with the device-aware additions of Chapter 13 §13.8, if hardware becomes available, to compare crossover lengths across hardware.
5. **A compiled tiled kernel** (`torch.compile`, or a small C++ extension), to separate "algorithm" from "Python dispatch" in the tiled-vs-fused gap.

Key points to remember

- CPU-only, shared machine, single-seed tiny models: read every result within those limits.
- Within-sweep ratios and memory measurements are the most trustworthy results.
- Open: MQAR binding, distractor brittleness, linear attention's OOD hint, hardware-counter profiles.
- Next: seeds, a clean re-run, MQAR follow-ups, GPU, a compiled tiled kernel.

Chapter 24 — Glossary

Accumulator (dtype). The number format used for running sums inside an algorithm. Using fp32 accumulators with bf16 inputs was the key to accuracy (Chapter 11).

Activation. An intermediate value computed in a network's forward pass; training keeps many of them for backward.

AdamW. The optimizer used for training: adaptive per-parameter step sizes plus weight decay.

Additive mask. A table of numbers added to the attention scores; `-∞` removes a pair.

Approximate attention. Any method that does not compute exact softmax attention: restricted (local, block sparse) or a different function (linear).

Autograd. PyTorch's automatic differentiation: it records operations in the forward pass and computes gradients backward.

AVX2. The vector instruction set of this CPU (8 fp32 numbers per instruction). No native bf16 arithmetic.

Backward pass. Computing gradients of the loss with respect to all parameters and inputs.

Batch (B). Number of independent sequences processed together.

bf16 (bfloat16). 16-bit floating point with fp32's range but only 8 significant bits ($u \approx 3.9 \times 10^{-3}$).

Bitwise equality. Identical bit patterns, which is stricter than numerical equivalence.

Block-sparse attention. Exact softmax restricted to a layout of query-block × key-block pairs.

Cache (L1/L2/L3). Small, fast memory between the CPU and RAM. L3 here is 16 MB.

Causal mask. Query `i` may attend only keys `j ≤ i` (top-left aligned).

Chunk. A group of consecutive positions processed together (local window, linear attention).

Contended. A timing record during which other processes used more than one logical CPU.

Crossover length. The length beyond which one method becomes faster than another.

Cross-entropy loss. `-log(probability of the correct answer)`. Uniform guessing among n options gives log n.

Deviation. The difference between an approximate method's output and full softmax attention. It is not called error.

Dispatch overhead. The fixed cost (microseconds) of issuing one PyTorch operation from Python.

dtype. Data type, such as fp64, fp32, bf16.

Embedding. A learned table mapping token IDs to vectors.

Exact attention. Computes $\text{softmax}(QK^T/\sqrt{d} + M)V$ up to floating-point rounding.

Exponent (fitted). The slope of log(cost) vs log(L): about 2 means quadratic, about 1 means linear.

FlashAttention. The fused, IO-aware GPU attention algorithm and library by Dao et al. (2022). Not used here; its algorithm is reproduced in Python, and PyTorch's CPU flash kernel applies the same ideas.

FLOP. Floating-point operation; one multiply-add counts as 2.

fp32 / fp64. 32-bit and 64-bit floating point ($u \approx 6 \times 10^{-8}$ and 1.1×10^{-16}).

Fully-masked row. A query with no allowed keys; the project outputs 0 by policy.

Fused kernel. One compiled function doing several steps on data held in cache, without writing intermediates to RAM.

Gradient. The vector of derivatives of the loss with respect to parameters.

gradcheck. PyTorch's finite-difference check of a backward pass.

Head (H). One of several independent attention computations in parallel; head dimension `d`.

Induction (task / head). Find the earlier occurrence of the current token and copy what followed it. Variable-offset induction requires content matching.

Inference mode. Running a model without recording gradients.

IQR. Inter-quartile range: the spread between the 25th and 75th percentiles.

Isolated worker. A fresh Python process running one benchmark configuration.

Key / Query / Value. The three projections of each token: what it offers to be matched, what it looks for, and what it hands over.

Layout. The block-level allowed/forbidden table of block-sparse attention.

Linear attention. Attention with weights $\varphi(q)\cdot\varphi(k)$ (here $\varphi = \text{elu} + 1$), computable in $O(L \cdot d^2)$.

Load monitor. The project's background sampler of other processes' CPU use.

Local-window attention. Exact softmax restricted to keys within w positions of the query.

Logit / score. The scaled dot product $q \cdot k/\sqrt{d}$ before softmax.

Log-sum-exp (lse). $\log \Sigma \exp(s_j)$ per row. Stored by memory-efficient backward passes.

MaskSpec. The project class defining mask semantics and serving masks per tile.

Materialize. To actually create a tensor in memory (for example, the full $L \times L$ score matrix).

Memory-bandwidth-bound. Limited by data movement rather than arithmetic.

Memory guard. Refuses benchmark modes whose estimated memory exceeds the budget, and records the refusal.

MQAR. Multi-query associative recall: retrieve values bound to keys.

Numerical equivalence. Equal within a justified tolerance.

Online softmax. Computing a softmax-weighted sum incrementally by rescaling running sums when the max changes.

OpenMP. The threading library PyTorch uses on CPUs.

Padding mask (key_padding_mask). Marks real (True) vs padding (False) keys.

Probe. A hand-constructed input testing whether one attention operation can retrieve a known target.

Profiler. PyTorch's tool recording operators and memory events.

Recompute (backward). Recomputing score tiles in the backward pass instead of storing them.

Registry. The table of all attention methods with their metadata.

RoPE. Rotary position embedding; it makes attention depend on relative positions.

SDPA. PyTorch's `scaled_dot_product_attention`, with several backends.

Shortcut baseline. The best accuracy achievable without the intended skill (for example, without key–value binding).

SMT. Simultaneous multithreading: 2 hardware threads per core.

Softmax. $\exp(x_i)/\Sigma \exp(x_j)$: positive weights summing to 1.

Swap experiment. Evaluating a trained model with a different attention backend substituted, with the weights unchanged.

Tensor peak. Peak bytes allocated through PyTorch's CPU allocator during one call (profiler replay).

Tile. A small block of queries × keys processed at once.

Tolerance. The maximum accepted difference, derived from the dtype and the operation.

Unit roundoff (u). The maximum relative error of one rounding in a format.

Working set (peak). The OS's high-water mark of a process's resident memory.

Chapter 25 — Self-test: know this project by heart

Answer each question aloud, then check the answer and the chapter. Repeat until you get them all right without looking.

Part A — The project

1. **What is the project's central question?** How attention algorithms trade off complexity, memory traffic, numerical accuracy and practical speed as sequence length grows. *(Ch. 1)*
2. **Name the eight implementations and their families.** naive, sdpa_math, sdpa_flash_cpu, tiled, tiled_recompute (exact); local_window, block_sparse (restricted softmax); linear_elu (different function). *(Ch. 1, 6)*
3. **What machine was used, and what was special about it?** Ryzen 5 7530U, 6C/12T, AVX2, 16 MB L3, 16 GB RAM, no GPU. Shared with other projects' workloads (5–12 logical CPUs busy) and often under 1 GB of free RAM. *(Ch. 1, 6, 13)*
4. **Why is there a `lab.py` launcher?** So the lab runs without installing into the system Python; it adds `src/` to the path. *(Ch. 6)*

Part B — The mathematics

5. **Write the attention formula.** $\text{softmax}(QK^T/\sqrt{d} + M)V$. *(Ch. 4)*
6. **Why divide by √d?** The dot product of random unit-variance vectors has variance d; scaling restores unit variance and avoids saturated softmax with vanishing gradients. *(Ch. 4)*
7. **Why is attention quadratic, and in what?** QK^T is $L \times L$. Time is always $O(L^2 d)$ for exact attention; naive memory is $O(L^2)$, but tiling makes memory $O(L)$. *(Ch. 5)*
8. **How many matmul FLOPs does exact attention need per head?** About $4 \cdot L^2 \cdot d$. *(Ch. 5)*
9. **What does a training loss of log 4 mean?** Uniform guessing among 4 options. *(Ch. 2, 19)*

Part C — Algorithms

10. **State the online-softmax update.** m_new = max(m, max S); α = exp(m − m_new); P = exp(S − m_new); l = α·l + ΣP; acc = α·acc + P·V; finally out = acc/l, lse = m + log l. *(Ch. 8)*
11. **Online softmax on scores [1,3,2,5] and values [10,20,30,40], tiles of 2?** 36.880566, the same as the direct softmax. *(Ch. 8)*
12. **Why doesn't the tiled method with autograd save training memory?** Autograd keeps every tile's P alive, which adds up to $O(L^2)$. *(Ch. 8)*
13. **What does the recompute backward store, and how does it get P back?** Q, K, V, O and lse; P = exp(S − lse), with S recomputed per tile. *(Ch. 8)*
14. **What is D in the backward pass, and why does it matter?** D = rowsum(dO ∘ O) = Σ P·dP. It replaces a full row of P in the softmax Jacobian. *(Ch. 8)*
15. **What does the policy say about fully-masked rows, and where did it come from?** Zero output and zero gradient, matching what PyTorch's CPU SDPA was observed to do. *(Ch. 7)*
16. **Is `sdpa_flash_cpu` FlashAttention?** No: it is PyTorch's own fused CPU kernel `_scaled_dot_product_flash_attention_for_cpu`, identified with the profiler. *(Ch. 9)*
17. **How does local_window avoid the L^2 table?** Query chunks plus overlapping key windows via `unfold`, then a band mask: $O(L \cdot (C + 2w))$ scores. *(Ch. 10)*

18. **Why does block_sparse compute global rows separately?** A row that sees every block would force padding every row to full width. *(Ch. 10)*
19. **Why is linear attention O(L) non-causal?** Associativity: $\varphi(Q)(\varphi(K)^T V)$ shares one d × d matrix across queries. *(Ch. 10)*
20. **Is linear attention an approximation of softmax?** No. elu + 1 defines a different kernel. *(Ch. 10)*

Part D — Measurement

21. **What is the reference in the correctness lab?** Naive attention in fp64 on the *same rounded* inputs. *(Ch. 11)*
22. **What are the tolerances?** fp64 1e-12; fp32 2e-5 or $4 \cdot u \cdot \max|S|$; bf16 2× a mixed-precision baseline + slack. *(Ch. 11)*
23. **Tell the bf16-baseline story.** Naive-in-bf16 had up to 15 % error at magnitude 10 (38 % at 30), so "2× it" accepted anything. The baseline was switched to mixed precision; naive-bf16 now fails 14/53, reported as a finding. *(Ch. 11, 22)*
24. **Tell the fp32 "failure" story.** Flash at magnitude 10 had 2.04e-5 > 2e-5. The naive formula in fp32 had the same error; errors scale with $u \cdot \max|S|$ for every method. A logit term was added to the tolerance. *(Ch. 11)*
25. **Name the three memory instruments.** Tensor peak (profiler allocator replay, validated at exactly 15.0 MiB), process peak Δ (OS working set, isolated workers), theory (labeled). *(Ch. 13)*
26. **How was background load handled?** A load monitor recorded other processes' CPU use; CPU time was recorded; definitive sweeps ran on 1 thread; contaminated timings were discarded. *(Ch. 13)*
27. **Why median and not minimum?** The median is robust and typical; the minimum is the luckiest run. *(Ch. 13)*

Part E — Results

28. **Naive vs flash forward memory at L = 2048?** 194.1 MiB vs 4.3 MiB. *(Ch. 14)*
29. **Training-memory exponents of tiled-autograd vs tiled-recompute?** 1.72 vs 0.75. *(Ch. 14)*
30. **Time exponents of the exact methods?** All about 1.9–2.1 (autograd-tiled fwd+bwd 2.52). *(Ch. 14)*
31. **Flash vs naive forward on one thread at L = 4096?** 516 vs 1,540 ms, about 3×. *(Ch. 14)*
32. **Recompute vs autograd-tiled fwd+bwd at 2048?** 980 vs 1,847 ms: recompute is faster despite 25 % more FLOPs. *(Ch. 14)*
33. **Where do approximate methods overtake the fused kernel (non-causal forward)?** linear_elu at every L; local_window from about 2,048; block_sparse clearly from 4,096. *(Ch. 15)*
34. **Do approximate methods use less memory than the fused exact kernel?** No: 16–27× *more* at L = 4096. They save memory only relative to materializing methods. *(Ch. 15)*
35. **What happens to the Python tiled code with more threads?** It gets slower: 4.8× slower at 6 threads than at 1. *(Ch. 15)*
36. **What did the distance probe show for local_window w = 64?** 100 % up to distance 64, 0 % from 65. *(Ch. 16)*
37. **Why did block_sparse succeed at distance 1023?** The target was in the global block 0. *(Ch. 16)*
38. **Which methods resisted decoys, and why?** local_window and block_sparse: the decoys were outside their pattern. *(Ch. 16)*
39. **What did the β-sweep show about linear attention?** It improves with scale (L = 16: 6 % → 73 %) but stays far below softmax, because the sharpness is polynomial, not exponential. *(Ch. 16)*

40. **Task A: what happened when a window was swapped into the exact model at L = 256?** 86.9 % → 100 %. *(Ch. 18)*
41. **How did local_window score 49 % on induction with offsets beyond its window?** Two layers relay 16 tokens: 99 % within 8–16 back, 6 % beyond. *(Ch. 18)*
42. **Why is block_sparse at L = 32 not sparsity evidence?** Its layout covered 528/528 causal pairs. *(Ch. 18)*
43. **Is the distractor-gap collapse caused by ambiguity?** No: ambiguous targets went 30.1 % → 36.7 %, while accuracy went 100 % → 18 %. *(Ch. 18)*

Part F — The MQAR story

44. **What caused the v1 plateau?** Values drawn with replacement; "predict the most frequent value" scored 25.7 %. *(Ch. 19)*
45. **What did D1 (overfit) show?** 100 % train, 20 % held-out: memorization, no circuit. *(Ch. 19)*
46. **What did D2 show?** Plain induction was solved positionally: 100 % at a fixed offset, 18 % at random offsets. *(Ch. 19)*
47. **What did D3 show?** Content-based induction is learnable: 100 % by step 1000. *(Ch. 19)*
48. **Why was the MQAR investigation stopped?** It had established that the attention code was not at fault, which is the project's question; the remaining causes are a separate research topic. *(Ch. 19)*

Part G — Operating

49. **How do you reproduce everything?** `python lab.py reproduce` (or `--dry-run` to list the steps). *(Ch. 20)*
50. **How do you add a new attention method?** Write a function with the shared signature, add a registry entry, add theory formulas, add tests. *(Ch. 20)*

Chapter 26 — File-by-file index

Every file in the project, what it is for, and where the book discusses it.

Top level

File	Purpose	Chapter
`lab.py`	launcher: adds `src/` to the path and runs the CLI	6, 20
`pyproject.toml`	package metadata; pytest settings (paths, markers, warning filters, `-p no:asyncio`)	6, 21
`README.md`	quick start, command list, documentation map	20
`.gitignore`	excludes caches, result/report folders and checkpoints from version control	6
`.claude/launch.json`	launch configuration used to start the UI for in-app testing	20

`src/attention_lab/algorithms/`

File	Purpose	Chapter
`base.py`	`validate_qkv`, `default_scale`, `MaskSpec`, `apply_tile_mask`, `safe_row_max`; mask semantics and the fully-masked-row policy	7
`naive.py`	reference attention; `attention_probabilities`	7
`tiled_exact.py`	`online_softmax_update`, `tiled_forward_kernel`, `tiled_attention`, `_TiledRecomputeFn`, `tiled_attention_recompute`	8
`sdpa.py`	adapters forcing SDPA's flash-CPU or math backend; mask translation	9
`local_window.py`	sliding-window attention (chunks + unfold); `band_mask`, `band_entry_count`	10
`block_sparse.py`	`make_layout`, `layout_to_element_mask`, `block_sparse_attention` (gather)	10
`linear.py`	`linear_attention` (associative and chunked causal), `linear_attention_quadratic_reference`	10
`registry.py`	`AttentionMethod`, `REGISTRY`, `EXACT_METHODS`, `get_method`, `run_attention`	6

Other library modules

File	Purpose	Chapter
`correctness/metrics.py`	`compare`, the tolerance policy (`tolerance_for`, unit roundoffs, logit factor)	11
`correctness/lab.py`	`CorrectnessCase`, `evaluate_case`, `default_cases`, `run_suite`, `unstable_softmax_demo`	11
`complexity/theory.py`	`theoretical_cost`, `estimated_peak_bytes`	12
`benchmarking/env.py`	environment capture	13
`benchmarking/timing.py`	`time_callable`, `summarize`, CPU time per call	13
`benchmarking/memory.py`	`tensor_peak_bytes` (profiler replay), `ProcessPeak`	13
`benchmarking/load.py`	`LoadMonitor`, `snapshot` (background load)	13
`benchmarking/runner.py`	`BenchConfig`, `guard`, `run_one`, `run_isolated`	13
`benchmarking/worker.py`	isolated-process entry point	13
`benchmarking/scheduler.py`	`expand`, `run_sweep`, `wait_for_quiet`, `load_results`, `list_runs`	13

File	Purpose	Chapter
`tasks/probes.py`	`make_probe`, `run_probe`, `probe_suite`	16
`tasks/synthetic.py`	`local_copy`, `mqar`, `induction`, `fixed_eval_set`, `accuracy_by_distance`, `shortcut_baseline`, `TASK_VERSION`	17, 19
`models/transformer.py`	`ModelConfig`, `TinyTransformer`, RoPE, sinusoidal encodings, `lm_loss`	17
`training/train.py`	`TrainConfig`, `train`, `evaluate`, `run_task_experiment` (swap evals, checkpoints)	17
`diagnostics/recall.py`	instrumented training: gradient norms, update ratios, prediction categories, circuit probes	19
`reporting/report.py`	`results_frame`, `fit_exponent`, `exponent_table`, `summary_table`, `build_report`	13, 20
`reporting/tasks_report.py`	probe and task reports	16, 18
`visualization/plots.py`	runtime, memory (small multiples and shared-axis comparison), metric and bar plots	14, 15
`cli/main.py`	every command, including `reproduce`	20
`ui/app.py`	Streamlit interface (9 tabs)	20

`scripts/`

File	Purpose	Chapter
`diagnose_recall.py`	one MQAR/induction diagnostic per invocation	19
`summarize_diag.py`	print selected columns of a diagnostic log	19
`build_book.py`	assemble this book into one Markdown, HTML and PDF file	front matter

`experiments/`

Path	Contents
`configs/*.yaml`	`smoke`, `scaling_exact`, `scaling_exact_timing`, `scaling_approx`, `tile_sizes`, `threads`, `shapes_dtypes`
`results/20261011-191916_smoke`	smoke benchmark
`results/20261011-191952_scaling_exact`	E1 memory (timing contaminated)
`results/20261011-194506_correctness`	C1 full correctness (795 comparisons)
`results/20261011-195411_probes`	first probe run (before the sharpness sweep)
`results/20261011-195716_probes`	Q1 probes (used in the book)
`results/20261011-200406_tasks_quick`	quick tasks (MQAR v1, with a NOTE about the shortcut)
`results/20261011-205830_tasks`	Q2 trained tasks (A and B/C/D)
`results/20261011-210946_scaling_exact_timing`	E1b timing, 1 thread
`results/20261011-212017_scaling_approx`	E2 approximate methods
`results/20261011-215718_tile_sizes`	E3 tile sizes
`results/20261011-221221_threads`	E4 threads

Path	Contents
`results/…_shapes_dtypes`	E5 shapes and dtypes
`diagnostics/mqar_v2/`	exploration log snapshot, D1–D3 logs, exploration scripts
`reports/<run>/`	generated summary.md, CSVs and figures for each run
`checkpoints/`	trained model weights (local only)

`docs/`

`results.md` (all findings with run names), `attention_math.md`, `online_softmax.md`, `complexity_analysis.md`, `systems_guide.md`, `numerical_guide.md`, `benchmark_methodology.md`, `research_guide.md`, `interview_guide.md`, `mqar_diagnosis.md`, `limitations.md`, `references.md`, `architecture.md`, and `book/` (this book).